



KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Approved by AICET, New Delhi & Affiliated to Anna University, Chennai
Accredited by NBA (CSE, ECE, EEE, Mech), NAAC,
Recognized by UGC with 2(f) & 12(B) An ISO 9001:2005 Certified Institution
Namakkal - Trichy Main Road,
Tholurpatti (post), Thottiam, Trichy – 621215, Tamil Nadu

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

COURSE MATERIAL

Course Name with Code : 24AD402/BIGDATA ANALYTICS

Course Faculty : E.MATHUMITHA

Batch : 2024 – 2028

Year / Semester / Section : II / IV

S.NO	TOPICS COVERED	PAGE NO
UNIT – I INTRODUCTION TO BIGDATA		
1	Introduction to Big Data and Hadoop	1
2	Types of Digital Data	1
3	Introduction to Big Data	1
4	Big Data Analytics	8
5	History of Hadoop	8
6	Analyzing Data With Hadoop	10
7	Hadoop Streaming	14
8	Hadoop Eco System	17
9	Application of Big Data	22
UNIT – II HADOOP DISTRIBUTED FILE SYSTEM		
10	HDFS (Hadoop Distributed File System)	24
11	Design of HDFS – HDFS Concepts	28
12	Command Line Interface	31
13	Hadoop file system interfaces	32
14	Data flow	33

15	Data Ingest with Flume and Sqoop	37
16	Hadoop I/O: Compression	42
17	Serialization	43
18	Avro and File Based Data structures.	44
UNIT – III MAPREDUCE PROGRAMMING		
19	Map Reduce: Anatomy of a MapReduce	56
20	Job Run	63
21	Failures	68
22	Job Scheduling	69
23	Shuffle and Sort	70
24	Task Execution	72
25	MapReduce Type and Formats	73
26	MapReduce Features	78
27	Composing MapReduce calculation	79
UNIT – IV PIG AND HIVE		
28	Hadoop Ecosystem: Introduction to PIG, Execution Modes of Pig	82
29	Comparison of Pig with Databases	83
30	Grunt, Pig Latin ,User Defined Functions	83
31	Data Processing operators, Hbase	85
32	Data model and implementations, Hbase clients ,Hbase examples	86
33	Hive,data types and file formats	98
34	HiveQL data definition	103
35	HiveQL data manipulation	105
36	HiveQL queries.	109
UNIT – V DATA STREAMS		
37	Mining Data Streams	112
38	Stream Data Model	113
39	Sampling Data in the Stream	115
40	Filtering Streams	116
41	Counting Distance Elements in a Stream	117
42	Estimating Moments	121

43	Counting Ones in Window	123
44	Decaying Windows	1291
45	Case Studies - Real Time Sentiment Analysis, Stock Market Predictions	133
46	Using Graph Analytics for Big Data: Graph Analytics.	138
47	Big Data Analytics with Big R.	141

Course Faculty Signature

HoD

UNIT I

BIG DATA AND HADOOP

Introduction to Big Data and Hadoop: Types of Digital Data – Introduction to Big Data – Big Data Analytics – History of Hadoop – Analyzing Data with Hadoop – Hadoop Streaming – Hadoop Eco System – Applications of Big Data: marketing – fraud detection – risk assessment – credit risk management – healthcare – medicine – advertising.

1. Introduction to Big Data and Hadoop: Types of Digital Data

Data refers to raw facts, figures, or information that can be processed, analyzed, and interpreted to derive meaning or insights. It can take various forms and be collected from various sources. Data by itself doesn't carry any context or meaning until it's processed or analyzed.

Types of Data:

1. **Qualitative Data:** Describes attributes or qualities (e.g., color, texture, opinion).
2. **Quantitative Data:** Numerical data that can be measured and quantified (e.g., height, weight, temperature).

Examples of Data:

- **Raw Data:** A list of numbers, names, or measurements (e.g., "John, 25, 175 cm").
- **Unprocessed Facts:** Pieces of information that may not yet have context or meaning (e.g., "10 cars in a parking lot").
- **Sensory Inputs:** Data gathered from sensors (e.g., temperature readings from a thermometer).

Digital Data:

Digital Data refers to data that is represented in a binary format (0s and 1s) and is processed or stored electronically using computers, software, or digital systems. It contrasts with analog data, which is continuous and varies in a smooth, uninterrupted flow. Digital data is discrete and can be easily stored, processed, and transmitted by digital devices.

Types of Digital Data:

1. **Structured Data:** Highly organized, easily searchable, and stored in predefined formats (e.g., databases, spreadsheets).
2. **Semi-Structured Data:** Contains some structure but does not fully adhere to rigid formats (e.g., XML, JSON files).

3. **Unstructured Data:** Data that lacks a predefined format and is more difficult to categorize or analyze (e.g., text documents, videos, audio files).

Examples of Digital Data:

- **Text Files:** Written information in formats like .txt, .docx, etc.
- **Images and Videos:** Data in formats like .jpeg, .png, .mp4.
- **Databases:** Information stored in relational databases such as SQL.
- **Web Data:** Data from websites, including HTML, JavaScript, and user-generated content like comments and posts.

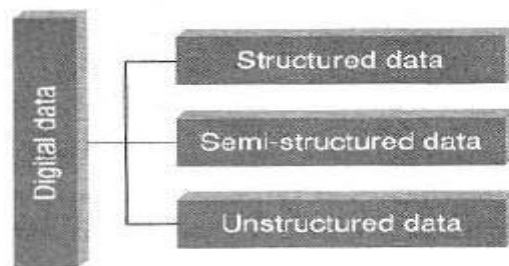


Figure 1.1 Classification of digital data.

1.1 Structured Data:

Ease of working with Structured Data:

Structured data provides the ease of working with it. The ease is with respect to the following:

1. **Insert/update/delete:** The Data Manipulation Language (DML) operations provide the required ease with data input, storage, access, process, analysis, etc.
2. **Security:** How does one ensure the security of information? There are available strong encryption and tokenization solutions to warrant the security of information throughout its lifecycle. Organizations are able to retain control and maintain compliance adherence by ensuring that only authorized individuals are able to decrypt and view sensitive information.
3. **Indexing:** An index is a data structure that speeds up the data retrieval operations (primarily the SELECT DML statement) at the cost of additional writes and storage space, but the benefits that ensue in search operation are worth the additional writes and storage space.

4. **Scalability:** The storage and processing capabilities of the traditional RDBMS can be easily scaled up by increasing the horsepower of the database server (increasing the primary and secondary or peripheral storage capacity, processing capacity of the processor, etc.).
5. **Transaction processing:** RDBMS has support for Atomicity, Consistency, Isolation, and Durability (ACID) properties of transaction. Given next is a quick explanation of the ACID properties:
 - **Atomicity:** A transaction is atomic, meaning that either it happens in its entirety or none of it at all.
 - **Consistency:** The database moves from one consistent state to another consistent state. In other words, if the same piece of information is stored at two or more places, they are in complete agreement.
 - **Isolation:** The resource allocation to the transaction happens such that the transaction gets the impression that it is the only transaction happening in isolation.
 - **Durability:** All changes made to the database during a transaction are permanent and that accounts for the durability of the transaction.

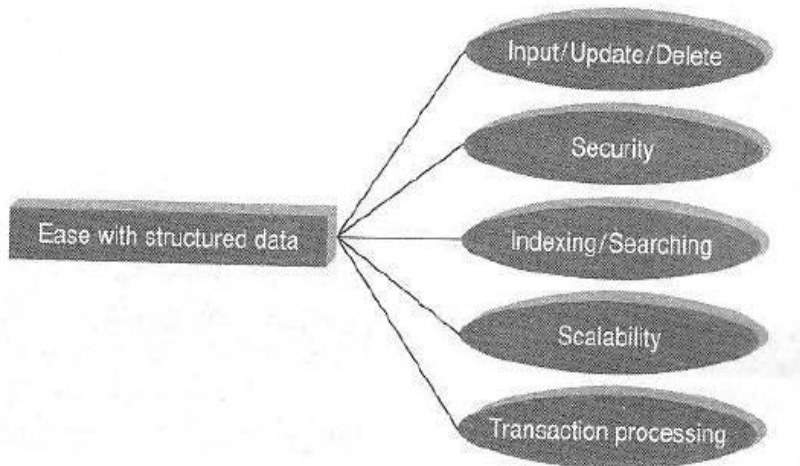


Figure 1.2 Ease of working with structured data

1.2 Semi-Structured Data:

Semi-structured data is also referred to as self-describing structure. It has the following features:

1. It does not conform to the data models that one typically associates with relational databases or any other form of data tables.
2. It uses tags to segregate semantic elements.

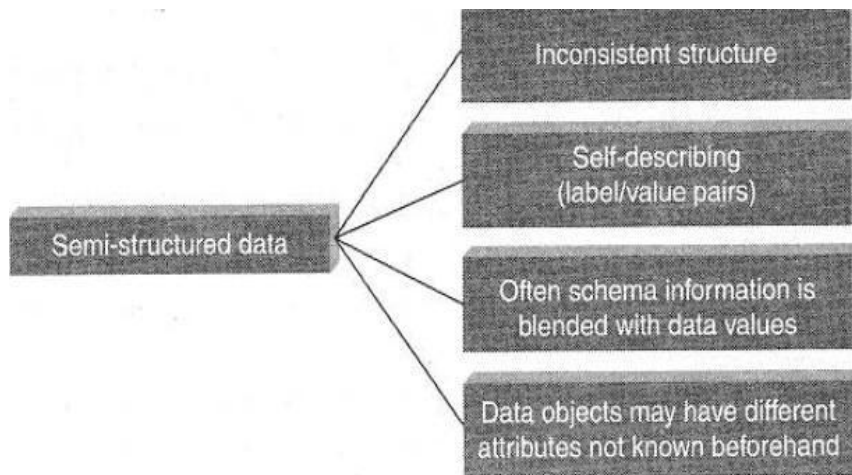


Figure 1.3 characteristics of semi-structured data

3. Tags are also used to enforce hierarchies of records and fields within data.
4. There is no separation between the data and the schema. The amount of structure used is dictated by the purpose at hand.
5. In semi-structured data, entities belonging to the same class and also grouped together need not necessarily have the same set of attributes. And if at all, they have the same set of attributes, the order of attributes may not be similar and for all practical purposes, it is not important as well.

1.4. Introduction to Big Data

1.3 .1 Characteristics of Data

Data has three key characteristics:

1. **Composition:** The composition of data deals with the structure of data, that is, the sources of data, the granularity, the types, and the nature of data as to whether it is static or real-time streaming.
2. **Condition:** The condition of data deals with the state of data, that is, "Can one use this data as is for analysis?" or "Does it require cleansing for further enhancement and enrichment?"

3. **Context:** The context of data deals with "Where has this data been generated?" "Why was this data generated?" "How sensitive is this data?" "What are the events associated with this data?" and so on.

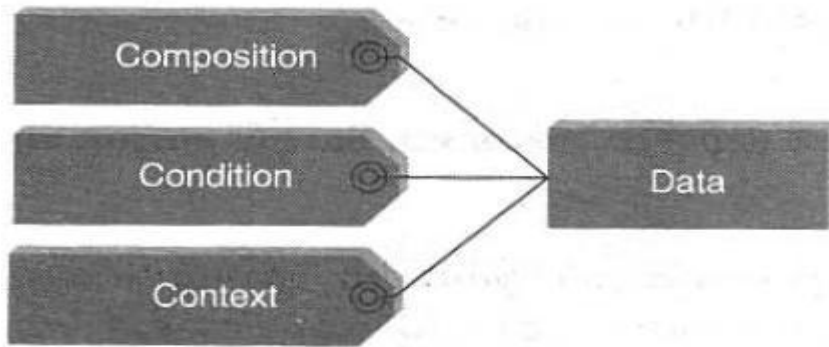


Figure 1.7 characteristics of data

1.4.2 Definition of Big data

According to Gartner, the definition of Big Data-

"Big data" is high-volume, velocity, and variety information assets that demand cost-effective, innovative forms of information processing for enhanced insight and decision making."

This definition clearly answers the "What is Big Data?" question – Big Data refers to complex and large data sets that have to be processed and analyzed to uncover valuable information that can benefit businesses and organizations.

However, there are certain basic tenets of Big Data that will make it even simpler to answer what is Big Data:

- It refers to a massive amount of data that keeps on growing exponentially with time.
- It is so voluminous that it cannot be processed or analyzed using conventional data processing techniques.
- It includes data mining, data storage, data analysis, data sharing, and data visualization.
- The term is an all-comprehensive one including data, data frameworks, along with the tools and techniques used to process and analyze the data.

1.4.3 Characteristics of Big Data

Big Data was defined by the “3 Vs” but now there are “6 Vs” of Big Data which are also termed as the characteristics of Big Data. These six characteristics help define what Big Data really means and how it can be managed effectively.

1. Volume
2. Velocity
3. Variety
4. Veracity
5. Value
6. Variability

1. Volume – Scale of Data

- The name ‘Big Data’ itself is related to a size which is enormous. Volume is a huge amount of data. To determine the value of data, size of data plays a very crucial role.
- If the volume of data is very large, then it is actually considered as a ‘Big Data’.

This means whether a particular data can actually be considered as a Big Data or not, is dependent upon the volume of data.

***Example:** In the year 2016, the estimated global mobile traffic was 6.2 Exabytes (6.2 billion GB) per month. Also, by the year 2020 we will have almost 40000 Exabytes of data.*

2. Velocity – Speed of Data

- Velocity refers to the high speed of accumulation of data.
- In Big Data velocity data flows in from sources like machines, networks, social media, mobile phones etc. There is a massive and continuous flow of data.
- This determines the potential of data that how fast the data is generated and processed to meet the demands. Sampling data can help in dealing with the issue like ‘velocity’.

***Example:** There are more than 3.5 billion searches per day are made on Google. Also, Facebook users are increasing by 22%(Approx.) year by year.*

3. Variety – Diversity of Data

- It refers to nature of data that is structured, semi-structured and unstructured data. It also refers to heterogeneous sources.
- It can be structured, semi-structured and unstructured.
- **sStructured data:** This data is basically an organized data. It generally refers to data that has defined the length and format of data.
- **Semi- Structured data:** This data is basically a semi-organised data. It is

generally a form of data that do not conform to the formal structure of data. Log files are the examples of this type of data.

- **Unstructured data:** This data basically refers to unorganized data. It generally refers to data that doesn't fit neatly into the traditional row and column structure of the relational database. Texts, pictures, videos etc. are the examples of unstructured data which can't be stored in the form of rows and columns.

4. Veracity – Trustworthiness of Data

- It refers to inconsistencies and uncertainty in data, that is data which is available can sometimes get messy and quality and accuracy are difficult to control.
- Big Data is also variable because of the multitude of data dimensions resulting from multiple disparate data types and sources.

Example: Data in bulk could create confusion whereas less amount of data could convey half or Incomplete Information.

5. Value – Usefulness of Data

- After having the 4 V's into account there comes one more V which stands for Value.
- The bulk of Data having no Value is of no good to the company, unless you turn it into something useful.
- Data in itself is of no use or importance but it needs to be converted into something valuable to extract Information. Hence, you can state that Value is the most important V of all the 6V's.

6. Variability – Inconsistency of Data

- Variability refers to the changes in data flow rates, formats, or meanings over time. For instance, during sales events or trending news, data may spike drastically.
- Additionally, words or behavior may have different meanings in different contexts, making interpretation harder.

Example: if you are eating same ice-cream daily and the taste just keep changing. or A tweet saying "bad" might be negative, or in slang, it could mean "good." This inconsistency must be addressed when analyzing large, dynamic datasets.

1.4.4 BIG DATA ANALYTICS

Definition of Big Data Analytics:

- Big Data analytics is a process used to extract meaningful insights, such as hidden patterns, unknown correlations, market trends, and customer preferences.
- Big Data analytics provides various advantages—it can be used for better decision making, preventing fraudulent activities, among other things.

Why is big data analytics important?

- In today's world, Big Data analytics is fuelling everything we do online—in every industry.
- Take the music streaming platform Spotify for example. The company has nearly 96 million users that generate a tremendous amount of data every day. Through this information, the cloud-based platform automatically generates suggested songs—through a smart recommendation engine—based on likes, shares, search history, and more. What enables this is the techniques, tools, and frameworks that are a result of Big Data analytics.

1.4.5 History of Hadoop

- Hadoop is an open-source software framework for storing and processing large datasets ranging in size from gigabytes to petabytes.
- Hadoop was developed at the Apache Software Foundation.
- In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data.
- In January 2008, Hadoop confirmed its success by becoming the top-level project at Apache.
- By this time, many other companies like Last.fm, Facebook, and the New York Times started using Hadoop.

Hadoop Defeated supercomputers

- In April 2008, Hadoop defeated supercomputers and became the fastest system on the planet by sorting an entire terabyte of data.
- In November 2008, Google reported that its Mapreduce implementation sorted 1 terabyte in 68 seconds.
- In April 2009, a team at Yahoo used Hadoop to sort 1 terabyte in 62 seconds, beaten Google MapReduce implementation.



Figure 1.9 History of Hadoop

Various Release of Hadoop

2011 – 2012

- On 27 December 2011, Apache released Hadoop version 1.0 that includes support for Security, Hbase, etc.
- On 10 March 2012, release 1.0.1 was available. This is a bug fix release for version 1.0.
- On 23 May 2012, the Hadoop 2.0.0-alpha version was released. This release contains YARN.
- The second (alpha) version in the Hadoop-2.x series with a more stable version of YARN was released on 9 October 2012.

2017 – now

- On 13 December 2017, release 3.0.0 was available
- On 25 March 2018, Apache released Hadoop 3.0.1, which contains 49 bug fixes in Hadoop 3.0.0.
- On 6 April 2018, Hadoop release 3.1.0 came that contains 768 bug fixes, improvements, and enhancements since 3.0.0.
- Later, in May 2018, Hadoop 3.0.3 was released.
- On 8 August 2018, Apache 3.1.1 was released.
- Hadoop 3.1.3 is the latest version of Hadoop.

1.4.6 Analysing Data with Hadoop

- Hadoop is an open-source framework designed for distributed storage and parallel processing of large datasets across clusters of computers.
- It is widely used for big data analysis, allowing organizations to extract insights from structured, semi-structured, and unstructured data.

1.4.6.1 Components of Hadoop in Data Analysis:

1. HDFS (Hadoop Distributed File System)

- Stores massive data sets by splitting them into blocks and distributing them across the cluster.
- Provides high **fault tolerance** and **scalability**.

2. MapReduce

Programming model used to process data in parallel.

Consists of:

Map phase → Splits input data into key-value pairs.

Reduce phase → Aggregates results and produces the final output.

3. YARN (Yet Another Resource Negotiator)

Handles cluster resource management and job scheduling.

4. Hadoop Ecosystem Tools for Data Analysis

- **Hive** → SQL-like querying for big data.
- **Pig** → High-level data flow scripting.
- **HBase** → NoSQL database for random, real-time read/write.
- **Spark** (works with Hadoop) → Faster in-memory data analysis.

1.4.6.2 Process of Analyzing Data with Hadoop

1. **Data Collection** -Gather raw data from logs, social media, IoT devices, transactions, etc.
2. **Data Storage** - Load data into HDFS. Data is divided into blocks and replicated across the cluster.
3. **Data Processing**-Use MapReduce, Hive, Pig, or Spark to clean, transform, and process the data.
4. **Data Analysis**- Perform filtering, aggregation, joins, and statistical analysis. Machine Learning libraries like MLlib (Spark) or Mahout (Hadoop) can be used.
5. **Visualization** - Export results to BI tools (Tableau, Power BI) or dashboards for visualization.

Map and Reduce:

- MapReduce works by breaking the processing into two phases: the map phase and the reduce phase.
- Each phase has key-value pairs as input and output, the types of which may be chosen by the programmer.

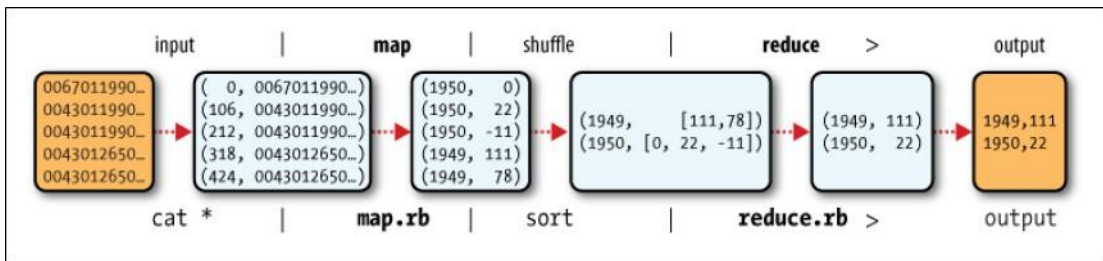


Figure 1.10 MapReduce Logical data Flow

Java MapReduce:

The next step is to express it in code. We need three things: a map function, a reduce function, and some code to run the job. The map function is represented by the Mapper class, which declares an abstract map() method. The following Example shows the implementation of our map method.

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class MaxTemperatureMapper
extends Mapper<LongWritable, Text, Text, IntWritable> {
private static final int MISSING = 9999;
@Override
public void map(LongWritable key, Text value, Context context)
throws IOException, InterruptedException {
String line = value.toString();
String year = line.substring(15, 19);
int airTemperature;
if (line.charAt(87) == '+') { // parseInt doesn't like leading plus signs
airTemperature = Integer.parseInt(line.substring(88, 92));
} else {
airTemperature = Integer.parseInt(line.substring(87, 92));
}
String quality = line.substring(92, 93);
if (airTemperature != MISSING && quality.matches("[01459]")) {
context.write(new Text(year), new IntWritable(airTemperature));
}
```

```

}
}
}

```

- The Mapper class is a generic type, with four formal type parameters that specify the input key, input value, output key, and output value types of the map function.
- For the present example, the input key is a long integer offset, the input value is a line of text, the output key is a year, and the output value is an air temperature (an integer).
- Hadoop provides its own set of basic types that are optimized for network serialization. These are found in the `org.apache.hadoop.io` package.
- Here we use `LongWritable`, which corresponds to a Java Long, `Text` (like Java String), and `IntWritable` (like Java Integer).
- The `map()` method is passed a key and a value. We convert the `Text` value containing the line of input into a Java String, then use its `substring()` method to extract the columns we are interested in.
- The `map()` method also provides an instance of `Context` to write the output to. In this case, we write the year as a `Text` object (since we are just using it as a key), and the temperature is wrapped in an `IntWritable`.
- We write an output record only if the temperature is present and the quality code indicates the temperature reading is OK.
- The reduce function is similarly defined using a `Reducer`, as illustrated in following example.

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class MaxTemperatureReducer
extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
Context context)
throws IOException, InterruptedException {
    int maxVal = Integer.MIN_VALUE;
    for (IntWritable value : values) {
        maxVal = Math.max(maxVal, value.get());
    }
    context.write(key, new IntWritable(maxVal));
}
}

```

Again, four formal type parameters are used to specify the input and output types, this time for the reduce function.

The input types of the reduce function must match the output types of the map function: Text and IntWritable. And in this case, the output types of the reduce function are Text and IntWritable, for a year and its maximum temperature, which we find by iterating through the temperatures and comparing each with a record of the highest found so far.

The third piece of code runs the MapReduce job. *Application to find the maximum temperature in the weather dataset*

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MaxTemperature {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: MaxTemperature <input path> <output path>");
            System.exit(-1);
        }
        Job job = new Job();
        job.setJarByClass(MaxTemperature.class);
        job.setJobName("Max temperature");
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        job.setMapperClass(MaxTemperatureMapper.class);
        job.setReducerClass(MaxTemperatureReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

- A Job object forms the specification of the job. It gives you control over how the job is run. When we run this job on a Hadoop cluster, we will package the code into a JAR file (which Hadoop will distribute around the cluster). Rather than explicitly specify the name of the JAR file, we can pass a class in the Job's setJarByClass() method, which Hadoop will use to locate the relevant JAR file by looking for the JAR file containing this class.

- An input path is specified by calling the static `addInputPath()` method on `FileInputFormat`, and it can be a single file, a directory (in which case, the input forms all the files in that directory), or a file pattern. As the name suggests, `addInputPath()` can be called more than once to use input from multiple paths.
- The output path (of which there is only one) is specified by the static `setOutputPath()` method on `FileOutputFormat`. It specifies a directory where the output files from the reducer functions are written. The directory shouldn't exist before running the job, as Hadoop will complain and not run the job.
- Next, we specify the map and reduce types to use via the `setMapperClass()` and `setReducerClass()` methods.
 - The `setOutputKeyClass()` and `setOutputValueClass()` methods control the output types for the map and the reduce functions, which are often the same, as they are in our case.
 - If they are different, then the map output types can be set using the methods `setMapOutputKeyClass()` and `setMapOutputValueClass()`.
- The input types are controlled via the input format, which we have not explicitly set since we are using the default `TextInputFormat`.
- After setting the classes that define the map and reduce functions, we are ready to run the job.
- The `waitForCompletion()` method on `Job` submits the job and waits for it to finish. The method's boolean argument is a verbose flag, so in this case the job writes information about its progress to the console.
- The return value of the `waitForCompletion()` method is a boolean indicating success (true) or failure (false), which we translate into the program's exit code of 0 or 1.

Advantages of Using Hadoop for Data Analysis

- Handles large-scale data (terabytes to petabytes).
- Works with structured, semi-structured, and unstructured data.
- Cost-effective (runs on commodity hardware).
- Scalable and fault-tolerant.
- Integrates with advanced analytics frameworks.

1.4.10 Hadoop Streaming

- Hadoop Streaming, a versatile feature since Hadoop 0.14.1, empowers developers to write MapReduce programs in languages like Ruby, Perl, Python, C++, and more without being confined to Java.
- This utility leverages UNIX standard streams, allowing any program that reads from standard input (STDIN) and writes to standard output (STDOUT) to act as mappers or reducers.

- With Hadoop Streaming, non-Java developers find an accessible path to process vast amounts of data using familiar tools and languages, enhancing the Hadoop ecosystem's flexibility and inclusivity.

How Hadoop Streaming Works

- Hadoop Streaming works by using Unix pipes to connect the output of a mapper or reducer written in a non-Java language to the input of the next stage in the MapReduce pipeline.

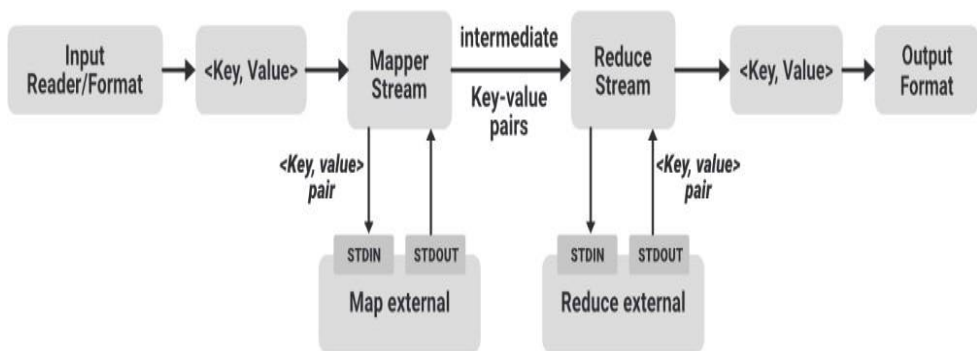


Figure 1.11 Hadoop Streaming Architecture

The six steps involved in the working of Hadoop Streaming are:

- **Step 1:** The input data is divided into chunks or blocks, typically 64MB to 128MB in size automatically. Each chunk of data is processed by a separate mapper.
- **Step 2:** The mapper reads the input data from standard input (stdin) and generates an intermediate key-value pair based on the logic of the mapper function which is written to standard output (stdout).
- **Step 3:** The intermediate key-value pairs are sorted and partitioned based on their keys ensuring that all values with the same key are directed to the same reducer.
- **Step 4:** The key-value pairs are passed to the reducers for further processing where each reducer receives a set of key-value pairs with the same key.
- **Step 5:** The reducer function, implemented by the developer, performs the required computations or aggregations on the data and generates the final output which is written to the standard output (stdout).
- **Step 6:** The final output generated by the reducers is stored in the specified output location in the HDFS.

- The distributed nature of Hadoop enables parallel execution of mappers and reducers across a cluster of machines, providing scalability and fault tolerance.
- The data processing is efficiently distributed across multiple nodes, allowing for faster processing of large-scale datasets.

Hadoop Streaming Commands

- To execute Hadoop Streaming jobs, you need to use the `hadoop jar` command and specify the streaming JAR file along with the necessary input and output paths.
- The **streaming JAR file** contains the required libraries and classes that enable the interaction between the Hadoop framework and the non-Java programs used as mappers and reducers.

Syntax:

```
hadoop jar hadoop-streaming.jar \
-input <input_path> \
-output <output_path> \
-mapper <mapper_command> \
-reducer <reducer_command>
```

- `hadoop-streaming.jar` The Hadoop Streaming JAR file.
- `<input_path>`: Input path in the Hadoop Distributed File System (HDFS).
- `<output_path>`: Output path in the HDFS.
- `<mapper_command>`: Commands or scripts to be used as the mapper.
- `<reducer_command>` Commands or scripts to be used as the reducer.

Streaming Command Options

Hadoop Streaming provides several command options to customize job execution.

Option	Explanation
<code>inputformat</code>	The input format class.
<code>-outputformat</code>	The output format class.
<code>-numReduceTasks</code>	Number of reducer tasks.
<code>-partitioner</code>	the partitioner class.
<code>-combiner</code>	The combiner command or script.
<code>-input</code>	The input path for the MapReduce job.
<code>-output</code>	The output path for the MapReduce job.
<code>-file</code>	Includes additional files with the job submission.
<code>-mapper</code>	The mapper command or script.

-reducer	The reducer command or script.
-cmdenv	Sets environment variables for the streaming tasks.
-inputDelimiter	The input delimiter for splitting lines into key/value.
-outputDelimiter	The output delimiter for key/value pairs.
-numMapTasks	The number of mapper tasks.
-cacheFile	The files to be cached on the cluster.
-cacheArchive	The archives are to be cached on the cluster.

Table 1.3 Hadoop Streaming Commands

1.4.11 Hadoop Ecosystem

- Hadoop is an open-source framework for storing and processing large-scale data across distributed clusters using commodity hardware.
- The Hadoop Ecosystem is a suite of tools and technologies built around Hadoop's core components (HDFS, YARN, MapReduce and Hadoop Common) to enhance its capabilities in data storage, processing, analysis and management.

Components of Hadoop Ecosystem:

Hadoop Ecosystem comprises several components that work together for efficient big data storage and processing:

- **HDFS (Hadoop Distributed File System):** Stores large datasets across distributed nodes.
- **YARN (Yet Another Resource Negotiator):** Manages cluster resources and job scheduling.
- **MapReduce:** A programming model for batch data processing.
- **Spark:** Provides fast, in-memory data processing.
- **Hive & Pig:** High-level tools for querying and analyzing large datasets.
- **HBase:** A NoSQL database for real-time read/write access.
- **Mahout & Spark MLlib:** Libraries for scalable machine learning.
- **Solr & Lucene:** Tools for full-text search and indexing.
- **Zookeeper:** Manages coordination and configuration across the cluster.
- **Oozie:** A workflow scheduler for managing Hadoop jobs.

HDFS

HDFS is a core component of Hadoop ecosystem, designed to store large volumes of structured or unstructured data across multiple nodes. It manages metadata through log files and splits storage tasks between two main parts:

- **NameNode (master):** Stores metadata (data about data) and requires fewer resources.
- **DataNodes (slaves):** Store actual data on commodity hardware, making Hadoop cost-effective.

HDFS handles coordination between clusters and hardware, serving as the backbone of entire Hadoop system.

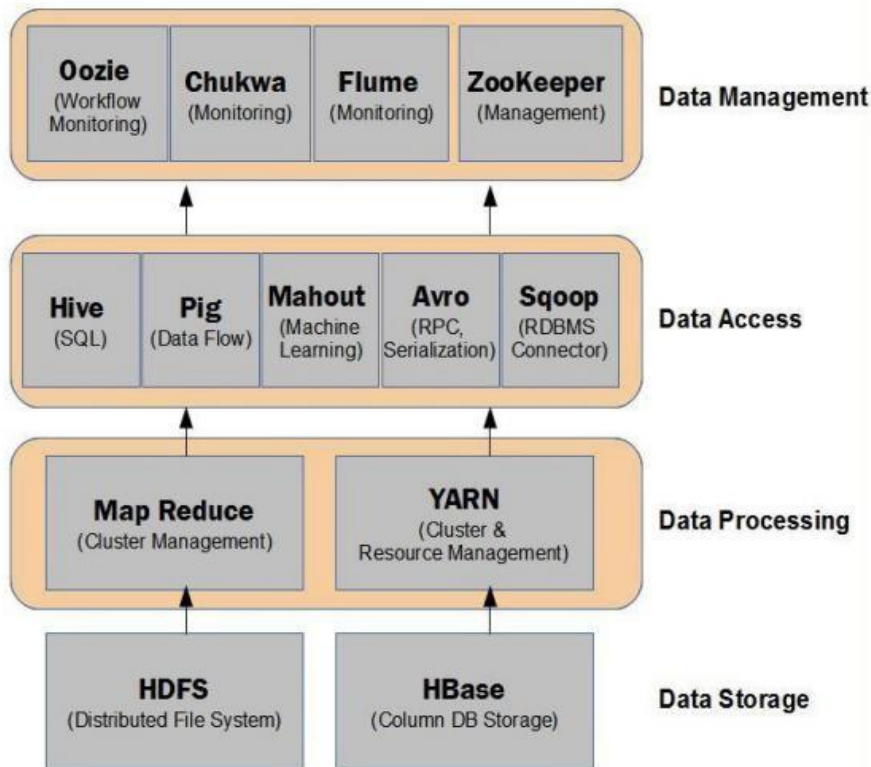


Figure 1.12 Hadoop Ecosystem

YARN

YARN (Yet Another Resource Negotiator) is resource management layer of Hadoop, responsible for scheduling and allocating resources across the cluster. It has three key components:

- **ResourceManager:** Allocates resources to various applications in the system.
- **NodeManager:** Manages resources (CPU, memory, etc.) on individual nodes and reports to ResourceManager.
- **ApplicationMaster:** Acts as a bridge between ResourceManager and NodeManager, handling resource negotiation for each application.

Together, they ensure efficient resource utilization and smooth execution of jobs in the Hadoop cluster.

MapReduce

MapReduce enables distributed and parallel data processing on large datasets. It allows developers to write programs that transform big data into manageable results.

- **Map():** Processes input data by filtering, sorting and organizing it into key-value pairs.
- **Reduce():** Takes the output from Map(), aggregates the data and summarizes it into a smaller, consolidated set of results.

Together, they efficiently handle large-scale data transformations across the Hadoop cluster.

PIG

Pig is a platform developed by Yahoo for analyzing large datasets using **Pig Latin**, a SQL-like scripting language designed for data processing.

- It simplifies complex data flows and handles MapReduce operations internally.
- The processed results are stored in HDFS.
- **Pig Latin** runs on **Pig Runtime**, similar to Java on JVM.
- It enhances programming ease and optimization, making it a key part of Hadoop ecosystem.

HIVE

Hive uses a SQL-like interface (**HQL: Hive Query Language**) to read and write large datasets.

- It supports both real-time and batch processing, making it highly scalable.
- Hive supports all standard SQL datatypes, easing query operations.

It has two main components:

- **JDBC/ODBC drivers:** manage data connections and access permissions.
- **Hive Command Line:** used for query execution and processing.

Mahout

Mahout brings **machine learning capabilities** to Hadoop-based systems by enabling applications to learn from data using patterns and algorithms.

- It provides built-in libraries for clustering, classification and collaborative filtering.
- Users can invoke algorithms as needed through Mahout's scalable, distributed libraries.

Apache Spark

Apache Spark is a powerful platform for batch, real-time, interactive, iterative processing and graph computations.

- It uses **in-memory computing**, making it faster and more efficient than traditional systems.
- Spark is ideal for **real-time data**, while Hadoop suits batch or structured data, so both are often used together in organizations.

Apache HBase

HBase is a **NoSQL database** in Hadoop ecosystem that supports all data types and handles large datasets efficiently, similar to Google's BigTable.

- It is ideal for fast read/write operations on small portions of data within massive datasets.

- HBase offers a **fault-tolerant** and efficient way to store and retrieve data quickly, making it useful for real-time lookups.

Other Components

Apart from core components, Hadoop also includes important tools like:

Solr & Lucene: Used for searching and indexing. Lucene (Java-based) offers features like spell check and Solr acts as its powerful search platform.

Zookeeper: Handles coordination and synchronization between Hadoop components, ensuring consistent communication and grouping across the cluster.

Oozie: A job scheduler that manages workflows. It supports two types of jobs:

- **Workflow jobs** (executed in sequence)
- **Coordinator jobs** (triggered by data or time-based events).

Apache Sqoop

- Sqoop is a tool designed to transfer data between Hadoop and relational database servers.
- It is used to import data from relational databases such as MySQL, Oracle to Hadoop HDFS, and export from Hadoop file system to relational databases. It is provided by the Apache Software Foundation.
- **--SqoopImport** The import tool imports individual tables from RDBMS to HDFS. Each row in a table is treated as a record in HDFS. All records are stored as text data in text files or as binary data in Avro and Sequence files.
- **--Sqoop Export** The export tool exports a set of files from HDFS back to an RDBMS. The files given as input to Sqoop contain records, which are called as rows in table. Those are read and parsed into a set of records and delimited with user-specified delimiter.

Apache Chukwa

- Chukwa aims to provide a flexible and powerful platform for distributed data collection and rapid data processing.
- It is an open source data collection system for monitoring large distributed system and is built on top of the Hadoop Distributed File System (HDFS) and Map/Reduce framework that inherits Hadoop's scalability and robustness.
- Chukwa also includes a flexible and powerful toolkit for displaying, monitoring and analyzing results to make the best use of the collected data.
- In order to maintain this flexibility, Chukwa is structured as a pipeline of collection and processing stages, with clean and narrow interfaces between stages.
- Chukwa has four primary components:
 1. Agents that run on each machine and emit data.
 2. Collectors that receive data from the agent and write it to stable storage.
 3. MapReduce jobs for parsing and archiving the data.
 4. HICC, the Hadoop Infrastructure Care Center; a webportal style interface for displaying data.

Apache Flume

- Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data.
- It has a simple and flexible architecture based on streaming data flows. It is robust and fault tolerant with tunable reliability mechanisms and many failover and recovery mechanisms.
- It uses a simple extensible data model that allows for online analytic application.

Features of Flume

- Flume ingests log data from multiple web servers into a centralized store (HDFS, HBase) efficiently.
- Using Flume, we can get the data from multiple servers immediately into Hadoop.
- Along with the log files, Flume is also used to import huge volumes of event data produced by social networking sites.
- Flume supports a large set of sources and destinations types and can be scaled horizontally.
- Flume supports multi-hop flows, fan-in and fan out flows, contextual routing, etc.

Avro

- Avro is a data serialization system that allows for encoding the schema of Hadoop files. It is adept at parsing data and performing remote procedure calls.
- It was developed by Doug Cutting, the father of Hadoop. Since Hadoop writable classes lack language portability, Avro has become quite helpful, as it deals with data formats that can be processed by multiple languages.
- Avro is a preferred tool to serialize data in Hadoop. Avro has a schema-based system.
- A language-independent schema is associated with its read and writes operations. Avro serializes the data which has a built-in schema.
- Avro serializes the data into a compact binary format, which can be deserialized by any application.
- Avro uses JSON format to declare the data structures. Presently, it supports languages such as Java, C, C++, C#, Python, and Ruby.

Below are some of the prominent features of Avro:

- Avro is a language-neutral data serialization system. It can be processed by many languages (currently C, C++, C#, Java, Python, and Ruby).
- Avro creates binary structured format that is both compressible and splittable. Hence it can be efficiently used as the input to Hadoop MapReduce jobs.
- Avro provides rich data structures. For example, you can create a record that contains an array, an enumerated type, and a sub record. These data types can be created in any language can be processed in Hadoop, and the results can be fed to a third language.

- Avro schemas defined in JSON facilitate implementation in the languages that already have JSON libraries.
- Avro creates a self-describing file named Avro Data File, in which it stores data along with its schema in the metadata section.

1.4.12 Applications of Big Data

In today's world, there are a lot of data. Big companies utilize those data for their business growth. By analyzing this data, the useful decision can be made in various cases as discussed below:

1. Tracking Customer Spending Habit, Shopping Behavior: In big retailers store (like Amazon, Walmart, Big Bazar etc.) management team has to keep data of customer's spending habit (in which product customer spent, in which brand they wish to spend, how frequently they spent), shopping behavior, customer's most liked product (so that they can keep those products in the store). Which product is being searched/sold most, based on that data, production/collection rate of that product get fixed.

- Banking sector uses their customer's spending behavior-related data so that they can provide the offer to a particular customer to buy his particular liked product by using bank's credit or debit card with discount or cashback. By this way, they can send the right offer to the right person at the right time.

2. Recommendation: By tracking customer spending habit, shopping behavior, Big retailers store provide a recommendation to the customer. E-commerce site like Amazon, Walmart, Flipkart does product recommendation. They track what product a customer is searching, based on that data they recommend that type of product to that customer.

- As an example, suppose any customer searched bed cover on Amazon. So, Amazon got data that customer may be interested to buy bed cover. Next time when that customer will go to any google page, advertisement of various bed covers will be seen. Thus, advertisement of the right product to the right customer can be sent.

YouTube also shows recommend video based on user's previous liked, watched video type. Based on the content of a video, the user is watching, relevant advertisement is shown during video running. As an example, suppose someone watching a tutorial video of Big data, then advertisement of some other big data course will be shown during that video.

3. Smart Traffic System: Data about the condition of the traffic of different road, collected through camera kept beside the road, at entry and exit point of the city, GPS device placed in the vehicle (Ola, Uber cab, etc.). All such data are analyzed and jam-free or less jam way, less time taking ways are recommended. Such a way smart traffic system can be built in the city by Big data analysis. One more profit is fuel consumption can be reduced.

4. Secure Air Traffic System: At various places of flight (like propeller etc) sensors present. These sensors capture data like the speed of flight, moisture, temperature, other environmental condition. Based on such data analysis, an environmental parameter within flight are set up and varied.

By analyzing flight's machine-generated data, it can be estimated how long the machine can operate flawlessly when it to be replaced/repaired.

5. Auto Driving Car: Big data analysis helps drive a car without human interpretation. In the various spot of car camera, a sensor placed, that gather data like the size of the surrounding car, obstacle, distance from those, etc. These data are being analyzed, then various calculation like how many angles to rotate, what should be speed, when to stop, etc carried out. These calculations help to take action automatically.

6. Virtual Personal Assistant Tool: Big data analysis helps virtual personal assistant tool (like Siri in Apple Device, Cortana in Windows, Google Assistant in Android) to provide the answer of the various question asked by users. This tool tracks the location of the user, their local time, season, other data related to question asked, etc. Analyzing all such data, it provides an answer.

As an example, suppose one user asks "Do I need to take Umbrella?", the tool collects data like location of the user, season and weather condition at that location, then analyze these data to conclude if there is a chance of raining, then provide the answer.

7. IoT:

- Manufacturing company install IOT sensor into machines to collect operational data. Analyzing such data, it can be predicted how long machine will work without any problem when it requires repairing so that company can take action before the situation when machine facing a lot of issues or gets totally down. Thus, the cost to replace the whole machine can be saved.
- In the Healthcare field, big data is providing a significant contribution. Using big data tool, data regarding patient experience is collected and is used by doctors to give better treatment. IoT device can sense a symptom of probable coming disease in the human body and prevent it from giving advance treatment. IoT Sensor placed near-patient, new-born baby constantly keeps track of various health condition like heart bit rate, blood presser, etc. Whenever any parameter crosses the safe limit, an alarm sent to a doctor, so that they can take step remotely very soon.

8. Education Sector: Online educational course conducting organization utilize big data to search candidate, interested in that course. If someone searches for YouTube tutorial video on a subject, then online or offline course provider organization on that subject send ad online to that person about their course.

9. Energy Sector: Smart electric meter read consumed power every 15 minutes and sends this read data to the server, where data analyzed and it can be estimated what is the time in a day when the power load is less throughout the city.

UNIT II

HADOOP DISTRIBUTED FILE SYSTEM

HDFS (Hadoop Distributed File System): Design of HDFS – HDFS Concepts – Command Line Interface – Hadoop file system interfaces – Data flow – Data Ingest with Flume and Sqoop – Hadoop I/O: Compression – Serialization – Avro and File-Based Data structures.

2.1 HDFS (Hadoop Distributed File System)

- When a dataset outgrows the storage capacity of a single physical machine, it becomes necessary to partition it across a number of separate machines. Filesystems that manage the storage across a network of machines are called *distributed filesystems*.
- Since they are network-based, all the complications of network programming kick in, thus making distributed filesystems more complex than regular disk filesystems. For example, one of the biggest challenges is making the filesystem tolerate node failure without suffering data loss.
- Hadoop comes with a distributed filesystem called HDFS, which stands for *Hadoop Distributed Filesystem*. (You may sometimes see references to “DFS”—informally or in older documentation or configurations—which is the same thing.)
- The Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware.
- It has many similarities with existing distributed file systems. However, the differences from other distributed file systems are significant.
- HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware.
- HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

- HDFS relaxes a few POSIX requirements to enable streaming access to file system data.
- HDFS was originally built as infrastructure for the Apache Nutch web search engine project. HDFS is now an Apache Hadoop subproject.
- Hadoop is an open-source Java framework that stores and processes massive data using clusters of inexpensive (commodity) hardware. Based on Google's MapReduce programming model, it enables distributed, parallel processing. Big companies like Facebook, Yahoo, Netflix and eBay use Hadoop to handle large-scale data efficiently.

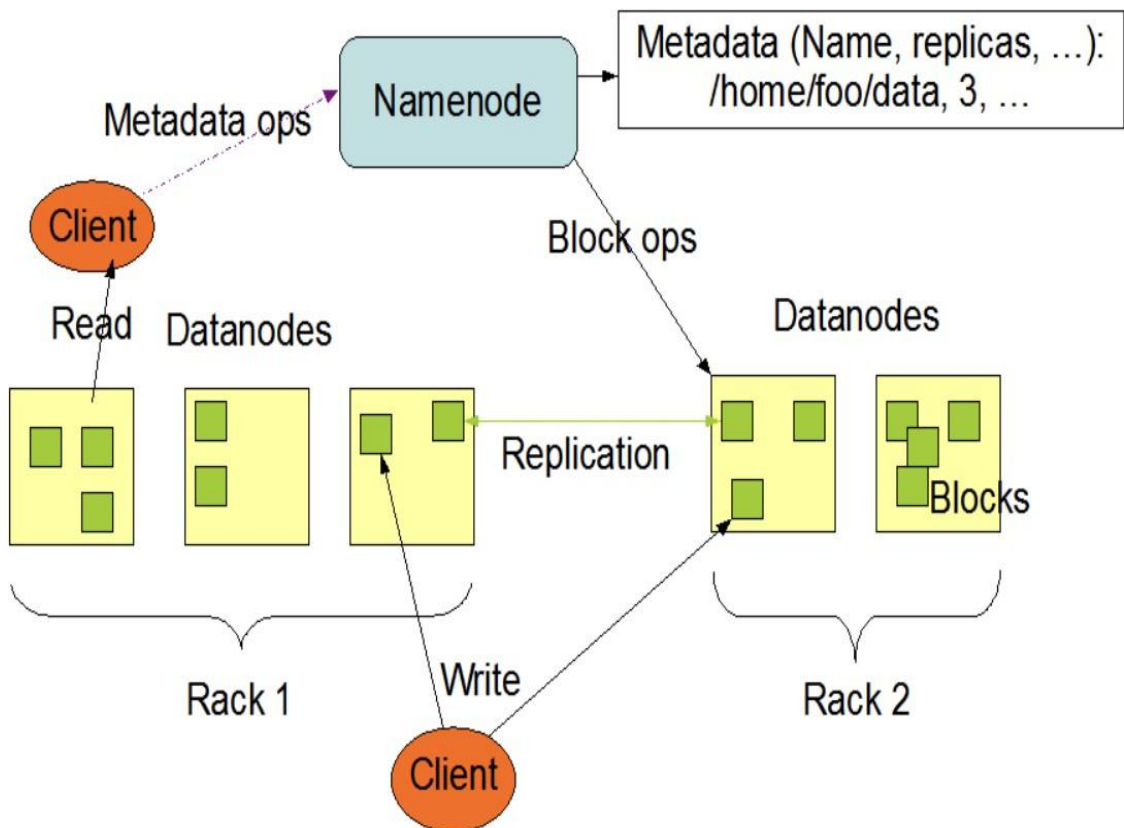


Figure 2.1 HDFS Architecture

Name Node and Data Node:

- HDFS has a master/slave architecture. An HDFS cluster consists of a single NameNode, a master server that manages the file system namespace and regulates access to files by clients.
- In addition, there are a number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on.
- HDFS exposes a file system namespace and allows user data to be stored in files. Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.
- The NameNode executes file system namespace operations like opening, closing, and renaming files and directories.
- It also determines the mapping of blocks to DataNodes. The DataNodes are responsible for serving read and write requests from the file system's clients.
- The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.
- The NameNode and DataNode are pieces of software designed to run on commodity machines. These machines typically run a GNU/Linux operating system (OS).
- HDFS is built using the Java language; any machine that supports Java can run the NameNode or the DataNode software.
- Usage of the highly portable Java language means that HDFS can be deployed on a wide range of machines. A typical deployment has a dedicated machine that runs only the NameNode software.
- Each of the other machines in the cluster runs one instance of the DataNode software. The architecture does not preclude running multiple DataNodes on the same machine but in a real deployment that is rarely the case.
- The existence of a single NameNode in a cluster greatly simplifies the architecture of the system.

- Files in HDFS are write-once (except for appends and truncates) and have strictly one writer at any time.
- The NameNode makes all decisions regarding replication of blocks. It periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster.
- Receipt of a Heartbeat implies that the DataNode is functioning properly. A Block report contains a list of all blocks on a DataNode.

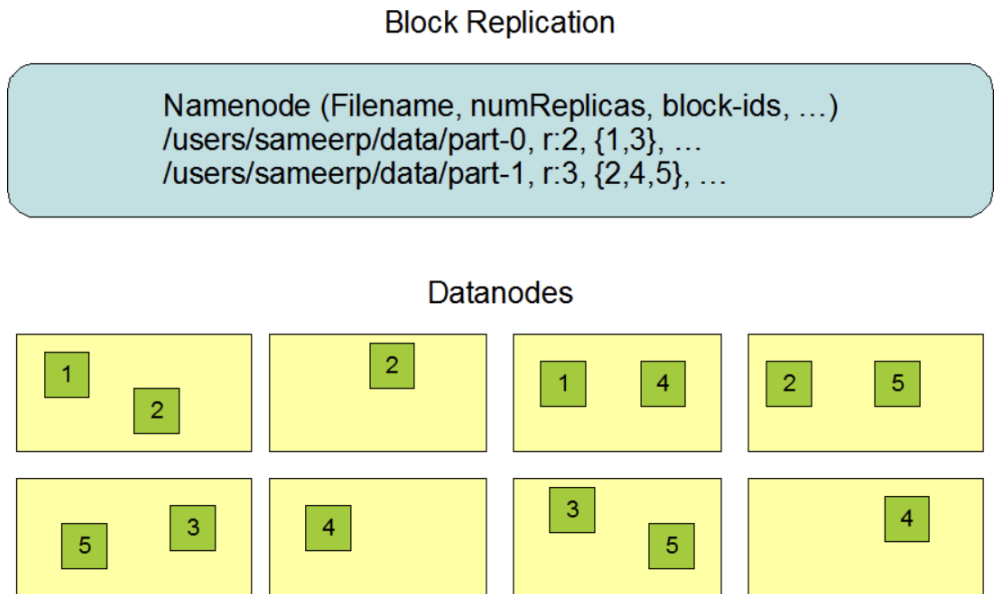


Figure 2.2 HDFS – Block Replication

The Communication Protocols

- All HDFS communication protocols are layered on top of the TCP/IP protocol. A client establishes a connection to a configurable TCP port on the NameNode machine.
- It talks the ClientProtocol with the NameNode. The DataNodes talk to the NameNode using the DataNode Protocol.

- A Remote Procedure Call (RPC) abstraction wraps both the Client Protocol and the DataNode Protocol. By design, the NameNode never initiates any RPCs. Instead, it only responds to RPC requests issued by DataNodes or clients.

Robustness

- The primary objective of HDFS is to store data reliably even in the presence of failures. The three common types of failures are NameNode failures, DataNode failures and network partitions.

Data Organization

- HDFS is designed to support very large files. Applications that are compatible with HDFS are those that deal with large data sets. These applications write their data only once but they read it one or more times and require these reads to be satisfied at streaming speeds.
- HDFS supports write-once-read-many semantics on files. A typical block size used by HDFS is 128 MB. Thus, an HDFS file is chopped up into 128 MB chunks, and if possible, each chunk will reside on a different DataNode.

2.2 Design of HDFS

- HDFS is a filesystem designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.

1. Very large files

- “Very large” in this context means files that are hundreds of megabytes, gigabytes, or terabytes in size. There are Hadoop clusters running today that store petabytes of data.

2. *Streaming data access*

- HDFS is built around the idea that the most efficient data processing pattern is a write-once, read-many-times pattern.
- A dataset is typically generated or copied from source, then various analyses are performed on that dataset over time.

- Each analysis will involve a large proportion, if not all, of the dataset, so the time to read the whole dataset is more important than the latency in reading the first record.

3. *Commodity hardware*

- Hadoop doesn't require expensive, highly reliable hardware to run on. Its designed to run on clusters of commodity hardware (commonly available hardware available)

4. Low-latency data access

- Applications that require low-latency access to data, in the tens of milliseconds range, will not work well with HDFS. Remember, HDFS is optimized for delivering a high throughput of data, and this may be at the expense of latency.
- HBase is currently a better choice for low-latency access.

5. Lots of small files

- Since the namenode holds filesystem metadata in memory, the limit to the number of files in a filesystem is governed by the amount of memory on the namenode.
- As a rule of thumb, each file, directory, and block takes about 150 bytes. So, for example, if you had one million files, each taking one block, you would need at least 300 MB of memory.
- While storing millions of files is feasible, billions is beyond the capability of current hardware.

6. Multiple writers, arbitrary file modifications

- Files in HDFS may be written to by a single writer. Writes are always made at the end of the file. There is no support for multiple writers, or for modifications at arbitrary offsets in the file.

2.3 HDFS Concepts

1. Blocks:

- HDFS, too, has the concept of a block, but it is a much larger unit—64 MB by default.
- Like in a filesystem for a single disk, files in HDFS are broken into block-sized chunks, which are stored as independent units.
- Unlike a filesystem for a single disk, a file in HDFS that is smaller than a single block does not occupy a full blocks worth of underlying storage.
- a block abstraction for a distributed filesystem brings several benefits. The first
- benefit is the most obvious: a file can be larger than any single disk in the network.
- There's nothing that requires the blocks from a file to be stored on the same disk, so they can take advantage of any of the disks in the cluster.
- Second, making the unit of abstraction a block rather than a file simplifies the storage subsystem.
- Blocks fit well with replication for providing fault tolerance and availability.
- To insure against corrupted blocks and disk and machine failure, each block is replicated to a small number of physically separate machines (typically three).
- If a block becomes unavailable, a copy can be read from another location in a way that is transparent to the client.
- A block that is no longer available due to corruption or machine failure can be replicated from its alternative locations to other live machines to bring the replication factor back to the normal level.

2.4 The Command-Line Interface

- There are many other interfaces to HDFS, but the command line is one of the simplest and, to many developers, the most familiar.
- There are two properties that we set in the pseudo-distributed configuration that deserve further explanation.
- The first is `fs.default.name`, set to `hdfs://localhost/`, which is used to set a default filesystem for Hadoop. Filesystems are specified by a URI, and here we have used an `hdfs` URI to configure Hadoop to use HDFS by default.
- The HDFS daemons will use this property to determine the host and port for the HDFS namenode.
- We'll be running it on localhost, on the default HDFS port, 8020. And HDFS clients will use this property to work out where the namenode is running so they can connect to it.
- We set the second property, `dfs.replication`, to 1 so that HDFS doesn't replicate filesystem blocks by the default factor of three.
- When running with a single datanode, HDFS can't replicate blocks to three datanodes, so it would perpetually warn about blocks being under-replicated. This setting solves that problem.

Basic Filesystem Operations:

The usual filesystem operations such as

- i) Reading files,
- ii) Creating directories,
- iii) Moving files,
- iv) Deleting data, and
- v) Listing directories

Start by copying a file from the local filesystem to HDFS:

```
%hadoop fs -copyFromLocal input/docs/quangle.txt
```

2.5 Hadoop Filesystems

- Hadoop has an abstract notion of filesystem, of which HDFS is just one implementation.
- The Java abstract class `org.apache.hadoop.fs.FileSystem` represents a filesystem in Hadoop, and there are several concrete implementations, which are described in below given Table.

Filesystem	URI scheme	Java implementation (all under <code>org.apache.hadoop</code>)	Description
Local	<i>file</i>	<code>fs.LocalFileSystem</code>	A filesystem for a locally connected disk with client-side checksums. Use <code>RawLocalFileSystem</code> for a local filesystem with no checksums. See “LocalFileSystem” on page 84 .
HDFS	<i>hdfs</i>	<code>hdfs.DistributedFileSystem</code>	Hadoop’s distributed filesystem. HDFS is designed to work efficiently in conjunction with MapReduce.
HFTP	<i>hftp</i>	<code>hdfs.HftpFileSystem</code>	A filesystem providing read-only access to HDFS over HTTP. (Despite its name, HFTP has no connection with FTP.) Often used with <i>distcp</i> (see “Parallel Copying with distcp” on page 76) to copy data between HDFS clusters running different versions.
HSFTP	<i>hsftp</i>	<code>hdfs.HsftpFileSystem</code>	A filesystem providing read-only access to HDFS over HTTPS. (Again, this has no connection with FTP.)
WebHDFS	<i>webhdfs</i>	<code>hdfs.web.WebHdfsFileSystem</code>	A filesystem providing secure read-write access to HDFS over HTTP. WebHDFS is intended as a replacement for HFTP and HSFTP.
HAR	<i>har</i>	<code>fs.HarFileSystem</code>	A filesystem layered on another filesystem for archiving files. Hadoop Archives are typically used for archiving files in HDFS to reduce the namenode’s memory usage. See “Hadoop Archives” on page 78 .
KFS (Cloud-Store)	<i>kfs</i>	<code>fs.kfs.KosmosFileSystem</code>	CloudStore (formerly Kosmos filesystem) is a distributed filesystem like HDFS or Google’s GFS, written in C++. Find more information about it at http://kosmosfs.sourceforge.net/ .
FTP	<i>ftp</i>	<code>fs.ftp.FTPFileSystem</code>	A filesystem backed by an FTP server.
S3 (native)	<i>s3n</i>	<code>fs.s3native.NativeS3FileSystem</code>	A filesystem backed by Amazon S3. See http://wiki.apache.org/hadoop/AmazonS3 .
S3 (block-based)	<i>s3</i>	<code>fs.s3.S3FileSystem</code>	A filesystem backed by Amazon S3, which stores files in blocks (much like HDFS) to overcome S3’s 5 GB file size limit.

C:

- Hadoop provides a C library called *libhdfs* that mirrors the Java FileSystem interface (it was written as a C library for accessing HDFS, but despite its name it can be used to access any Hadoop filesystem).
- It works using the *Java Native Interface* (JNI) to call a Java filesystem client.
- The C API is very similar to the Java one, but it typically lags the Java one, so newer features may not be supported.

FUSE:

- *Filesystem in Userspace* (FUSE) allows filesystems that are implemented in user space to be integrated as a Unix filesystem.
- Hadoop's Fuse-DFS contrib module allows any Hadoop filesystem (but typically HDFS) to be mounted as a standard filesystem.
- You can then use Unix utilities (such as ls and cat) to interact with the filesystem, as well as POSIX libraries to access the filesystem from any programming language.
- Fuse-DFS is implemented in C using *libhdfs* as the interface to HDFS. Documentation for compiling and running Fuse-DFS is located in the *src/contrib/fuse-dfs* directory of the Hadoop distribution.

2.6 Data Flow

2.6.1 Anatomy of a File Read:

- To get an idea of how data flows between the client interacting with HDFS, the namenode and the datanodes, which shows the main sequence of events when reading a file.
- The client opens the file it wishes to read by calling `open()` on the `FileSystem` object, which for HDFS is an instance of `DistributedFileSystem`.
- `DistributedFileSystem` calls the namenode, using RPC, to determine the locations of the blocks for the first few blocks in the file (step 2).
- For each block, the namenode returns the addresses of the datanodes that have a copy of that block.

- The client then calls `read()` on the stream (step 3). `DFSInputStream`, which has stored the datanode addresses for the first few blocks in the file, then connects to the first (closest) datanode for the first block in the file. Data is streamed from the datanode back to the client, which calls `read()` repeatedly on the stream (step 4). When the end of the block is reached, `DFSInputStream` will close the connection to the datanode, then find the best datanode for the next block (step 5).
- This happens transparently to the client, which from its point of view is just reading a continuous stream.

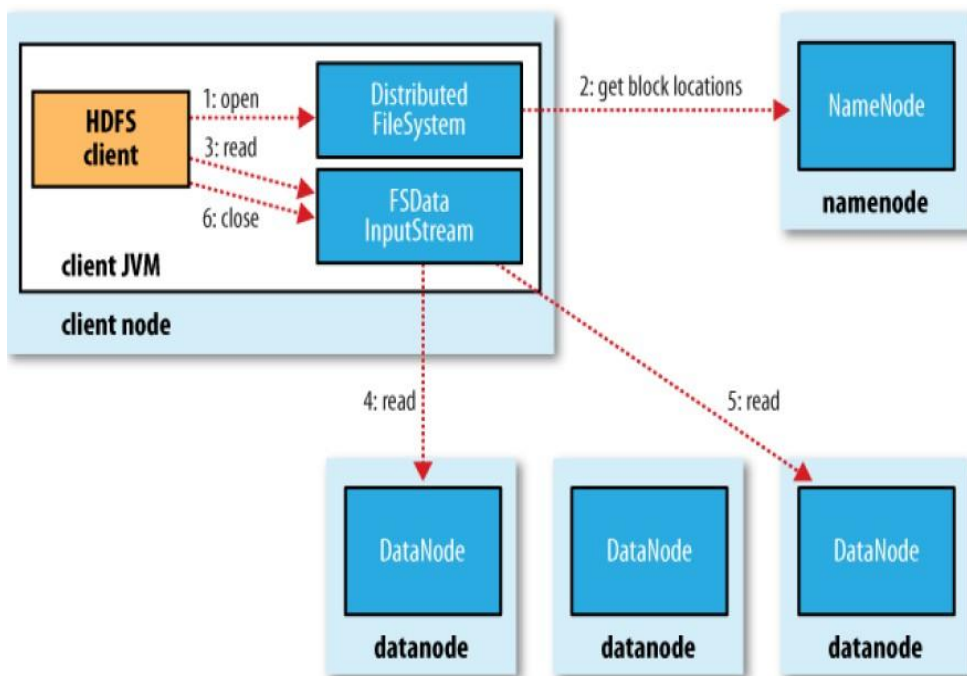


Figure 2.4 *A client reading data from HDFS*

- Blocks are read in order with the `DFSInputStream` opening new connections to datanodes as the client reads through the stream. It will also call the namenode to retrieve the datanode locations for the next batch of blocks as needed.
- When the client has finished reading, it calls `close()` on the `FSDataInputStream` (step 6).

- During reading, if the DFSInputStream encounters an error while communicating with a datanode, then it will try the next closest one for that block. It will also remember datanodes that have failed so that it doesn't needlessly retry them for later blocks.
- The DFSInputStream also verifies checksums for the data transferred to it from the datanode.
- If a corrupted block is found, it is reported to the namenode before the DFSInputStream attempts to read a replica of the block from another datanode.

2.6.2 Anatomy of a File Write

- The client creates the file by calling create() on DistributedFileSystem (step 1 in DistributedFileSystem makes an RPC call to the namenode to create a new file in the filesystem's namespace, with no blocks associated with it (step 2).
- The namenode performs various checks to make sure the file doesn't already exist, and that the client has the right permissions to create the file.
- If these checks pass, the namenode makes a record of the new file; otherwise, file creation fails and the client is thrown an IOException.
- The DistributedFileSystem returns an FSDataOutputStream for the client to start writing data to.
- Just as in the read case, FSDataOutputStream wraps a DFSOutputStream, which handles communication with the datanodes and namenode.

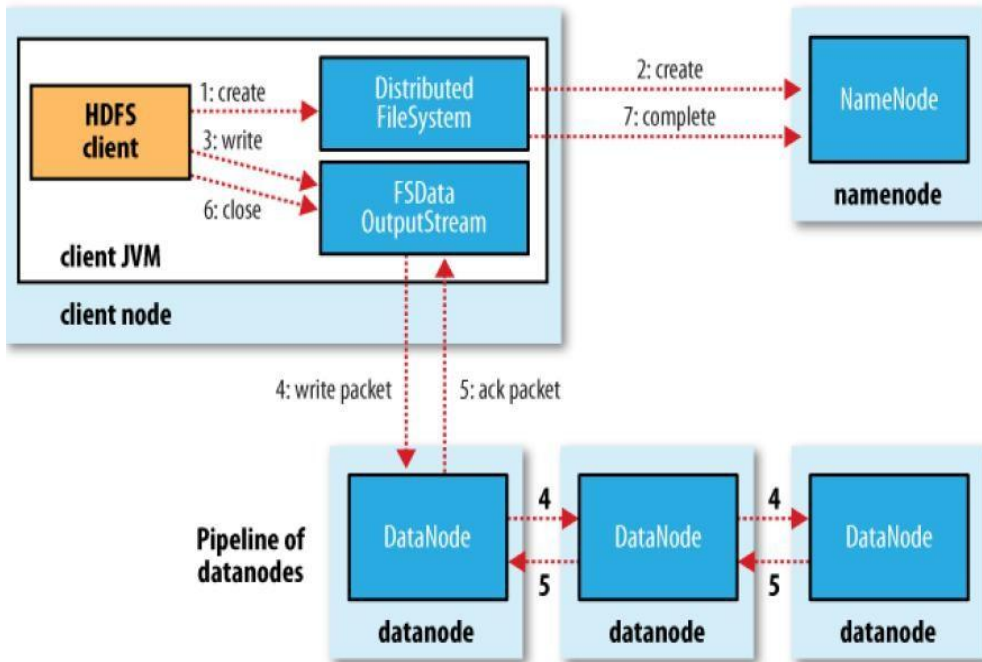


Figure 2.5 *A client writing data to HDFS*

- As the client writes data (step 3), DFSOutputStream splits it into packets, which it writes to an internal queue, called the *data queue*.
- The data queue is consumed by the Data Streamer, whose responsibility it is to ask the namenode to allocate new blocks by picking a list of suitable datanodes to store the replicas.
- The DataStreamer streams the packets to the first datanode in the pipeline, which stores the packet and forwards it to the second datanode in the pipeline. Similarly, the second datanode stores the packet and forwards it to the third (and last) datanode in the pipeline (step 4).
- DFSOutputStream also maintains an internal queue of packets that are waiting to be acknowledged by datanodes, called the *ack queue*.
- A packet is removed from the ack queue only when it has been acknowledged by all the datanodes in the pipeline (step 5).

2.7 Data Ingest with Flume and Sqoop

- Data ingestion is the process of bringing data from various sources into a central system for storage, analysis, or processing. It is primarily responsible for moving data from one place to another and forms the “Extract” phase in the ETL/ELT process.

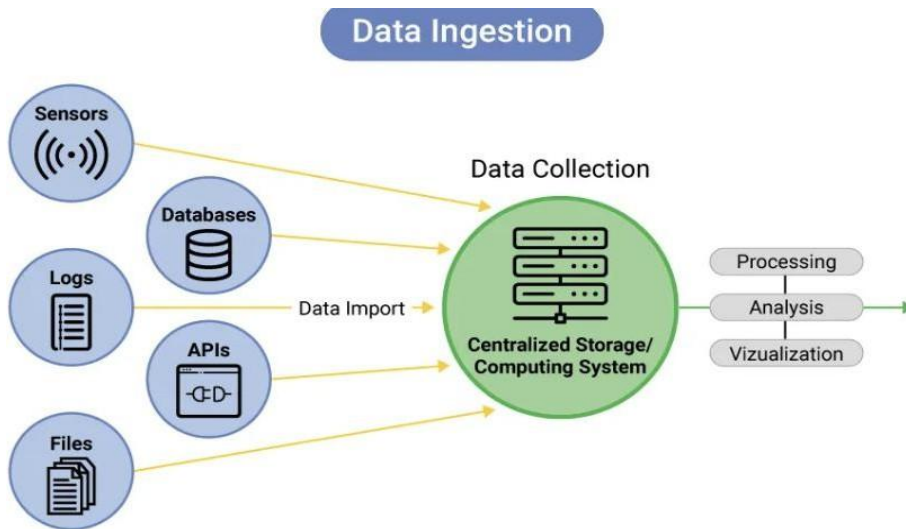


Figure 2.6 Data Ingestion process

2.7.1 Apache Flume

- Apache Flume is a tool/service/data ingestion mechanism for collecting aggregating and transporting large amounts of streaming data such as log files, events (etc...) from various sources to a centralized data store.
- Flume is a highly reliable, distributed, and configurable tool. It is principally designed to copy streaming data (log data) from various web servers to HDFS.
- Flume ingests log data from multiple web servers into a centralized store (HDFS, HBase) efficiently.
- Using Flume, we can get the data from multiple servers immediately into Hadoop.

- Flume supports multi-hop flows, fan-in fan-out flows, contextual routing, etc.
- Flume can be scaled horizontally.

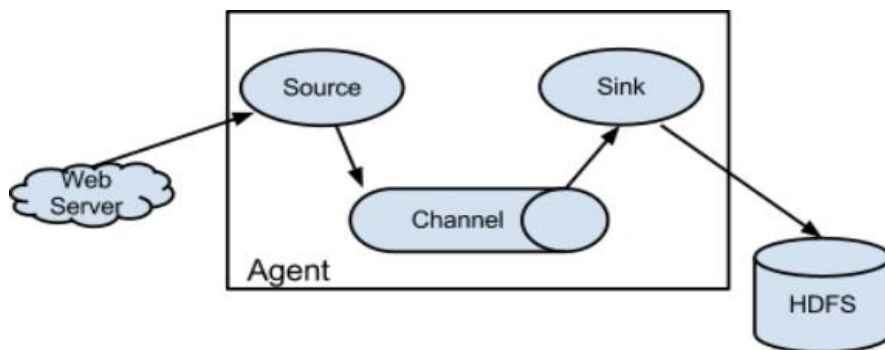
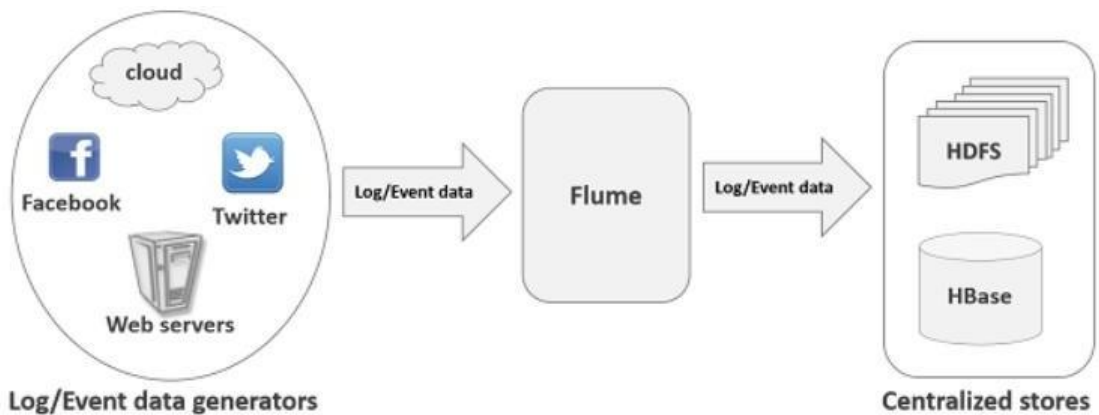


Figure 2.7 Flume Architecture

- Flume is a framework which is used to move log data into HDFS. Generally, events and log data are generated by the log servers and these servers have Flume agents running on them. These agents receive the data from the data generators.
- The data in these agents will be collected by an intermediate node known as Collector. Just like agents, there can be multiple collectors in Flume.

- Finally, the data from all these collectors will be aggregated and pushed to a centralized store such as HBase or HDFS. The following diagram explains the data flow in Flume.
- Within Flume, there can be multiple agents and before reaching the final destination, an event may travel through more than one agent. This is known as **multi-hop flow**.

Fan-out Flow

- The dataflow from one source to multiple channels is known as fan-out flow. It is of two types –
- Replicating – The data flow where the data will be replicated in all the configured channels.
- Multiplexing – The data flow where the data will be sent to a selected channel which is mentioned in the header of the event.
- The data flow in which the data will be transferred from many sources to one channel is known as **fan-in flow**.

2.7.2 Sqoop – SQL to Hadoop and Hadoop to SQL

- Sqoop is a tool designed to transfer data between Hadoop and relational database servers. It is used to import data from relational databases such as MySQL, Oracle to Hadoop HDFS, and export from Hadoop file system to relational databases. It is provided by the Apache Software Foundation.

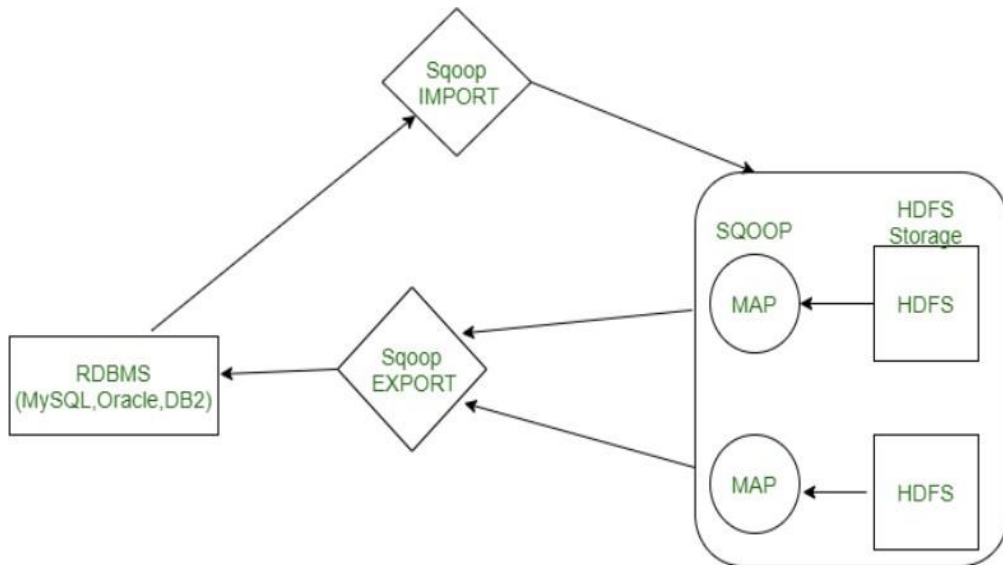


Figure 2.7 Sqoop Architecture

- Sqoop also helps us to connect the result from the SQL Queries into Hadoop distributed file system.
- Sqoop helps us to load the processed data directly into the hive or Hbase.
- It performs the security operation of data with the help of Kerberos.
- With the help of Sqoop, we can perform compression of processed data.
- Sqoop is highly powerful and efficient in nature.

There are two major operations performed in Sqoop :

- 1. Import**
- 2. Export**

Sqoop Import :

- Sqoop import command helps in implementation of the operation. With the help of the import command, we can import a table from the Relational database management system to the Hadoop database server.

- Records in Hadoop structure are stored in text files and each record is imported as a separate record in Hadoop database server.
- We can also create load and partition in Hive while importing data..Sqoop also supports incremental import of data which means in case we have imported a database and we want to add some more rows, so with the help of these functions we can only add the new rows to existing database, not the complete database.

Sqoop Export :

- Sqoop export command helps in the implementation of operation. With the help of the export command which works as a reverse process of operation.
- Herewith the help of the export command we can transfer the data from the Hadoop database file system to the Relational database management system.
- The data which will be exported is processed into records before operation is completed. The export of data is done with two steps, first is to examine the database for metadata and second step involves migration of data.

Advantages of Sqoop :

- With the help of Sqoop, we can perform transfer operations of data with a variety of structured data stores like Oracle, Teradata, etc.
- Sqoop helps us to perform ETL operations in a very fast and cost-effective manner.
- With the help of Sqoop, we can perform parallel processing of data which leads to fasten the overall process.
- Sqoop uses the MapReduce mechanism for its operations which also supports fault tolerance.

Disadvantages of Sqoop :

- The failure occurs during the implementation of operation needed a special solution to handle the problem.

2.8 Hadoop I/O

- Hadoop comes with a set of primitives for data I/O.
- HDFS transparently checksums all data written to it and by default verifies checksums when reading data.

2.8.1 Compression

- File compression brings two major benefits: it reduces the space needed to store files, and it speeds up data transfer across the network, or to or from disk.
- When dealing with large volumes of data, both of these savings can be significant, so it pays to carefully consider how to use compression in Hadoop.
- There are many different compression formats, tools and algorithms, each with different characteristics. Table 2.2 lists some of the more common ones that can be used with Hadoop.

Compression format	Tool	Algorithm	Filename extension	Splittable
DEFLATE ^a	N/A	DEFLATE	<i>.deflate</i>	No
gzip	<i>gzip</i>	DEFLATE	<i>.gz</i>	No
bzip2	<i>bzip2</i>	bzip2	<i>.bz2</i>	Yes
LZO	<i>lzop</i>	LZO	<i>.lzo</i>	No ^b
Snappy	N/A	Snappy	<i>.snappy</i>	No

Table 2.2 summary of compression formats

- All compression algorithms exhibit a space/time trade-off: faster compression and decompression speeds usually come at the expense of smaller space savings.

2.8.2 Serialization

- Serialization is the process of turning structured objects into a byte stream for transmission over a network or for writing to persistent storage. Deserialization is the reverse process of turning a byte stream back into a series of structured objects.
- Serialization appears in two quite distinct areas of distributed data processing: for interprocess communication and for persistent storage.
- In Hadoop, interprocess communication between nodes in the system is implemented

using *remote procedure calls* (RPCs).

- The RPC protocol uses serialization to render the message into a binary stream to be sent to the remote node, which then deserializes the binary stream into the original message.

In general, it is desirable that an RPC serialization format is:

Compact

- A compact format makes the best use of network bandwidth, which is the most scarce resource in a data center.

Fast

- Interprocess communication forms the backbone for a distributed system, so it is essential that there is as little performance overhead as possible for the serialization and deserialization process.

Extensible

- Protocols change over time to meet new requirements, so it should be

straightforward to evolve the protocol in a controlled manner for clients and servers.

For example, it should be possible to add a new argument to a method call, and have the new servers accept messages in the old format (without the new argument) from old clients.

Interoperable

- For some systems, it is desirable to be able to support clients that are written in different languages to the server, so the format needs to be designed to make this possible.

2.8.3 Avro Serialization

- Apache Avro is a language-neutral data serialization system. The project was created by Doug Cutting (the creator of Hadoop) to address the major downside of Hadoop Writables: lack of language portability.
- Having a data format that can be processed by many languages (currently C, C++, C#, Java, Python, and Ruby) makes it easier to share datasets with a wider audience than one tied to a single language.
- It is also more future-proof, allowing data to potentially outlive the language used to read and write it.
- Avro data is described using a language-independent *schema*.
- Avro schemas are usually written in JSON, and data is usually encoded using a binary format, but there are other options, too.

- There is a higher-level language called Avro IDL, for writing schemas in a C-like language that is more familiar to developers.
- There is also a JSON-based data encoder, which, being human-readable, is useful for prototyping and debugging Avro data.
- The *Avro specification* precisely defines the binary format that all implementations must support.
- It also specifies many of the other features of Avro that implementations should support.
- Avro has rich *schema resolution* capabilities.
- Avro specifies an *object container format* for sequences of objects—similar to Hadoop’s sequence file.
- An *Avro data file* has a metadata section where the schema is stored, which makes the file self-describing.
- Avro data files support compression and are splittable, which is crucial for a MapReduce data input format.

Avro data types and schemas:

- Avro defines a small number of data types, which can be used to build application specific data structures by writing schemas. For interoperability, implementations must support all Avro types.

- Each Avro language API has a representation for each Avro type that is specific to the language.

In-memory serialization and deserialization:

- Avro provides APIs for serialization and deserialization, which are useful when you want to integrate Avro with an existing system, such as a messaging system where the framing format is already defined. In other cases, consider using Avro's data file format.

Java program to read and write Avro data to and from streams.

```
{
"type": "record",
"name": "StringPair",
"doc": "A pair of strings.",
"fields": [
{"name": "left", "type": "string"},
{"name": "right", "type": "string"}
]
}
```

If this schema is saved in a file on the classpath called *StringPair.avsc* (.avsc is the conventional extension for an Avro schema), then we can load it using the following two lines of code:

```
Schema.Parser parser = new Schema.Parser();
```

```
Schema schema = parser.parse(getClass().getResourceAsStream("StringPair.avsc"));
```

We can create an instance of an Avro record using the generic API as follows:

```
GenericRecord datum = new GenericData.Record(schema);
```

```
datum.put("left", "L");
```

```
datum.put("right", "R");
```

Next, we serialize the record to an output stream:

```
ByteArrayOutputStream out = new ByteArrayOutputStream();
```

```
DatumWriter<GenericRecord>writer=new  
GenericDatumWriter<GenericRecord>(schema);
```

```
Encoder encoder = EncoderFactory.get().binaryEncoder(out, null);
```

```
writer.write(datum, encoder);
```

```
encoder.flush();
```

```
out.close();
```

- There are two important objects here: the `DatumWriter` and the `Encoder`.
- A `DatumWriter` translates data objects into the types understood by an `Encoder`, which the latter writes to the output stream.

Avro data files:

- Avro's object container file format is for storing sequences of Avro objects. It is very similar in design to Hadoop's sequence files.
- A data file has a header containing metadata, including the Avro schema and a *sync marker*, followed by a series of (optionally compressed) blocks containing the serialized Avro objects.
- Blocks are separated by a sync marker that is unique to the file (the marker for a particular file is found in the header) and that permits rapid resynchronization with a block boundary after seeking to an arbitrary point in the file, such as an HDFS block boundary.
- Thus, Avro data files are splittable, which makes them amenable to efficient MapReduce processing.

A Python program for writing Avro record pairs to a data file

```
import os

import string

import sys

from avro import schema

from avro import io

from avro import datafile

if __name__ == '__main__':

    if len(sys.argv) != 2:

        sys.exit('Usage: %s <data_file>' % sys.argv[0])

    avro_file = sys.argv[1]

    writer = open(avro_file, 'wb')

    datum_writer = io.DatumWriter()

    schema_object = schema.parse("""\

{ "type": "record",

  "name": "StringPair",

  "doc": "A pair of strings.",

  "fields": [

    {"name": "left", "type": "string"},

    {"name": "right", "type": "string"}

  ]

}""")
```

```
dfw = datafile.DataFileWriter(writer, datum_writer, schema_object)

for line in sys.stdin.readlines():

    (left, right) = string.split(line.strip(), ',')

    dfw.append({'left':left, 'right':right});

dfw.close()
```

- The `avro_file_reader_read` function accepts a schema as its second argument to support the case where the schema for reading is different to the one used when the file was written (this is explained in the next section), but we simply pass in `NULL`, which tells Avro to use the data file’s schema.
- The third argument is a pointer to a `avro_datum_t` object, which is populated with the contents of the next record read from the file.

New schema	Writer	Reader	Action
Added field	Old	New	The reader uses the default value of the new field, since it is not written by the writer.
	New	Old	The reader does not know about the new field written by the writer, so it is ignored. (Projection).
Removed field	Old	New	The reader ignores the removed field. (Projection).
	New	Old	The removed field is not written by the writer. If the old schema had a default defined for the field, then the reader uses this, otherwise it gets an error. In this case, it is best to update the reader’s schema at the same time as, or before, the writer’s.

Table 2.6 Schema resolution of records

2.8.4 File-Based Data Structures:

- For some applications, you need a specialized data structure to hold your data. For doing MapReduce-based processing, putting each blob of binary data into its own

file doesn't scale, so Hadoop developed a number of higher-level containers for these situations.

SequenceFile:

- Imagine a logfile, where each log record is a new line of text. If you want to log binary types, plain text isn't a suitable format. Hadoop's SequenceFile class fits the bill in this situation, providing a persistent data structure for binary key-value pairs.
- To use it as a logfile format, you would choose a key, such as timestamp represented by a LongWritable, and the value is a Writable that represents the quantity being logged.
- SequenceFiles also work well as containers for smaller files.
- HDFS and MapReduce are optimized for large files, so packing files into a SequenceFile makes storing and processing the smaller files more efficient.

Writing a SequenceFile:

- To create a SequenceFile, use one of its createWriter() static methods, which returns a SequenceFile.Writer instance.

Example 1. Writing a SequenceFile

```
public class SequenceFileWriteDemo {  
private static final String[] DATA = {  
"One, two, buckle my shoe",  
"Three, four, shut the door",  
"Five, six, pick up sticks",  
"Seven, eight, lay them straight",  
"Nine, ten, a big fat hen"  
};
```

```

public static void main(String[] args) throws IOException {
    String uri = args[0];
    Configuration conf = new Configuration();
    FileSystem fs = FileSystem.get(URL.create(uri), conf);
    Path path = new Path(uri);
    IntWritable key = new IntWritable();
    Text value = new Text();
    SequenceFile.Writer writer = null;
    try {
        writer = SequenceFile.createWriter(fs, conf, path,
            key.getClass(), value.getClass());
        for (int i = 0; i < 100; i++) {
            key.set(100 - i);
            value.set(DATA[i % DATA.length]);
            System.out.printf("[%s]\t%s\t%s\n", writer.getLength(), key, value);
            writer.append(key, value);
        }
    } finally {
        IOUtils.closeStream(writer);
    }
}

```

Reading a SequenceFile:

- Reading sequence files from beginning to end is a matter of creating an instance of `SequenceFile.Reader` and iterating over records by repeatedly invoking one of the `next()` methods.

Example 2. Reading a SequenceFile

```
public class SequenceFileReadDemo {  
  
public static void main(String[] args) throws IOException {  
  
String uri = args[0];  
  
Configuration conf = new Configuration();  
  
FileSystem fs = FileSystem.get(URI.create(uri), conf);  
  
Path path = new Path(uri);  
  
SequenceFile.Reader reader = null;  
  
try {  
  
reader = new SequenceFile.Reader(fs, path, conf);  
  
Writable key = (Writable)  
ReflectionUtils.newInstance(reader.getKeyClass(), conf);  
  
Writable value = (Writable)  
ReflectionUtils.newInstance(reader.getValueClass(), conf);  
  
long position = reader.getPosition();  
  
while (reader.next(key, value)) {  
  
String syncSeen = reader.syncSeen() ? "*" : "";  
  
System.out.printf("[%s%s]\t%s\t%s\n", position, syncSeen, key, value);  
  
position = reader.getPosition(); // beginning of next record
```

```
}  
  
} finally {  
  
    IOUtils.closeStream(reader);  
  
}}
```

- There are two ways to seek to a given position in a sequence file. The first is the `seek()` method, which positions the reader at the given point in the file.

Sorting and merging SequenceFiles:

- The most powerful way of sorting (and merging) one or more sequence files is to use MapReduce. MapReduce is inherently parallel and will let you specify the number of reducers to use, which determines the number of output partitions.

The SequenceFile format

- A sequence file consists of a header followed by one or more records.
- The first three bytes of a sequence file are the bytes SEQ, which acts a magic number, followed by a single byte representing the version number.
- The header contains other fields including the names of the key and value classes, compression details, user-defined metadata, and the sync marker.

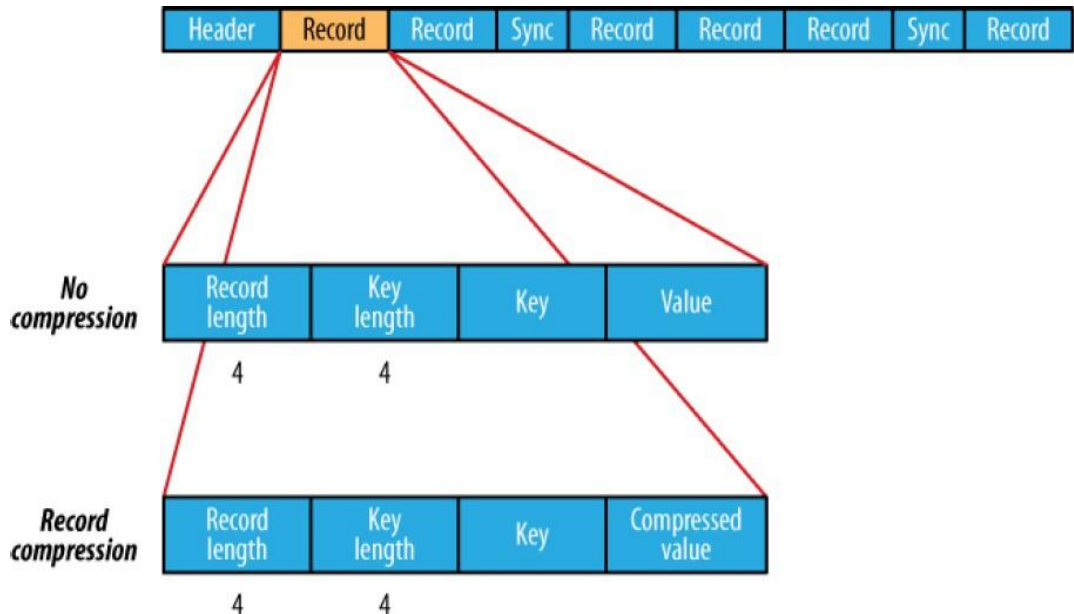


Figure 2.8 The internal structure of a sequence file with no compression and record compression

MapFile:

- A MapFile is a sorted SequenceFile with an index to permit lookups by key. MapFile can be thought of as a persistent form of `java.util.Map` (although it doesn't implement this interface), which is able to grow beyond the size of a Map that is kept in memory.

Writing a MapFile: Example:

```
public class MapFileWriteDemo {
    private static final String[] DATA = {
        "One, two, buckle my shoe",
        "Three, four, shut the door",
        "Five, six, pick up sticks",
        "Seven, eight, lay them straight",
    }
}
```

```

"Nine, ten, a big fat hen"

};

public static void main(String[] args) throws IOException {

String uri = args[0];

Configuration conf = new Configuration();

FileSystem fs = FileSystem.get(URL.create(uri), conf);

IntWritable key = new IntWritable();

Text value = new Text();

MapFile.Writer writer = null;

try {

writer = new MapFile.Writer(conf, fs, uri,

key.getClass(), value.getClass());

for (int i = 0; i < 1024; i++) {

key.set(i + 1);

value.set(DATA[i % DATA.length]);

writer.append(key, value);

}

} finally {

IOUtils.closeStream(writer);

}

}

}

```

UNIT III

MAP REDUCE PROGRAMMING

Map Reduce: Anatomy of a Map Reduce - Job Run – Failures – Job Scheduling – Shuffle and Sort – Task Execution – Map Reduce Types and Formats– Map Reduce Features - Composing map reduce calculations.

3.1 MAPREDUCE WORKFLOWS

MapReduce is a programming model and data processing technique that was popularized by Google and is widely used for processing and generating large datasets that can be parallelized across a distributed cluster of computers. MapReduce workflows consist of two main phases: the "Map" phase and the "Reduce" phase. These phases are typically orchestrated by a framework like Apache Hadoop. Here's an overview of the MapReduce workflow:

1. **Input Data:** The first step in a MapReduce workflow is to have a large dataset that you want to process. This dataset is typically stored in a distributed file system like Hadoop Distributed File System (HDFS).
2. **Map Phase:**
 - **Map Function:** In this phase, the input dataset is divided into smaller chunks called "splits," and each split is processed by a separate Mapper task. A user-defined "Map" function is applied to each record within the split. The Map function takes the input data and generates a set of key- value pairs as intermediate outputs. These key-value pairs are not sorted.
 - **Shuffling and Sorting:** After the Map phase, all the intermediate key-value pairs from all Mappers are grouped by key and sorted. This is necessary because the Reduce phase works on grouped data.

3. Shuffle and Sort Phase:

- In this phase, the framework sorts and shuffles the intermediate key-value pairs generated by the Mapper tasks. The purpose is to group all values associated with the same key together, which allows the Reducer tasks to process the data efficiently.

4. Reduce Phase:

- **Reduce Function:** The Reducer tasks take the sorted and shuffled key-value pairs from the Map phase and apply a user-defined "Reduce" function to process and aggregate the data. Each Reducer receives a group of key-value pairs with the same key. The Reduce function can perform various operations on the values, such as summarization, aggregation, or filtering.

- 5. Final Output:** After all the Reducer tasks have completed, the final result is available in the output location specified for the job.

MapReduce workflows are highly scalable and fault-tolerant, making them suitable for processing large-scale data in distributed computing environments. While the core concept of MapReduce remains the same, there are various implementations and tools available, such as Apache Hadoop and Apache Spark, that offer more features and flexibility for different data processing tasks. Additionally, developers need to design their Map and Reduce functions based on the specific requirements of their data processing tasks.

3.2 UNIT TESTS WITH MRUNIT

MRUnit is a Java library that helps you write unit tests for your Apache Hadoop MapReduce jobs. Unit testing your MapReduce code is crucial to ensure that your data processing logic works as expected before deploying it to a Hadoop cluster.

MRUnit provides a framework for creating and running unit tests for your Mapper and Reducer classes. Here's how you can write unit tests with MRUnit:

Set up your project:

First, you need to set up your Java project with the necessary dependencies, including MRUnit. You can typically include MRUnit in your project using Maven or by manually adding the MRUnit JAR file to your project's class path.

Write your unit test:

Next, you can create a unit test class for your Mapper or Reducer. Here's a basic example for testing a Mapper:

```
``java

import org.apache.hadoop.io.LongWritable; import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

import org.junit.Before;

import org.junit.Test;

public class MyMapperTest {

    private Mapper<LongWritable, Text, Text, IntWritable> mapper;

    @Before

    public void setUp() {

        MyMapper mapper = new MyMapper(); // Replace with your Mapper class
        mapper = Mapper.newMapper(mapper);

    }

    @Test

    public void testMapper() {

        mapper.withInput(new LongWritable(1), new Text("Hello, world!"))

        .withOutput(new Text("Hello,"), new IntWritable(1))

        .withOutput(new Text("world!"), new IntWritable(1))

        .runTest();

    }

}
```

In this example, we use MRUnit's `MapDriver` to test a Mapper. Replace `MyMapper` with the actual name of your Mapper class. We provide sample input data and expected output data and then run the test.

For Reducer tests, you can use `ReduceDriver` in a similar way.

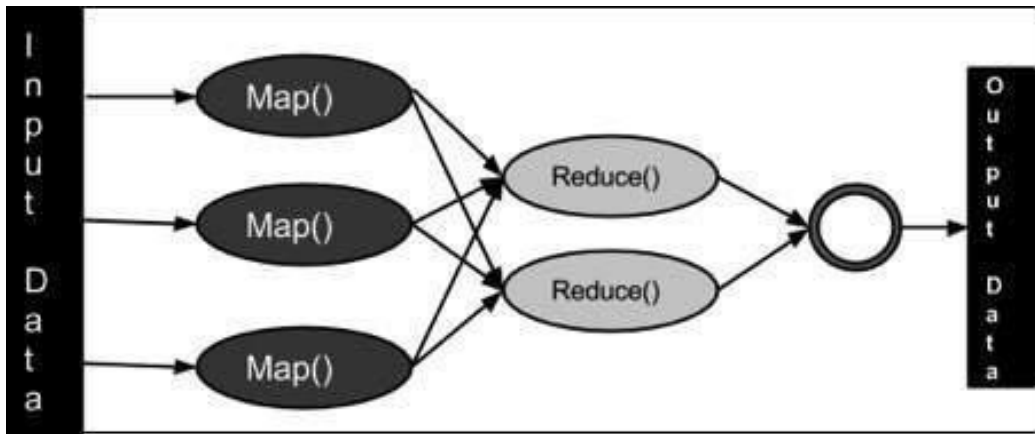
What is MapReduce?

MapReduce is a processing technique and a program model for distributed computing based on java. The MapReduce algorithm contains two important tasks, namely Map and Reduce. Map takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs). Secondly, reduce task, which takes the output from a map as an input and combines those data tuples into a smaller set of tuples. As the sequence of the name MapReduce implies, the reduce task is always performed after the map job.

The major advantage of MapReduce is that it is easy to scale data processing over multiple computing nodes. Under the MapReduce model, the data processing primitives are called mappers and reducers. Decomposing a data processing application into *mappers* and *reducers* is sometimes nontrivial. But, once we write an application in the MapReduce form, scaling the application to run over hundreds, thousands, or even tens of thousands of machines in a cluster is merely a configuration change. This simple scalability is what has attracted many programmers to use the MapReduce model.

The Algorithm

- Generally MapReduce paradigm is based on sending the computer to where the data resides!
- MapReduce program executes in three stages, namely map stage, shuffle stage, and reduce stage.
 - **Map stage** – The map or mapper's job is to process the input data. Generally the input data is in the form of file or directory and is stored in the Hadoop file system (HDFS). The input file is passed to the mapper function line by line. The mapper processes the data and creates several small chunks of data.
 - **Reduce stage** – This stage is the combination of the **Shuffle** stage and the **Reduce** stage. The Reducer's job is to process the data that comes from the mapper. After processing, it produces a new set of output, which will be stored in the HDFS.



3.1 Inputs and Outputs (Java Perspective)

The MapReduce framework operates on $\langle \text{key}, \text{value} \rangle$ pairs, that is, the framework views the input to the job as a set of $\langle \text{key}, \text{value} \rangle$ pairs and produces a set of $\langle \text{key}, \text{value} \rangle$ pairs as the output of the job, conceivably of different types.

The key and the value classes should be in serialized manner by the framework and hence, need to implement the Writable interface. Additionally, the key classes have to implement the Writable-Comparable interface to facilitate sorting by the framework. Input and Output types of a **MapReduce job** – (Input) $\langle k1, v1 \rangle \rightarrow \text{map} \rightarrow \langle k2, v2 \rangle \rightarrow \text{reduce} \rightarrow \langle k3, v3 \rangle$ (Output).

Input Output

Map $\langle k1, v1 \rangle$ list $\langle k2, v2 \rangle$

Reduce $\langle k2, \text{list}(v2) \rangle$ list $\langle k3, v3 \rangle$

Terminology

- **PayLoad** – Applications implement the Map and the Reduce functions, and form the core of the job.
- **Mapper** – Mapper maps the input key/value pairs to a set of intermediate key/value pair.
- **NamedNode** – Node that manages the Hadoop Distributed File System (HDFS).

- **DataNode** – Node where data is presented in advance before any processing takes place.
- **MasterNode** – Node where JobTracker runs and which accepts job requests from clients.
- **SlaveNode** – Node where Map and Reduce program runs.
- **JobTracker** – Schedules jobs and tracks the assign jobs to Task tracker.
- **Task Tracker** – Tracks the task and reports status to JobTracker.
- **Job** – A program is an execution of a Mapper and Reducer across a dataset.
- **Task** – An execution of a Mapper or a Reducer on a slice of data.
- **Task Attempt** – A particular instance of an attempt to execute a task on a SlaveNode.

3.3 TEST DATA AND LOCAL TESTS

When developing MapReduce jobs using Hadoop or similar frameworks, it's essential to perform unit testing to ensure that your code works correctly. One crucial aspect of unit testing is creating test data and running local tests to verify the behavior of your MapReduce jobs. Here's a step-by- step guide on how to create test data and conduct local tests for your MapReduce jobs:

Create Test Data:

Generate Sample Input Data: You can manually create small sample input data in a text file or use tools to generate synthetic data for testing purposes. Ensure that your test data covers various scenarios, including edge cases and potential issues that your MapReduce job might encounter.

Store Test Data in a Local Directory: Place your sample input data in a directory on your local machine. This directory will serve as the input source for your local tests.

Write Unit Tests:

- 1.1 Set Up a Testing Framework:** Use a testing framework like JUnit, TestNG, or a Hadoop- specific testing framework like MRUnit to write unit tests for your MapReduce job. Ensure that your project is properly configured to use the testing framework.
- 1.2 Write Test Cases:** Create test cases that cover various aspects of your MapReduce job, including Mapper and Reducer functionality, handling of different input data scenarios, and edge cases. Your test cases should include both positive and negative test scenarios.
- 1.3 Configure Test Inputs and Outputs:** In your test cases, specify the test input data, including the directory or file containing your sample input data. Define the expected output for each test case.
- 1.4 Invoke MapReduce Job:** In your test cases, invoke the MapReduce job by creating an instance of your Mapper and Reducer classes and running them with the test input data. Capture the output.
- 1.5 Assertions:** Use assertions to compare the actual output of your MapReduce job with the expected output defined in your test cases. Ensure that the results match your expectations.

Run Local Tests:

- 1.6 Execute Unit Tests:** Run your unit tests using the testing framework you've set up. The framework will execute your MapReduce job locally using the sample input data and validate the results.
- 1.7 Interpret Test Results:** Review the test results to identify any failures or issues. The test framework will report which test cases passed and which ones failed.
- 1.8 Debug and Refine:** If any test cases fail, use the debugging tools provided by your development environment to diagnose and fix the issues in your MapReduce code. Refine your code and tests iteratively until all test cases pass successfully.

Repeat for Different Test Scenarios:

1.9 Edge Cases: Make sure to test edge cases, such as empty input data, data with missing fields, or data that could potentially cause exceptions in your MapReduce code.

1.10 Performance Testing (Optional): If your MapReduce job is expected to process large volumes of data, you may want to test its performance with larger datasets. You can generate larger test data sets for such scenarios.

By creating test data and running local tests, you can catch and fix issues early in the development process, which helps ensure the correctness and reliability of your MapReduce jobs before deploying them to a Hadoop cluster for processing large datasets.

3.4 ANATOMY OF MAPREDUCE JOB RUN

The anatomy of a classic MapReduce job run involves several key components and stages. MapReduce is a programming model for processing and generating large datasets that can be parallelized across a distributed cluster of computers. Below, I'll outline the essential components and the sequence of steps involved in the execution of a classic MapReduce job:

1. Input Data:

- A MapReduce job begins with a large dataset that you want to process. This dataset is typically stored in a distributed file system like Hadoop Distributed File System (HDFS).

2. Mapper Phase:

- The input data is divided into smaller chunks or "splits."
- Each split is assigned to a mapper task. Mappers are responsible for processing these splits independently.
- The user-defined Mapper function is applied to each record within a split. The Mapper function emits key-value pairs as intermediate outputs.

3. Shuffling and Sorting:

- The intermediate key-value pairs generated by mappers are grouped by key across all the mappers.
- These groups of key-value pairs are then sorted by key. This step is crucial because it ensures that all values for a particular key end up in the same reducer.

4. Reducer Phase:

- Reducers are responsible for processing the grouped and sorted key-value pairs.
- User-defined Reducer functions are applied to each group of values associated with the same key.
- Reducers emit the final output, which is typically aggregated or transformed data.

5. Output Data:

- The output from the Reducer phase is written to an external storage system, often HDFS.
- This output represents the result of the MapReduce job, which can be used for further analysis, reporting, or other purposes.

6. Job Configuration:

- Before running the job, you need to configure various parameters, such as input and output paths, the number of mappers and reducers, and other job-specific settings.
- This configuration is typically done using a job configuration object, and it specifies how the input data should be processed.

7. Job Submission:

- Once the MapReduce job is configured, it is submitted to a cluster's resource manager (e.g., Hadoop YARN) for execution.
- The resource manager allocates resources (nodes) for running the job's tasks (mappers and reducers) based on cluster availability and job priorities.

8. Task Execution:

- Mapper and Reducer tasks are distributed across the cluster's nodes.
- Each task runs independently, processing its portion of the data.
- Task progress and status are monitored by the resource manager.

9. Job Monitoring and Logging:

- During the job run, logs and statistics are generated, which can be used for monitoring job progress, debugging, and performance analysis.

1. Job Completion:

- When all the tasks have completed successfully, the MapReduce job is considered finished.
- The final output data is available in the specified output directory in HDFS or the designated storage system.

2. Cleanup:

- After the job is completed, any temporary resources or data created during the job run may be cleaned up.

3. Results Analysis:

- The output data can be further analyzed or used for various purposes, such as generating reports, feeding into machine learning models, or populating databases.

3.5 CLASSIC MAP-REDUCE

This classic MapReduce workflow is the foundation for distributed data processing in frameworks like Hadoop. While it's a powerful and scalable approach, newer data processing frameworks like Apache Spark have gained popularity due to their improved performance and ease of use. However, the core concepts of MapReduce are still relevant in understanding distributed data processing.

"Classic MapReduce" refers to the original programming model and framework for processing and generating large datasets that can be parallelized across a distributed cluster of computers. It was popularized by Google in a 2004 paper written by Jeffrey Dean and Sanjay Ghemawat. Google used this model to process vast amounts of data across their infrastructure, and the ideas presented in the paper served as the foundation for open-source implementations like Apache Hadoop.

Here are the key concepts and components of the classic MapReduce model:

1. **Mapper:** The Mapper is responsible for processing input data and emitting a set of key-value pairs as intermediate data. Each Mapper task works independently on a subset of the input data.
2. **Reducer:** The Reducer takes the intermediate key-value pairs produced by the Mappers, groups them by key, and processes them to produce the final output. Reducers work in parallel, and each handles a different set of keys.
3. **Input Data:** The input data is typically a large dataset stored in a distributed file system like HDFS. It's divided into smaller chunks or "splits" that are processed by individual Mapper tasks.
4. **Output Data:** The output from the Reducer phase is written to an external storage system, often the distributed file system, and represents the result of the MapReduce job.
5. **Shuffling and Sorting:** After the Mapper phase, there is a shuffling and sorting step where the intermediate key-value pairs generated by Mappers are grouped by key and sorted. This step is crucial because it ensures that all values for a particular key end up in the same Reducer.
6. **Job Configuration:** Before running the job, you need to configure various parameters, such as input and output paths, the number of Mappers and Reducers, and other job-specific settings. This configuration is typically done using a job configuration object.
7. **Job Submission:** Once the MapReduce job is configured, it is submitted to a cluster's resource manager for execution. The resource manager allocates resources (nodes) for running the job's tasks based on cluster availability and job priorities.

8. **Task Execution:** Mapper and Reducer tasks are distributed across the cluster's nodes. Each task runs independently, processing its portion of the data. Task progress and status are monitored by the resource manager.
9. **Job Monitoring and Logging:** During the job run, logs and statistics are generated, which can be used for monitoring job progress, debugging, and performance analysis.
10. **Job Completion:** When all the tasks have completed successfully, the MapReduce job is considered finished. The final output data is available in the specified output directory in the distributed file system.

Classic MapReduce was groundbreaking because it allowed for the processing of vast amounts of data on commodity hardware in a fault-tolerant manner. While newer data processing frameworks like Apache Spark have gained popularity due to their improved performance and ease of use, the core concepts of MapReduce remain relevant and are foundational in understanding distributed data processing.

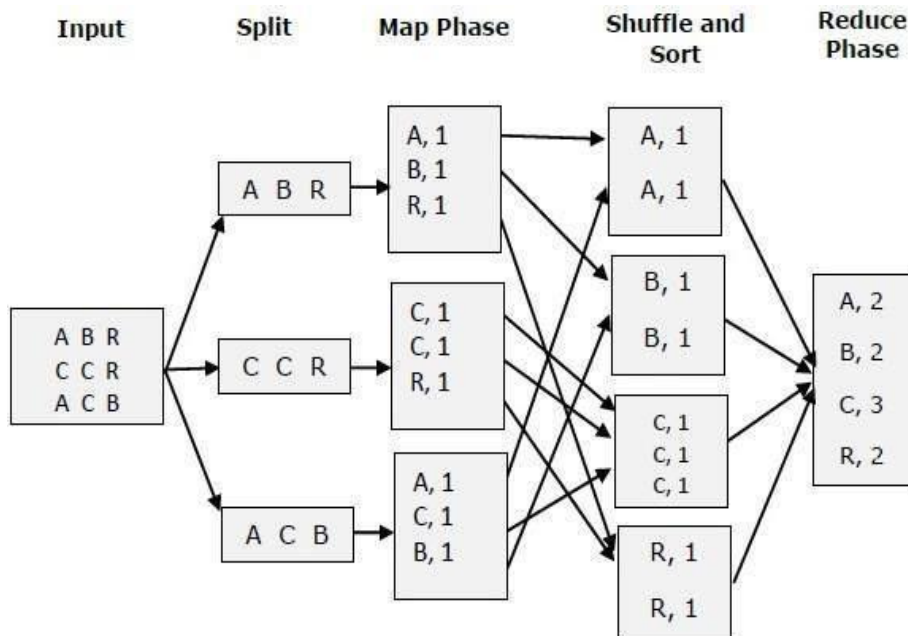
3.6 YARN

YARN is a resource management and job scheduling component in the Hadoop ecosystem. It is designed to separate the resource management layer from the processing framework, providing a more flexible and scalable resource management platform. YARN replaced the older

MapReduce Job Tracker and TaskTracker architecture, allowing Hadoop to support various processing frameworks beyond MapReduce.

Key components of YARN include:

1. **Resource Manager (RM):** The Resource Manager is the central resource allocation and management component in YARN. It manages cluster resources and schedules applications.
2. **Node Manager (NM):** Node Managers run on individual nodes in the cluster and are responsible for monitoring resource usage and reporting it back to the Resource Manager.
3. **Application Master (AM):** Each application running on the cluster has its own Application Master. The Application Master is responsible for negotiating resources with the Resource Manager and monitoring the execution of its application's tasks.



3.7 FAILURES IN CLASSIC MAP-REDUCE AND YARN

Failures can occur in both classic MapReduce and YARN due to various reasons. Handling failures robustly is a critical aspect of distributed computing. Here are some common types of failures in these frameworks:

Failures in Classic MapReduce:

- 1. Task Failures:** Individual Mapper or Reducer tasks can fail due to hardware issues, software errors, or other factors. Task failures are relatively common.
- 2. Job Tracker Failures:** In the classic MapReduce model, the Job Tracker is a single point of failure. If the Job Tracker fails, it can result in job failures or delays.
- 3. Data Node Failures:** If a Data Node hosting HDFS data block fails, it can lead to data unavailability for MapReduce jobs.
- 4. Network Failures:** Network issues can disrupt communication between nodes and cause job failures or delays.

Failures in YARN:

- 1. Resource Manager Failures:** The Resource Manager in YARN can be a single point of failure. If it fails, it can disrupt resource allocation and application scheduling.
- 2. Node Manager Failures:** Node Managers can fail, impacting the availability of resources on specific nodes.
- 3. Application Master Failures:** If an Application Master fails, the corresponding application may fail to manage its tasks, affecting the job's progress.
- 4. Container Failures:** Containers running on nodes can fail due to hardware or software issues, causing task failures.
- 5. Network Failures:** Network problems can disrupt communication between Resource Manager, Node Managers, and Application Masters.

To address these failures, Hadoop and YARN incorporate mechanisms for fault tolerance and recovery, including task retry, speculative execution, and high availability configurations for resource Manager and Name Node (in HDFS). Additionally, monitoring, logging, and alerting systems are often employed to detect and respond to failures in a timely manner.

It's essential to design and configure Hadoop clusters to handle failures gracefully to ensure the reliability and availability of big data processing jobs. More recent versions of these frameworks may have introduced additional features and improvements in failure handling and recovery.

3.8 JOB SCHEDULING

Job scheduling in the context of Hadoop MapReduce refers to the process of allocating resources and managing the execution of MapReduce jobs within a cluster. Proper job scheduling is crucial for efficient resource utilization and achieving good performance in a Hadoop cluster. Here are some key points related to job scheduling:

- 1. Resource Manager (YARN):** In the Hadoop ecosystem, YARN (Yet Another Resource Negotiator) is responsible for job scheduling. The Resource Manager (RM) is the central component that allocates cluster resources to various applications.

2. **Job Submission:** Users or applications submit MapReduce jobs to the Resource Manager. Each job specifies its resource requirements and priority.
3. **Resource Allocation:** The Resource Manager allocates containers (CPU, memory, etc.) on cluster nodes to run tasks for each job. It takes into account factors such as available resources, job priority, and fair allocation.
4. **Queue Management:** YARN supports the concept of queues to manage resource allocation. Users can be assigned to specific queues, and administrators can configure queue capacities and priorities.
5. **Fair Scheduler:** Hadoop provides a Fair Scheduler that aims to distribute resources fairly among multiple jobs. It assigns resources based on job weights and can be configured to ensure that no job starves for resources.
6. **Capacity Scheduler:** The Capacity Scheduler allows for resource allocation based on pre-defined capacities for different queues, making it suitable for multi-tenant clusters.
7. **Job Prioritization:** Jobs can be assigned different priorities, allowing higher-priority jobs to receive resources ahead of lower-priority ones.
8. **Job Progress Monitoring:** The Resource Manager monitors the progress of running jobs and detects job failures. It also handles task retries and re-allocations in case of task failures.
9. **Speculative Execution:** Hadoop can perform speculative execution of tasks, where a task is executed redundantly on different nodes to take advantage of available resources and complete jobs faster.
10. **Efficient job scheduling** ensures that cluster resources are used optimally, jobs complete in a timely manner, and different users or applications are allocated resources fairly.

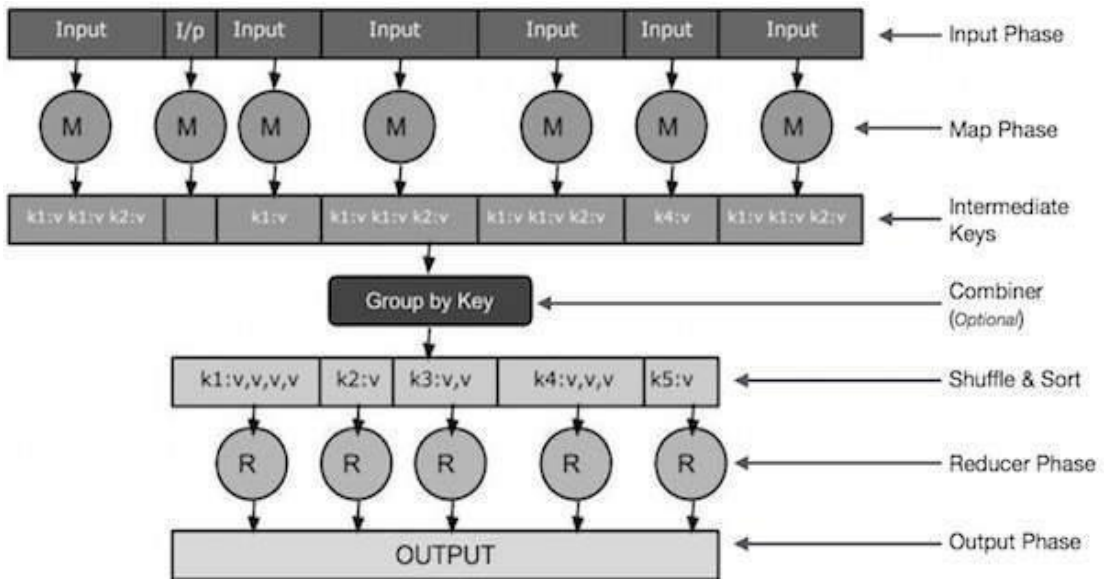
3.9 SHUFFLE AND SORT

Shuffle and Sort are critical phases in the MapReduce framework, occurring between the Mapper and Reducer phases. These phases are responsible for organizing and transferring data from Mapper tasks to Reducer tasks. Here's an overview of Shuffle and Sort:

1. **Mapper Output:** During the Mapper phase, each Mapper processes input data and generates key-value pairs as intermediate output. These key-value pairs are not yet in the final sorted order.
2. **Shuffling:** Shuffling is the process of transferring and grouping intermediate key-value pairs by key across all Mapper tasks. All values associated with the same key are collected together. This process is crucial because it ensures that all relevant data for a particular key ends up in the same Reducer task.
3. **Sorting:** After shuffling, the intermediate key-value pairs are sorted by key. Sorting is essential because it allows Reducer tasks to process data efficiently in key-sorted order. The sorting can be either by default, using the natural order of keys, or custom-defined if a comparator is provided.
4. **Reducer Input:** The sorted and grouped key-value pairs serve as the input to Reducer tasks. Each Reducer receives a group of key-value pairs with the same key, making it easier to perform aggregate operations or computations based on keys.
5. **Data Transfer:** Shuffling involves significant data transfer across the cluster, as data from multiple Mapper tasks must be collected, sorted, and distributed to Reducer tasks. Efficient data transfer is crucial for job performance.

The Shuffle and Sort phases are resource-intensive and can be a bottleneck in MapReduce jobs. Optimizing these phases, such as by minimizing data transfer or using combiners (mini-reducers) to pre-aggregate data during shuffling, can significantly improve job performance.

Overall, Shuffle and Sort ensure that MapReduce jobs can process and aggregate data efficiently in a distributed and parallelized manner, making them essential components of the MapReduce framework.



3.10 TASK EXECUTION

Task execution is a fundamental aspect of the MapReduce framework, where the actual data processing takes place. In a MapReduce job, tasks are executed in parallel across a cluster of nodes. There are two main types of tasks in a MapReduce job:

1. **Mapper Tasks:** These tasks are responsible for processing a portion of the input data. Mappers apply a user-defined function to each record within their assigned data split. The output of the Mapper is a set of intermediate key-value pairs.
2. **Reducer Tasks:** Reducer tasks take the intermediate key-value pairs generated by the Mappers and process them. Each Reducer receives a group of key-value pairs with the same key, and it applies a user-defined Reduce function to these pairs. The output of the Reducer is the final result of the MapReduce job.

Key points about task execution in MapReduce:

- Task execution is parallelized, with multiple Mapper and Reducer tasks running concurrently on different cluster nodes.
- Tasks are assigned to nodes based on data locality whenever possible, which reduces data transfer overhead.
- Task progress and status are monitored by the Resource Manager (in YARN) or the Job Tracker (in classic MapReduce).
- In the event of task failures, the Resource Manager or Job Tracker can reschedule tasks on available nodes.

Efficient task execution is critical for achieving good performance in MapReduce jobs, as it involves the actual processing of data.

3.11 MAPREDUCE TYPES

MapReduce jobs can be categorized into several types based on their use cases and data processing requirements. Here are some common types of MapReduce jobs:

1. Batch Processing:

- Batch processing MapReduce jobs are the most common type. They are used for processing large volumes of data in a batch mode.
- Examples include data cleaning, transformation, ETL (Extract, Transform, Load) processes, log analysis, and generating reports.

2. Iterative Processing:

- Some algorithms require multiple iterations over the data, such as machine learning algorithms (e.g., iterative clustering or graph algorithms).
- Iterative MapReduce jobs involve running the same Map and Reduce functions multiple times with modified input.

3. Real-time Processing (Streaming):

- Real-time MapReduce jobs process data as it arrives, making them suitable for stream processing.
- Examples include real-time fraud detection, social media sentiment analysis, and

4. Graph Processing: Graph algorithms, like PageRank or community detection, can be implemented using MapReduce. These jobs involve complex data transformations and traversals on graph data structures.

5. Data Joining:

- Data joining MapReduce jobs combine data from multiple sources based on common keys. Examples include merging user profiles from different data sets or joining logs with customer data.

6. Search and Indexing:

- MapReduce is used for building search engine indexes or performing large-scale text search operations.

7. Log Processing:

- Log processing jobs analyze server logs, application logs, or sensor data to extract insights, detect anomalies, or troubleshoot issues. The type of MapReduce job depends on the specific use case and data processing requirements. MapReduce's ability to scale horizontally and process vast amounts of data makes it a versatile choice for a wide range of big data processing tasks. However, it's worth noting that more recent frameworks like Apache Spark have gained popularity for certain use cases due to their improved performance and support for interactive and iterative processing.

3.12 INPUT FORMATS

Input formats in Hadoop MapReduce specify how input data is read and processed by the MapReduce job. Hadoop supports various input formats to handle different types of data sources and formats efficiently. Here are some common input formats:

1. **Text Input Format (Default):** This is the default input format for text-based data. Each line in the input file is treated as a separate record, and the key is the byte offset of the line, while the value is the content of the line.
2. **Key Value Text Input Format:** This format is used when input data is in the form of key-value pairs separated by a delimiter. It allows you to specify a delimiter character to split key and value pairs.
3. **Sequence File Input Format:** Sequence files are a binary format that stores key-value pairs. This input format is efficient for storing and processing large datasets.
4. **Avro, Parquet, and ORC Input Formats:** These formats are used for reading data stored in Avro, Parquet, or ORC file formats, which are columnar storage formats optimized for big data processing.
5. **Text Input Format (Custom Delimiter):** You can use the Text Input Format with a custom delimiter if your text data is not newline-separated but instead uses a different delimiter.
6. **Custom Input Formats:** You can create custom input formats to handle specialized data sources or formats not covered by the built-in input formats. This requires implementing specific logic for reading and parsing input data.

The choice of input format depends on the nature of your data and how it is stored. Hadoop allows you to configure the input format for your MapReduce job so that it can effectively read and process the data.

3.13 OUTPUT FORMATS

Output formats in Hadoop MapReduce specify how the results of a MapReduce job are written to the distributed file system or external storage. Similar to input formats, Hadoop provides various output formats to handle different data storage requirements. Here are some common output formats:

1. **Text Output Format (Default):** This is the default output format for text-based data. Each key-value pair is written as a line in the output file.

2. **Sequence File Output Format:** Sequence files can be used as an output format, especially when you want to store key-value pairs in a binary format for efficient storage and subsequent processing.
3. **Avro, Parquet, and ORC Output Formats:** These formats allow you to write data in Avro, Parquet, or ORC formats, which are optimized for efficient storage and query performance.
4. **Multiple Output Format:** This format allows you to write output to multiple files or directories based on specific criteria, such as keys or other attributes. It's useful for scenarios where you need to partition or distribute the output data.
5. **Custom Output Formats:** Just like with input formats, you can create custom output formats to write data to specialized storage systems or formats.

The choice of output format depends on how you want to store and use the results of your MapReduce job. Different formats offer trade-offs between storage efficiency, query performance, and ease of use. It's essential to select the appropriate output format to ensure that the processed data is readily accessible and fits your downstream processing or analysis needs.

Simple Programming Model

- You only need to write two functions: **Map** (process / filter / transform input into intermediate key-value pairs) and **Reduce** (aggregate / summarize results by key).
- The framework handles the rest (splitting data, scheduling tasks, handling failures).

Parallel and Distributed Processing

- Large datasets are split across many nodes; many map tasks run in parallel.
- After map, there's a shuffle & sort phase, then reduce tasks also execute in distributed fashion.

Scalability

- Can handle very large data volumes (terabytes to petabytes).

- New nodes can be added to a cluster to increase capacity without rewriting job logic.

Fault Tolerance

- Tasks or nodes may fail; MapReduce detects failures and reassigns work to other nodes.
- Input data is often replicated (e.g., in HDFS) so if some block is lost, another copy is available.

Data Locality

- The computation is pushed to where data resides to minimize network data transfer. I.e., map tasks are scheduled on nodes that hold the relevant data blocks.

High Throughput, Good for Batch Processing

- Designed to process large volumes of data in big batches. Good when latency isn't super tight, but you want robust, scalable processing.

Flexibility of Data Types and Formats

- Can handle structured, semi-structured, and unstructured data.
- Supports multiple data input/output formats. Users can define new formats or use standard ones.

Language-Independence

- While Hadoop MapReduce was originally Java-based, there are ways to write map/reduce jobs in other languages (Python, C++, etc.).

Load Balancing & Resource Utilization

- The framework divides tasks such that workload is spread out; scheduling ensures that nodes are used efficiently.
- There are features like speculative execution or backup tasks to cope with slow ("straggler") nodes.

Reliability & High Availability

- Data replication helps with reliability.
- Cluster components (such as Name Node in Hadoop) often have standby or redundant instances to avoid single point of failure.

3.14 Map Reduce Calculations

MapReduce Features

MapReduce is a programming model and software framework for processing large datasets in a distributed computing environment. Its key features, particularly relevant in big data contexts, include:

- **Scalability:**

MapReduce can effectively scale to handle massive datasets by distributing computation across a large cluster of commodity hardware. As data volumes grow, more nodes can be added to the cluster to maintain processing efficiency.

- **Fault Tolerance:**

It incorporates built-in mechanisms to handle node failures during processing. If a node fails, MapReduce automatically re-executes the failed tasks on other available nodes, ensuring data integrity and job completion.

- **Data Locality:**

MapReduce prioritizes processing data on the same node where it is stored in the Hadoop Distributed File System (HDFS). This minimizes data movement across the network, significantly improving performance for large-scale data processing.

- **Parallel Processing:**

The core of MapReduce lies in its ability to process data in parallel. Input data is split into independent chunks, and the "Map" phase processes these chunks concurrently across multiple nodes. The "Reduce" phase then aggregates the intermediate results.

- **Simplicity of Programming Model:**

While the underlying distributed computing is complex, MapReduce provides a relatively simple programming model based on user-defined "map" and "reduce" functions. Developers can focus on the data transformation logic without managing low-level distributed system details.

- **Cost-Effectiveness:**

By leveraging commodity hardware and an open-source framework like Hadoop, MapReduce offers a cost-effective solution for storing and processing extensive datasets compared to traditional enterprise data warehousing solutions.

- **Flexibility and Versatility:**

MapReduce can be applied to a wide range of data processing tasks, including data warehousing, data mining, search engine indexing, and real-time analytics, making it a versatile tool for various big data applications.

3.15 Composing map reduce calculations

This refers to how you build complex data processing pipelines using MapReduce. Some computations are simple (one map + reduce), others require multiple stages, aggregations, or combining intermediate results. You need to design your map & reduce functions such that they can compose well (i.e. combine intermediate outputs, allow incremental updates, etc.)

Principle	Explanation	Why It Matters
Reducer must operate on single key at a time	Each reducer receives all values for one key and outputs (usually) one result per key.	Ensures data locality, parallelization; limits what reduce can do.
Map tasks can only see one	Mappers output key/value pairs independently of others.	Imposes that you cannot compute many-record aggregations in a single map

Principle	Explanation	Why It Matters
record/input at a time		without passing to reduce or using combiners.
Intermediate data needs to be structured for composition	For example, computing average: instead of emitting average from each map, emit (sum, count) and combine those in the reducer.	If you just emit average per chunk, you can't combine averages correctly.

Examples / Patterns

Here are common cases and how they're composed:

1. Average per key

- Problem: Get average quantity sold per product.
- Wrong/simple: Each mapper emits average for its subset; reducer takes average of those → incorrect. Because averages are non-composable alone.
- Correct: Each mapper emits (key, (sum, count)). The reducer sums up all sums and all counts for each key, then calculates final average = total Sum / total Count.

2. Two-Stage MapReduce (Pipelining / Multi-Stage Aggregation) Useful when comparing over time (e.g. year-on-year, month-on-month sales growth) or when you need to transform/aggregate in steps.

- Stage 1: Aggregate raw data by (product, month, year) → compute total sales/count per product per month.
- Stage 2: Take output of stage 1; mappers tag whether record is from 2011 or prior years; reducer merges for each product+month: current year vs prior year data → compute growth, difference, ratio etc.

3. Counting

- Each map emits (key, 1) whenever a record matches the key grouping. Reducer sums these to count. Simple and composable.

4. Incremental / materialized view support

- When data is updated continuously, re-running full MapReduce, every time can be expensive. So design with incremental update in mind: only process new or changed data, merge with existing results. Requires map/reduce/combiners that can accommodate additive changes.

General Steps to Compose a Good MapReduce Calculation

1. Define the final outputs / KPIs

What do you need (e.g. average, difference, growth, ranking etc.)

2. Decide if it can be done in one map-reduce stage or needs multiple

If you need comparisons, temporal differences, or complex aggregates, often 2+ stages are better.

3. Design map outputs carefully

- Emit key(s) that allow grouping to get what you need.
- Emit value(s) in a structure that can be merged later (e.g. (sum, count), histograms, partial aggregates).

4. Use combiners if possible

Helps reduce intermediate data size, network load. Only works if reduce operation is associative & commutative.

5. Design reduce logic

- Combine intermediate values correctly.
- Compute final metric (average, growth, etc.).

UNIT IV

HADOOP ECOSYSTEM

Introduction to PIG – Execution Modes of Pig – Comparison of Pig with Databases – Grunt – Pig Latin – User Defined Functions – Data Processing Operators – HBase – Data Model and Implementations – HBase Clients – HBase Examples – Hive – Data Types and File Formats – HiveQL Data Definition – HiveQL Data Manipulation – HiveQL Queries.

4.1 Hadoop Ecosystem: Introduction to PIG

- Pig is a high-level platform or tool which is used to process large datasets. It provides a high level of abstraction for processing over MapReduce.
- It provides a high-level scripting language, known as Pig Latin which is used to develop the data analysis codes.
- Pig Latin and Pig Engine are the two main components of the Apache Pig tool. The result of Pig is always stored in the HDFS.
- One limitation of MapReduce is that the development cycle is very long. Writing the reducer and mapper, compiling packaging the code, submitting the job and retrieving the output is a time-consuming task.
- Apache Pig reduces the time of development using the multi-query approach. Pig is beneficial for programmers who are not from Java backgrounds.
- 200 lines of Java code can be written in only 10 lines using the Pig Latin language. Programmers who have SQL knowledge needed less effort to learn Pig Latin.

4.2 Execution Modes of Pig:

Apache Pig scripts can be executed in three ways:

Interactive Mode (Grunt shell):

- You can run Apache Pig in interactive mode using the Grunt shell.
- In this shell, you can enter the Pig Latin statements and get the output (using the Dump operator).

Batch Mode (Script) :

You can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with the **.pig** extension.

Embedded Mode (UDF) :

Apache Pig provides the provision of defining our own functions (User Defined Functions) in programming languages such as Java and using them in our script.

4.3 Comparison of Pig with Databases:

PIG	SQL
Pig Latin is a procedural language	SQL is a declarative language
In Apache Pig, the schema is optional. We can store data without designing a schema (values are stored as \$01, \$02 etc.)	Schema is mandatory in SQL.
The data model in Apache Pig is nested relational.	The data model used in SQL is flat relational.
Apache Pig provides limited opportunity for Query optimization.	There is more opportunity for query optimization in SQL.

4.4 Grunt:

- Grunt shell is a shell command.
- The Grunt shell of the Apache pig is mainly used to write pig Latin scripts.

- Pig script can be executed with grunt shell which is a native shell provided by Apache pig to execute pig queries.
- We can invoke shell commands using sh and fs.

Syntax of sh command:

Grunt>sh

ls

Syntax of fs command:

Grunt>fs -ls

4.5 Pig Latin :

The Pig Latin is a data flow language used by Apache Pig to analyze the data in Hadoop.

It is a textual language that abstracts the programming from the Java MapReduce idiom into a notation. The Pig Latin statements are used to process the data.

It is an operator that accepts a relation as an input and generates another relation as an output.

- It can span multiple lines.
- Each statement must end with a semi-colon.
- It may include expression and schemas.
- By default, these statements are processed using multi-query execution

4.6 User-Defined Functions:

Apache Pig provides extensive support for User Defined Functions (UDF's). Using these UDF's, we can define our own functions and use them. The UDF support is provided in six programming languages:

- Java
- Jython

- Python
- JavaScript
- Ruby
- Groovy

For writing UDF's, complete support is provided in Java and limited support is provided in all the remaining languages.

Using Java, you can write UDF's involving all parts of the processing like data load/store, column transformation, and aggregation.

Since Apache Pig has been written in Java, the UDF's written using Java language work efficiently compared to other languages.

Types of UDF's in Java :

Filter Functions :

- The filter functions are used as conditions in filter statements.
- These functions accept a Pig value as input and return a Boolean value.

Eval Functions :

- The Eval functions are used in FOREACH-GENERATE statements.
- These functions accept a Pig value as input and return a Pig result.

Algebraic Functions :

- The Algebraic functions act on inner bags in a FOREACHGENERATE statement.
- These functions are used to perform full MapReduce operations on an inner bag.

4.7 Data Processing Operators:

The Apache Pig Operators is a high-level procedural language for querying large data sets using Hadoop and the Map-Reduce Platform.

A Pig Latin statement is an operator that takes a relation as input and produces another relation as output. These operators are the main tools for Pig Latin provides to operate on the data.

They allow you to transform it by sorting, grouping, joining, projecting, and filtering. The Apache Pig operators can be classified as:

4.8. HBase

- HBase is an open source and sorted map data built on Hadoop. It is column oriented and horizontally scalable.
- It is based on Google's Big Table. It has a set of tables which keep data in key value format.
- HBase is well suited for sparse data sets which are very common in big data use cases.
- HBase provides APIs enabling development in practically any programming language.
- It is a part of the Hadoop ecosystem that provides random real-time read/write access to data in the Hadoop File System.

4.8.1. Features of HBase

- **Horizontally scalable:** You can add any number of columns at any time.
- **Automatic Failover:** Automatic failover is a resource that allows a system administrator to automatically switch data handling to a standby system in the event of system compromise.
- **Integrations with Map/Reduce framework:** All the commands and java codes internally implemented in Map/ Reduce to do the task and it is built over the Hadoop Distributed File System.

4.8.2. HBase Architecture:

HBase architecture has 3 main components: HMaster, Region Server, Zookeeper.

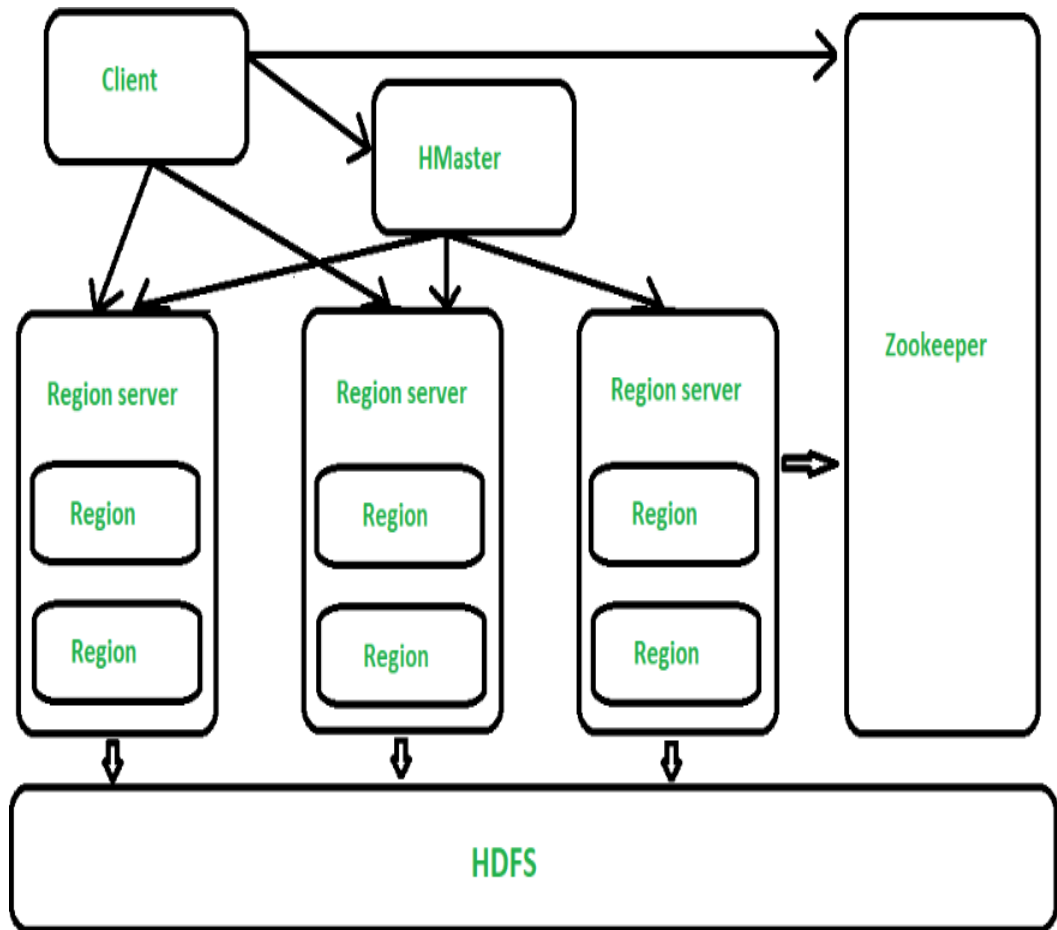


Figure 4.8.2.1: Architecture of HBase

All the 3 components are described below:

- **HMaster** – The implementation of Master Server in HBase is HMaster. It is a process in which regions are assigned to the region server as well as DDL (create, delete table) operations. It monitors all Region Server instances present in the cluster. In a distributed environment, Master runs several background threads. HMaster has many features like controlling load balancing, failover etc.

- **Region Server** – HBase Tables are divided horizontally by row key range into Regions. Regions are the basic building elements of HBase cluster that consists of the distribution of tables and are composed of Column families. Region Server runs on HDFS DataNode which is present in the Hadoop cluster. Regions of Region Server are responsible for several things, like handling, managing, executing as well as reads and writes of HBase operations on that set of regions. The default size of a region is 256 MB.
- **Zookeeper** – It is like a coordinator in HBase. It provides services like maintaining configuration information, naming, providing distributed synchronization, server failure notification etc. Clients communicate with region servers via zookeeper.

4.8.3 HDFS:

- HDFS is a Hadoop distributed File System, as the name implies it provides a distributed environment for the storage and it is a file system designed in a way to run on commodity hardware.
- It stores each file in multiple blocks and to maintain fault tolerance, the blocks are replicated across a Hadoop cluster.
- HDFS provides a high degree of fault –tolerance and runs on cheap commodity hardware. By adding nodes to the cluster and performing processing & storing by using the cheap commodity hardware, it will give the client better results as compared to the existing one.
- Here, the data stored in each block replicates into 3 nodes. In a case when any node goes down there will be no loss of data, it will have a proper backup recovery mechanism.
- HDFS gets in contact with the HBase components and stores a large amount of data in a distributed manner.

4.9 HBase Data Model

- HBase Data Model is a set of components that consists of Tables, Rows, Column families, Cells, Columns, and Versions.

- HBase tables contain column families and rows with elements defined as Primary keys.
- A column in the HBase data model table represents attributes to the objects.
- HBase Data Model consists of following elements,
 1. Set of tables: each table with column families and rows
 2. Each table must have an element defined as Primary Key.
 3. Row key acts as a Primary key in HBase.
 4. Any access to HBase tables uses this Primary Key
 5. Each column present in HBase denotes attribute corresponding to object.

4.9.1 Storage Mechanism in HBase

- HBase is a column-oriented database and data is stored in tables. The tables are sorted by RowId.
- As shown below, HBase has RowId, which is the collection of several column families that are present in the table.
- The column families that are present in the schema are key-value pairs. If we observe in detail each column family having multiple numbers of columns.
- The column values stored into disk memory. Each cell of the table has its own Metadata like timestamp and other information.

Coming to HBase the following are the key terms representing table schema

1. Table: collection of rows present.
2. Row: collection of column families.
3. Column family: collection of columns.
4. Column: collection of key-value pairs.

5. Namespace: logical grouping of tables.
6. Cell: a {row, column, version} tuple exactly specifies a cell definition in HBase.

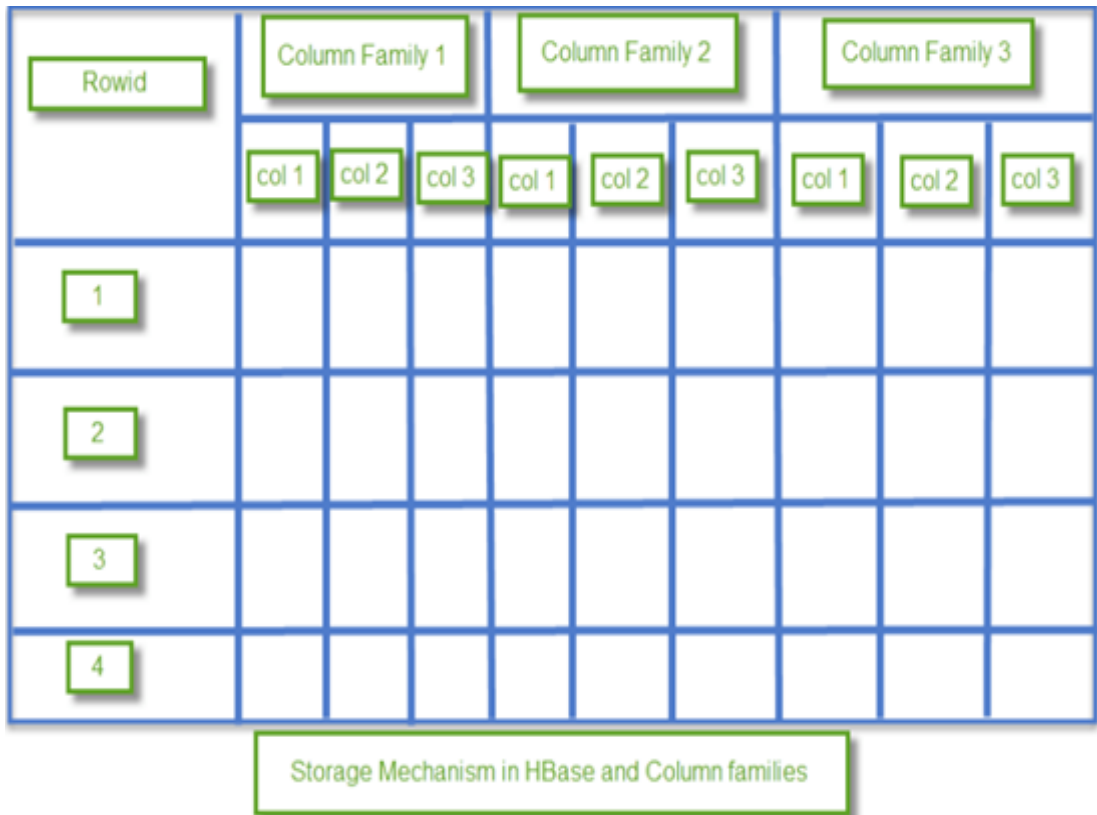


Figure 4.9.1.1: Storage Mechanism in HBase

4.9.2 Column-oriented Vs row-oriented storages

Column and row-oriented storages differ in their storage mechanism. As we all know traditional relational models store data in terms of row-based format like in terms of rows of data. Column-oriented storages store data tables in terms of columns and column families.

Row-oriented database	Column-oriented database
Data is stored and retrieved one row at a time and hence could read unnecessary data if some of the data in a row is required.	In this type of data store, data is stored and retrieved in columns and hence it is only able to read only the relevant data if required.
Records in row oriented data stores are easy to read and write.	In this type of data store, read and write operations are slower as compared to row-oriented.
Row-oriented data stores are best suited for online transaction systems.	Column-oriented stores are best suited for online analytical processing.
These are not efficient in performing operations applicable to the entire datasets and hence aggregation in row-oriented is an expensive job or operation.	These are efficient in performing operations applicable to the entire dataset and hence enable aggregation over many rows and columns.

4.10 HBase implementations

4.10.1 HBase Read and Write Data Explained

The read and write operations from client into hfile can be shown in the diagram below.

Step 1: Client wants to write data and in turn first communicates with regions server and then regions.

Step 2: Regions contacting memstore for storing associated with the column family.

Step 3: First data stores into memstore, where the data is sorted and after that, it flushes into hfile. The main reason for using memstore is to store data in a distributed file system based on row key. Memstore will be placed in region server main memory while hfiles are written into hdfs.

Step 4: Client wants to read data from regions

Step 5: In turn client can have direct access to mem store, and it can request for data.

Step 6: Client approaches hfiles to get the data. The data is fetched and retrieved by the client. Memstore holds in-memory modifications to the store.

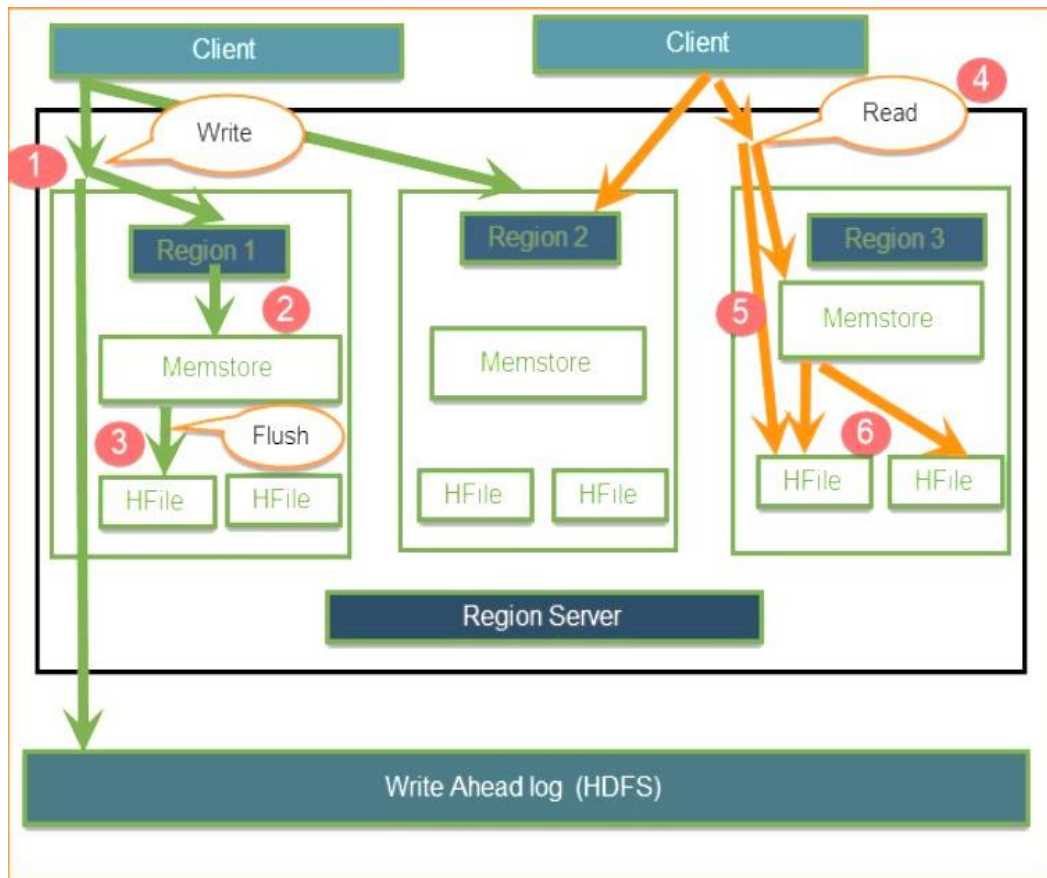


Figure 4.10.1.1: Read and Write Operations from Client into hfile

The hierarchy of objects in HBase regions is as shown from top to bottom in the below table.

Table	HBase table present in the HBase cluster
Region	Hregions for the presented tables
Store	It stores per column family for each region for the table
Memstore	• Memstore for each store for each region for the table. • It sorts data before flushing into hfiles. • Write and read performance will increase because of sorting.
Storefile	Storefiles for each store for each region for the table.
Block	Blocks present inside storefiles.

4.10.2 HBase vs. HDFS

HBase runs on top of hdfs and hadoop. Some key differences between hdfs and HBase are in terms of data operations and processing.

HBASE	HDFS
Low latency operations	High latency operations
Random reads and writes	Write once read many times
Accessed through shell commands, client api in java, rest, avro or thrift	Primarily accessed through mr (map reduce) jobs
Storage and process both can be perform	It's only for storage areas

4.11 HBase clients:

- There are a number of client options for interacting with an HBase cluster.

- Java HBase, like hadoop, is written in java.
- Basic table administration and access.

```

HTable table = new HTable(config, tableName);
try {
    for (int i = 1; i <= 3; i++) {
        byte[] row = Bytes.toBytes("row" + i);
        Put put = new Put(row);
        byte[] columnFamily = Bytes.toBytes("data");
        byte[] qualifier = Bytes.toBytes(String.valueOf(i));
        byte[] value = Bytes.toBytes("value" + i);
        put.add(columnFamily, qualifier, value);
        table.put(put);
    }

    Get get = new Get(Bytes.toBytes("row1"));
    Result result = table.get(get);
    System.out.println("Get: " + result);
    Scan scan = new Scan();
    ResultScanner scanner = table.getScanner(scan);
    try {
        for (Result scannerResult : scanner) {
            System.out.println("Scan: " + scannerResult);
        }
    } finally {
        scanner.close();
    }
    // Disable then drop the table
    admin.disableTable(tableName);
    admin.deleteTable(tableName);
} finally {
    table.close();
}
} finally {
    admin.close();
}
}

```

- The class has a `main()` method and uses the `HBaseConfiguration` class to create a configuration object that reads the HBase configuration from the `HBase-site.xml` and `HBase-default.xml` files.
- Finally, the code disables and deletes the table. The code makes use of HBase's bytes utility class to convert identifiers and values to the byte arrays that HBase requires.

Praxis:

- Praxis is a project in HBase that aims to provide a common data ingestion framework.
- Its main role is to import data from many different data sources (such as relational databases, file systems, web pages, etc.) Into HBase for analysis and processing.
- With praxis, users can define the mapping between the data source and the target HBase table with simple configuration, and specify data transformation and filtering rules.
- Praxis also provides a set of tools and apis for data import, making it easy to import unstructured or structured data into HBase. Best practices for using praxis include:
 1. Before using praxis, make sure you have a clear understanding of the structure and contents of the data source and how to map it to HBase tables.
 2. When writing configuration files, try to keep the configuration concise and easy to understand. You can manipulate data in a data source by using regular expressions, filters, and so on.
 3. In the process of data import, you can use the components and apis provided by praxis to realize incremental import or full import of data to meet different needs.
 4. Pay attention to the format and encoding of the data source to ensure that the data can be correctly converted and imported into HBase.

4.12 Hive

- Apache Hive is a data warehouse software project that is built on top of the Hadoop ecosystem.
- It provides an SQL-like interface to query and analyze large datasets stored in Hadoop's distributed file system (HDFS) or other compatible storage systems.
- Hive uses a language called HiveQL, which is similar to SQL, to allow users to express data queries, transformations, and analyses in a familiar syntax.
- HiveQL statements are compiled into MapReduce jobs, which are then executed on the Hadoop cluster to process the data.
- Hive includes many features that make it a useful tool for big data analysis, including support for partitioning, indexing, and user-defined functions (UDFs).
- It also provides a number of optimization techniques to improve query performance, such as predicate pushdown, column pruning, and query parallelization. Hive can be used for a variety of data processing tasks, such as data warehousing, ETL (extract, transform, load) pipelines, and ad-hoc data analysis.

4.12.1 Architecture of Hive:

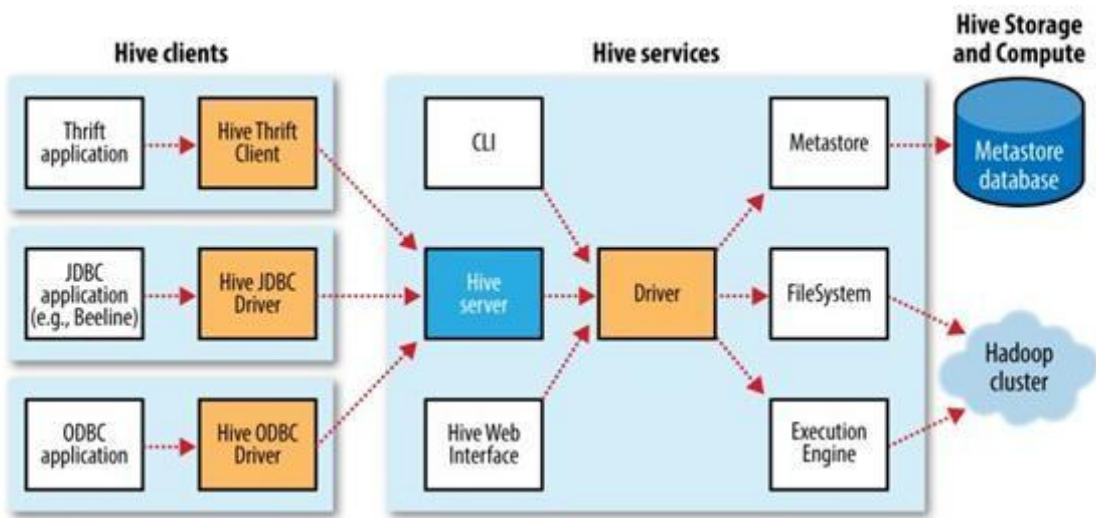


Figure 4.12.1.1: Hive Architecture

Hive chiefly consists of three core parts:

- **Hive Clients:** Hive offers a variety of drivers designed for communication with different applications. For example, Hive provides Thrift clients for Thrift-based applications. These clients and drivers then communicate with the Hive server, which falls under Hive services.
- **Hive Services:** Hive services perform client interactions with Hive. For example, if a client wants to perform a query, it must talk with Hive services.
- **Hive Storage and Computing:** Hive services such as file system, job client, and meta store then communicates with Hive storage and stores things like metadata table information and query results.
- **The Metastore:** The metastore is the central repository of Hive metadata. The metastore is divided into two pieces: a service and the backing store for the data. By default, the metastore service runs in the same JVM as the Hive service and contains an embedded Derby database instance backed by the local disk. This is called the embedded metastore configuration. Using an embedded metastore is a simple way to get started with Hive; however, only one embedded Derby database can access the database files on disk at any one time, which means you can have only one Hive session open at a time that accesses the same metastore. Trying to start a second session produces an error when it attempts to open a connection to the metastore.

Features of Hive

- Hive is fast and scalable.
- It provides SQL-like queries (i.e., HQL) that are implicitly transformed to MapReduce or Spark jobs.
- It is capable of analyzing large datasets stored in HDFS.
- It allows different storage types such as plain text, RCFile, and HBase.
- It uses indexing to accelerate queries.
- It can operate on compressed data stored in the Hadoop ecosystem.

- It supports user-defined functions (UDFs) where users can provide its functionality.

Limitations of Hive

- Hive is not capable of handling real-time data.
- It is not designed for online transaction processing.
- Hive queries contain high latency.

Differences between Hive and Pig

Hive	Pig
Hive is commonly used by Data Analysts.	Pig is commonly used by programmers.
It follows SQL-like queries.	It follows the data-flow language.
It can handle structured data.	It can handle semi-structured data.
It works on the server-side of the HDFS cluster.	It works on the client-side of the HDFS cluster.
Hive is slower than Pig.	Pig is comparatively faster than Hive.

4.13 Hive Data Types:

Hive supports both primitive and complex data types. Primitives include numeric, Boolean, string, and timestamp types. The complex data types include arrays, maps, and structs.

Primitive types:

- BOOLEAN type for storing true and false values.
- There are four signed integral types: TINYINT, SMALLINT, INT, and BIGINT, which are equivalent to Java's byte, short, int, and long primitive types, respectively (they are 1-byte, 2-byte, 4-byte, and 8-byte signed integers).

- Hive's floating-point types, FLOAT and DOUBLE, correspond to Java's float and double, which are 32-bit and 64-bit floating-point numbers.
- The DECIMAL data type is used to represent arbitrary-precision decimals. DECIMAL values are stored as unscaled integers.
- There are three Hive data types for storing text. STRING is a variable-length character string with no declared maximum length. (The theoretical maximum size STRING that may be stored is 2 GB.
- The BINARY data type is for storing variable-length binary data.
- The TIMESTAMP data type stores timestamps with nanosecond precision. Hive comes with UDFs for converting between Hive timestamps, Unix timestamps (seconds since the Unix epoch), and strings, which makes most common date operations tractable.
- The DATE data type stores a date with year, month, and day components.

Complex types:

- Hive has four complex types: ARRAY, MAP, STRUCT, and UNION. ARRAY and MAP.
- STRUCT is a record type that encapsulates a set of named fields.
- A UNION specifies a choice of data types; values must match exactly one of these types.
- Complex types permit an arbitrary level of nesting. Complex type declarations must specify the type of the fields in the collection, using an angled bracket notation.

➤ Hive Data Types

Category	Type	Description	Literal examples
Primitive	BOOLEAN	True/false value.	TRUE
	TINYINT	1-byte (8-bit) signed integer, from -128 to 127.	1Y
	SMALLINT	2-byte (16-bit) signed integer, from -32,768 to 32,767.	1S
	INT	4-byte (32-bit) signed integer, from -2,147,483,648 to 2,147,483,647.	1
	BIGINT	8-byte (64-bit) signed integer, from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.	1L
	FLOAT	4-byte (32-bit) single-precision floating-point number.	1.0
	DOUBLE	8-byte (64-bit) double-precision floating-point number.	1.0
	DECIMAL	Arbitrary-precision signed decimal number.	1.0
	STRING	Unbounded variable-length character string.	'a', "a"
	VARCHAR	Variable-length character string.	'a', "a"
	CHAR	Fixed-length character string.	'a', "a"
	BINARY	Byte array.	Not supported
	TIMESTAMP	Timestamp with nanosecond precision.	1325502245000, '2012-01-02 03:04:05.123456789'
	DATE	Date.	'2012-01-02'

Category	Type	Description	Literal examples
Complex	ARRAY	An ordered collection of fields. The fields must all be of the same type.	array(1, 2) ^a
	MAP	An unordered collection of key-value pairs. Keys must be primitives; values may be any type. For a particular map, the keys must be the same type, and the values must be the same type.	map('a', 1, 'b', 2)
	STRUCT	A collection of named fields. The fields may be of different types.	struct('a', 1, 1.0), named_struct('col1', 'a', 'col2', 1, 'col3', 1.0)
	UNION	A value that may be one of a number of defined data types. The value is tagged with an integer (zero-indexed) representing its data type in the union.	create_union(1, 'a', 63)

4.14 Hive File Formats:

Apache Hive supports several familiar file formats used in Apache Hadoop. Hive can load and query different data files created by other Hadoop components such as Pig or MapReduce. Different file formats and compression codecs work better for different data sets in Apache Hive. Following are the Apache Hive different file formats:

- Text File
- Sequence File
- RC File
- AVRO File
- ORC File
- Parquet File
- **Hive Text File Format**
 - Hive Text file format is a default storage format. You can use the text format to interchange the data with other client applications. The text file format is very common in most of the applications. Data is stored in lines, with each line being a record. Each line is terminated by a newline character (`\n`).
 - The text format is a simple plane file format. You can use the compression (BZIP2) on the text file to reduce the storage spaces.
 - Create a TEXT file by adding the storage option as ‘STORED AS TEXTFILE’ at the end of a Hive CREATE TABLE command.
 - Example:

Create table `textfile_table` (column_specs) stored as textfile;

- **Hive Sequence File Format:**
 - Sequence files are Hadoop flat files which store values in binary key-value pairs. The sequence files are in binary format and these files are able to split. The main advantage of using sequence files is to merge two or more files into one file.

- Create a sequence file by adding the storage option as ‘STORED AS SEQUENCEFILE’ at the end of a Hive CREATE TABLE command.
- Example:

Create table sequencefile_table (column_specs) stored as sequencefile;

➤ **Hive RC File Format**

- RCFile is row columnar file format. This is another form of Hive file format which offers high row level compression rates. If you have a requirement to perform multiple rows at a time then you can use RCFile format.
- The RCFile is very much similar to the sequence file format. This file format also stores the data as key-value pairs.
- Create RCFile by specifying ‘STORED AS RCFILE’ option at the end of a CREATE TABLE Command
- Example:

Create table RCfile_table (column_specs) stored as rcfile;

➤ **Hive AVRO File Format**

- AVRO is an open source project that provides data serialization and data exchange services for Hadoop. You can exchange data between the Hadoop ecosystem and programs written in any programming language. Avro is one of the popular file formats in Big Data Hadoop based applications.
- Create AVRO file by specifying ‘STORED AS AVRO’ option at the end of a CREATE TABLE Command.
- Example:

Create table avro_table (column_specs) stored as avro;

➤ **Hive ORC File Format**

- The ORC file stands for Optimized Row Columnar file format. The ORC file format provides a highly efficient way to store data in the Hive table. This file

system was actually designed to overcome limitations of the other Hive file formats. The Use of ORC files improves performance when Hive is reading, writing, and processing data from large tables.

- Create an ORC file by specifying ‘STORED AS ORC’ option at the end of a CREATE TABLE Command.
- Example:

Create table orc_table (column_specs) stored as orc;

➤ **Hive Parquet File Format**

- Parquet is a column-oriented binary file format. The parquet is highly efficient for the types of large-scale queries. Parquet is especially good for queries scanning particular columns within a particular table. The Parquet table uses compression Snappy, gzip; currently Snappy by default.
- Create a Parquet file by specifying ‘STORED AS PARQUET’ option at the end of a CREATE TABLE Command.
- Example:

Create table parquet_table (column_specs) stored as parquet;

4.15 HiveQL: Data definition

- HiveQL is the Hive query language. Like all SQL dialects in widespread use, it doesn’t fully conform to any particular revision of the ANSI SQL standard.
- It is perhaps closest to MySQL’s dialect, but with significant differences. Hive offers no support for row-level inserts, updates, and deletes.
- Hive doesn’t support transactions. Hive adds extensions to provide better performance in the context of Hadoop and to integrate with custom extensions and even external programs.

Databases in Hive:

- The Hive concept of a database is essentially just a catalog or namespace of tables. However, they are very useful for larger clusters with multiple teams and users,

DDL Commands On Databases in Hive

1. *CREATE DATABASE in Hive*

The **CREATE DATABASE** statement is used to create a database in the Hive. The DATABASE and SCHEMA are interchangeable. We can use either DATABASE or SCHEMA.

Syntax:

```
CREATE (DATABASE|SCHEMA) [IF NOT EXISTS] database_name  
[COMMENT database_comment]  
[LOCATION hdfs_path]  
[WITH DBPROPERTIES (property_name=property_value, ...)];
```

2. *SHOW DATABASE in Hive*

The **SHOW DATABASES** statement lists all the databases present in the Hive.

Syntax:

```
SHOW (DATABASES|SCHEMAS);
```

3. *DESCRIBE DATABASE in Hive*

The **DESCRIBE DATABASE** statement in Hive shows the name of Database in Hive, its comment (if set), and its location on the file system.

The **EXTENDED** can be used to get the database properties.

Syntax:

```
DESCRIBE DATABASE/SCHEMA [EXTENDED] db_name;
```

as a way of avoiding table name collisions. It's also common to use databases to organize production tables into logical groups.

4.USE DATABASE in Hive

The **USE** statement in Hive is used to select the specific database for a session on which all subsequent HiveQL statements would be executed.

Syntax:

```
USE database_name;
```

5.DROP DATABASE in Hive

The **DROP DATABASE** statement in Hive is used to Drop (delete) the database. The default behavior is **RESTRICT** which means that the database is dropped only when it is empty. To drop the database with tables, we can use **CASCADE**.

Syntax:

```
DROP (DATABASE|SCHEMA) [IF EXISTS] database_name  
[RESTRICT|CASCADE];
```

6.ALTER DATABASE in Hive

The **ALTER DATABASE** statement in Hive is used to change the metadata associated with the database in Hive. **Syntax for changing Database Location:**

```
ALTER (DATABASE|SCHEMA) database_name SET LOCATION hdfs_path;
```

Syntax for changing Database Properties:

```
ALTER (DATABASE|SCHEMA) database_name SET DBPROPERTIES  
(property_name=property_value, ...);
```

Syntax for changing Database owner:

```
ALTER (DATABASE|SCHEMA) database_name SET OWNER [USER|ROLE]  
user_or_role;
```

4.16 Hive: Data manipulation

Introduction to Hive DML commands

- Hive DML (Data Manipulation Language) commands are used to insert, update, retrieve, and delete data from the Hive table once the table and database schema has been defined using Hive DDL commands.
- The various Hive DML commands are:

- LOAD
- SELECT
- INSERT
- DELETE
- UPDATE
- EXPORT
- IMPORT

LOAD

The **LOAD** statement in Hive is used to move data files into the locations corresponding to Hive tables.

- If a **LOCAL** keyword is specified, then the LOAD command will look for the file path in the local filesystem.
- If the **LOCAL** keyword is not specified, then the Hive will need the absolute URI of the file.
- In case the keyword **OVERWRITE** is specified, then the contents of the target table/partition will be deleted and replaced by the files referred by filepath.
- If the **OVERWRITE** keyword is not specified, then the files referred by filepath will be appended to the table.

Syntax:

```
LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE
tablename [PARTITION (partcol1=val1, partcol2=val2 ...)];
```

The **SELECT** statement in Hive is similar to the **SELECT** statement in SQL used for retrieving data from the database.

Syntax:

```
SELECT col1,col2 FROM tablename;
```

INSERT

*The **INSERT** command in Hive loads the data into a Hive table.*

INSERT OVERWRITE

The **INSERT OVERWRITE** table overwrites the existing data in the table or partition.

Syntax:

INSERT OVERWRITE TABLE tablename1 [PARTITION (partcol1=val1, ..) **[IF NOT EXISTS)]** select_statement FROM from_statement;

a. INSERT INTO

The **INSERT INTO** statement appends the data into existing data in the table or partition. **INSERT INTO** statement works from Hive version 0.8.

Syntax:

INSERT INTO TABLE tablename1 [PARTITION (partcol1=val1, partcol2=val2 ...)] select_statement1 FROM from_statement;

b.INSERT .. VALUES

INSERT ..VALUES statement in Hive inserts data into the table directly from SQL. It is available from Hive 0.14.

Syntax:

INSERT INTO TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...)] **VALUES** values_row [, values_row ...];
Hive deletes the table data. If the **WHERE** clause is specified, then it deletes the rows that satisfy the condition in where clause.

DELETE

The **DELETE** statement can only be used on the hive tables that support ACID.

Syntax:

DELETE FROM tablename [WHERE expression];

[[=val1], partcol2[=val2] ...)] **VALUES** values_row [, values_row ...];

The update can be performed on the hive tables that support ACID.

Syntax:

DELETE FROM tablename [WHERE expression];

l1[=val1], partcol2[=val2] ...)] VALUES values_row [, values_row ...];

The update can be performed on the hive tables that support ACID.

UPDATE

The UPDATE statement in Hive deletes the table data. If the WHERE clause is specified, then it updates the column of the rows that satisfy the condition in WHERE clause.

Partitioning and **Bucketing** columns cannot be updated.

Syntax:

UPDATE tablename SET column = value [, column = value ...] [WHERE expression];

EXPORT

The Hive **EXPORT** statement exports the table or **partition** data along with the metadata to the specified output location in the HDFS.

Metadata is exported in a **_metadata** file, and data is exported in a subdirectory **'data.'**

Syntax:

EXPORT TABLE tablename [PARTITION (part_column="value"[, ...])] TO 'export_target_path' [**FOR** replication('eventid')];

IMPORT

The Hive IMPORT command imports the data from a specified location to a new table or already existing table.

Syntax:

IMPORT [[EXTERNAL] TABLE new_or_original_tablename [PARTITION (part_column="value"[, ...])]] FROM 'source_path' [LOCATION 'import_target_path'];

4.17 Hiveql queries

➤ Creation and loading of data into a table

The bulk load operation is used to insert data into managed tables as Hive does not support row-level insert, delete or update.

Code:

```
LOAD DATA LOCAL INPATH '$Home/students_address' OVERWRITE INTO  
TABLE students PARTITION (class = "12", section = "science");
```

With the above command, a directory is first created for the partition, and then all the files are copied in the directory. The keyword “local” is used to specify that the data is present in the local file system. “Partition” keyword can be omitted if the table does not have a partition key. Hive query will not check for the data being loaded to match the schema of the table.

The “INSERT” command is used to load data from a query into a table. “OVERWRITE” keyword is used to replace the data in a table. In Hive v0.8.0 or later, data will get appended into a table if overwrite keyword is omitted.

Code:

```
INSERT OVERWRITE TABLE students PARTITION (class = "12", section =  
"science")
```

```
Select * from students_data where class = "12" and section = "science";
```

All the partitions of the table students_data can be dynamically inserted by setting below properties:

- Set hive.exec.dynamic.partition = True;
- Set hive.exec.dynamic.partition.mode = unstrict
- Set hive.exec.max.dynamic.partition.pernode = 1000;

CREATE TABLE clause will also create a table, and schema will be taken from the select clause.

➤ Merge data in tables

Data can be merged from tables using classic SQL joins like inner, full outer, left, right join.

Code:

Select a.roll_number, class, section from students as a

inner join pass_table as b on a.roll_number = b.roll_number;

This will return class and section of all the roll numbers who have passed. Using a left join to this will return the “grade” for only pass students and “NULL” for the failed ones.

Code:

Select a.roll_number, class, section, b.grade from students as a

Left join pass_table as b on a.roll_number = b.roll_number;

UNION ALL and UNION are also used to append data present in two tables. However, few things need to be taken care of on doing so like, Schema of both the tables should be same. UNION is used to append the table and return unique records while UNION ALL returns all the records, including duplicates.

➤ Ordering a table

ORDER BY clause enables total ordering of the data set by passing all data through one reducer. This may take a long time for large data tables, so SORT BY clause can be used to achieve partial sorting, by sorting each reducer.

Code:

Select customer_id, spends from customer as a order by spends DESC limit 100

This will return the top 100 customers with highest spends.

➤ Aggregation of data in a table

Aggregation is done using aggregate functions that returns a single value after doing computation on many rows. These are count(col), sum(col), avg(col), min(col), max(col), stddev_pop(col), percentile_approx(int_expr, P, NB), where NB is number of histogram

bins for estimation), collect_set(col), this returns duplicate elements after removing collection column.

The set property which helps in improving the performance of aggregation is hive.map.aggr = true.

“GROUP BY” clause is used with an aggregate function.

Code:

```
Select year(date_yy), avg(spends) from customer_spends where merchant = "Retail"
group by year(date_yy)
```

HAVING clause is used to restrict the output from GROUP BY, which is done using a subquery.

➤ **Conditional statements**

CASE...WHEN...THEN clause is similar to if-else statements to perform a conditional operation on any column in a query.

Code:

```
Select customer, Case when percentage <40 then "Fail"
When percentage >=40 and percentage <80 then "Average" Else "Excellent"
End as rank From students;
```

UNIT V

DATA STREAMS

Mining Data Streams: Stream Data Model – Sampling Data in the Stream – Filtering Streams – Counting Distance Elements in a Stream – Estimating Moments – Counting Ones in Window – Decaying Windows. Case Studies - Real Time Sentiment Analysis, Stock Market Predictions. Using Graph Analytics for Big Data: Graph Analytics.

5.1 Mining Data Streams

Most of the algorithms described in this book assume that we are mining a database. That is, all our data is available when and if we want it. In this chapter, we shall make another assumption: data arrives in a stream or streams, and if it is not processed immediately or stored, then it is lost forever. Moreover, we shall assume that the data arrives so rapidly that it is not feasible to store it all in active storage (i.e., in a conventional database), and then interact with it at the time of our choosing.

The algorithms for processing streams each involve summarization of the stream in some way. We shall start by considering how to make a useful sample of a stream and how to filter a stream to eliminate most of the “undesirable” elements. We then show how to estimate the number of different elements in a stream using much less storage than would be required if we listed all the elements we have seen.

Another approach to summarizing a stream is to look at only a fixed-length “window” consisting of the last n elements for some (typically large) n . We then query the window as if it were a relation in a database. If there are many streams and/or n is large, we may not be able to store the entire window for every stream, so we need to summarize even the windows. We address the fundamental problem of maintaining an approximate count on the number of 1’s in the window of a bit stream, while using much less space than would be needed to store the entire window itself. This technique generalizes to approximating various kinds of sums.

5.2 The Stream Data Model

Let us begin by discussing the elements of streams and stream processing. We explain the difference between streams and databases and the special problems that arise when dealing with streams. Some typical applications where the stream model applies will be examined.

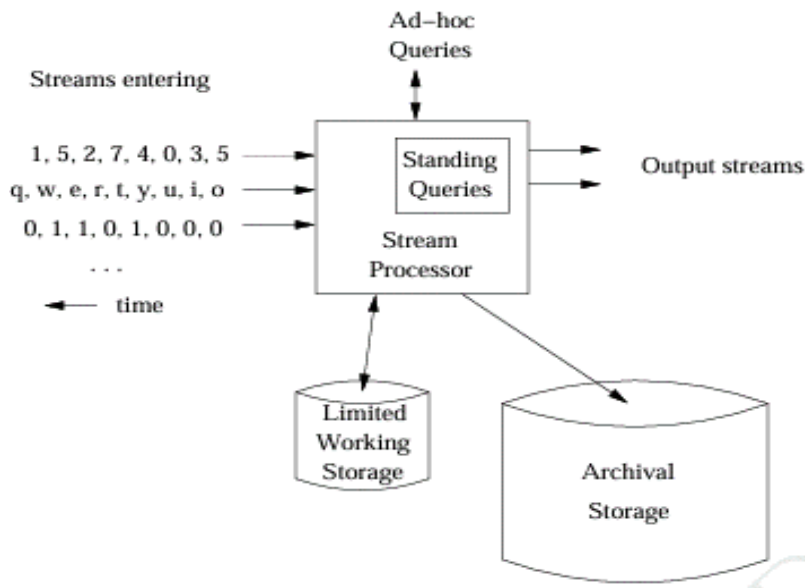


Figure 5.2.1: A data-stream-management system

5.2.1 A Data-Stream-Management System

In analogy to a database-management system, we can view a stream processor as a kind of data-management system, the high-level organization of which is suggested in Fig. 5.2.1. Any number of streams can enter the system. Each stream can provide elements at its own schedule; they need not have the same data rates or data types, and the time between elements of one stream need not be uniform. The fact that the rate of arrival of stream elements is not under the control of the system distinguishes stream processing from the processing of data that goes on within a database-management system. The latter system controls the rate at which data is read from the disk, and therefore never has to worry about data getting lost as it attempts to execute queries.

Streams may be archived in a large archival store, but we assume it is not possible to answer queries from the archival store. It could be examined only under special circumstances using time-consuming retrieval processes. There is also a working store, into which summaries or parts of streams may be placed, and which can be used for answering queries. The working store might be disk, or it might be main memory, depending on how fast we need to process queries. But either way, it is of sufficiently limited capacity that it cannot store all the data from all the streams.

5.2.2 Stream Queries

There are two ways that queries get asked about streams. We show in Fig. 5.2.1 a place within the processor where standing queries are stored. These queries are, in a sense, permanently executing, and produce outputs at appropriate times.

5.2.3 Issues in Stream Processing

Before proceeding to discuss algorithms, let us consider the constraints under which we work when dealing with streams. First, streams often deliver elements very rapidly. We must process elements in real time, or we lose the opportunity to process them at all, without accessing the archival storage.

As our first example of managing streaming data, we shall look at extracting reliable samples from a stream. As with many stream algorithms, the “trick” involves using hashing in a somewhat unusual way.

5.2.4 Obtaining a Representative Sample

The query of Section 5.3.1, like many queries about the statistics of typical users, cannot be answered by taking a sample of each user’s search queries. Thus, we must strive to pick 1/10th of the users, and take all their searches for the sample, while taking none of the searches from other users. If we can store a list of all users, and whether or not they are in the sample, then we could do the following. Each time a search query arrives in the stream, we look up the user to see whether or not they are in the sample. If so, we add this search query to the sample, and if not, then not. However, if we have no record of ever having seen this user before, then we generate a random integer between 0 and 9. If the number is 0, we add this user to our list with value “in,” and if the number is other than 0, we add the user with the value “out.”

That method works as long as we can afford to keep the list of all users and their in/out decision in main memory, because there isn't time to go to disk for every search that arrives. By using a hash function, one can avoid keeping the list of users. That is, we hash each user name to one of ten buckets, 0 through 9. If the user hashes to bucket 0, then accept this search query for the sample, and if not, then not.

Note we do not actually store the user in the bucket; in fact, there is no data in the buckets at all. Effectively, we use the hash function as a random-number generator, with the important property that, when applied to the same user several times, we always get the same "random" number. That is, without storing the in/out decision for any user, we can reconstruct that decision any time a search query by that user arrives.

More generally, we can obtain a sample consisting of any rational fraction a/b of the users by hashing user names to b buckets, 0 through $b - 1$. Add the search query to the sample if the hash value is less than a .

5.2.5 The General Sampling Problem

The running example is typical of the following general problem. Our stream consists of tuples with n components. A subset of the components are the key components, on which the selection of the sample will be based. In our running example, there are three components – user, query, and time – of which only user is in the key. However, we could also take a sample of queries by making query be the key, or even take a sample of user-query pairs by making both those components form the key.

5.2.6 Varying the Sample Size

Often, the sample will grow as more of the stream enters the system. In our running example, we retain all the search queries of the selected 1/10th of the users, forever. As time goes on, more searches for the same users will be accumulated, and new users that are selected for the sample will appear in the stream.

If we have a budget for how many tuples from the stream can be stored as the sample, then the fraction of key values must vary, lowering as time goes on. In order to assure that at all times, the sample consists of all tuples from a subset of the key values, we choose a hash function h from key values to a very large number of values $0, 1, \dots, B - 1$. We maintain a threshold t , which initially can be the largest bucket number, $B - 1$. At all times, the sample consists of those tuples whose key K satisfies $h(K) \leq t$. New tuples from the stream are added to the sample if and only if they satisfy the same condition.

If the number of stored tuples of the sample exceeds the allotted space, we lower t to $t - 1$ and remove from the sample all those tuples whose key K hashes to t . For efficiency, we can lower t by more than 1, and remove the tuples with several of the highest hash values, whenever we need to throw some key values out of the sample. Further efficiency is obtained by maintaining an index on the hash value, so we can find all those tuples whose keys hash to a particular value quickly.

5.3 Filtering Streams

Another common process on streams is selection, or filtering. We want to accept those tuples in the stream that meet a criterion. Accepted tuples are passed to another process as a stream, while other tuples are dropped. If the selection criterion is a property of the tuple that can be calculated (e.g., the first component is less than 10), then the selection is easy to do. The problem becomes harder when the criterion involves lookup for membership in a set. It is especially hard, when that set is too large to store in main memory. In this section, we shall discuss the technique known as “Bloom filtering” as a way to eliminate most of the tuples that do not meet the criterion.

5.3.1 The Bloom Filter

A Bloom filter consists of:

1. An array of n bits, initially all 0's.
2. A collection of hash functions $h_1, h_2, \dots, h_k$. Each hash function maps “key” values to n buckets, corresponding to the n bits of the bit-array.
3. A set S of m key values.

The purpose of the Bloom filter is to allow through all stream elements whose keys are in S , while rejecting most of the stream elements whose keys are not in S .

5.3.2 Analysis of Bloom Filtering

If a key value is in S , then the element will surely pass through the Bloom filter. However, if the key value is not in S , it might still pass. We need to understand how to calculate the probability of a false positive, as a function of n , the bit-array length, m the number of members of S , and k , the number of hash functions.

- The probability that a given dart will not hit a given target is $(x - 1)/x$.
- Using the approximation $(1 - q)^{1/q} = 1/e$ for small q , we conclude that the probability that none of the y darts hit a given target is $e^{-y/x}$

5.5. Counting Distinct Elements in a Stream

In this section we look at a third simple kind of processing we might want to do on a stream. As with the previous examples – sampling and filtering – it is somewhat tricky to do what we want in a reasonable amount of main memory, so we use a variety of hashing and a randomized algorithm to get approximately what we want with little space needed per stream.

5.5.1. The Count-Distinct Problem

Suppose stream elements are chosen from some universal set. We would like to know how many different elements have appeared in the stream, counting either from the beginning of the stream or from some known time in the past.

A similar problem is a Web site like Google that does not require login to issue a search query, and may be able to identify users only by the IP address from which they send the query. There are about 4 billion IP addresses, sequences of four 8-bit bytes will serve as the universal set in this case.

The obvious way to solve the problem is to keep in main memory a list of all the elements seen so far in the stream. Keep them in an efficient search structure such as a hash table or search tree, so one can quickly add new elements and check whether or not the element that just arrived on the stream was already seen. As long as the number of distinct elements is not too great, this structure can fit in main memory and there is little problem obtaining an exact answer to the question how many distinct elements appear in the stream.

However, if the number of distinct elements is too great, or if there are too many streams that need to be processed at once (e.g., Yahoo! wants to count the number of unique users viewing each of its pages in a month), then we cannot store the needed data in main memory. There are several options. We could use more machines, each machine handling only one or several of the streams. We could store most of the data structure in secondary memory and batch stream elements so whenever we brought a disk block to main memory there would be many tests and updates to be performed on the data in that block. Or we could use the strategy to be discussed in this section, where we only estimate the number of distinct elements but use much less memory than the number of distinct elements.

5.5.2 The Flajolet-Martin Algorithm

It is possible to estimate the number of distinct elements by hashing the elements of the universal set to a bit-string that is sufficiently long. The length of the bit-string must be sufficient that there are more possible results of the hash function than there are elements of the universal set. For example, 64 bits is sufficient to hash URL's. We shall pick many different hash functions and hash each element of the stream using these hash

functions. The important property of a hash function is that when applied to the same element, it always produces the same result.

The idea behind the Flajolet-Martin Algorithm is that the more different elements we see in the stream, the more different hash-values we shall see. As we see more different hash-values, it becomes more likely that one of these values will be “unusual.” The particular unusual property we shall exploit is that the value ends in many 0’s, although many other options exist.

Whenever we apply a hash function h to a stream element a , the bit string $h(a)$ will end in some number of 0’s, possibly none. Call this number the tail length for a and h . Let R be the maximum tail length of any a seen so far in the stream. Then we shall use estimate $2R$ for the number of distinct elements seen in the stream.

This estimate makes intuitive sense. The probability that a given stream element a has $h(a)$ ending in at least r 0’s is 2^{-r} . Suppose there are m distinct elements in the stream. Then the probability that none of them has tail length at least r is $(1 - 2^{-r})^m$. This sort of expression should be familiar by now.

We can rewrite it as $((1 - 2^{-r})^{2r})^{m/2r}$. Assuming r is reasonably large, the inner expression is of the form $(1 - e^{-1})^{1/e}$, which is approximately $1/e$. Thus, the probability of not finding a stream element with as many as r 0’s at the end of its hash value is $e^{-m/2r}$. We can conclude:

- If m is much larger than $2r$, then the probability that we shall find a tail of length at least r approaches 1.
- If m is much less than $2r$, then the probability of finding a tail length at least r approaches 0.

We conclude from these two points that the proposed estimate of m , which is $2R$ (recall R is the largest tail length for any stream element) is unlikely to be either much too high or much too low.

5.5.3 Combining Estimates

Unfortunately, there is a trap regarding the strategy for combining the estimates of m , the number of distinct elements that we obtain by using many different hash functions. Our first assumption would be that if we take the average of the values $2R$ that we get

from each hash function, we shall get a value that approaches the true m , the more hash functions we use. However, that is not the case, and the reason has to do with the influence an overestimate has on the average.

Consider a value of r such that $2r$ is much larger than m . There is some probability p that we shall discover r to be the largest number of 0's at the end of the hash value for any of the m stream elements. Then the probability of finding $r + 1$ to be the largest number of 0's instead is at least $p/2$. However, if we do increase by 1 the number of 0's at the end of a hash value, the value of $2R$ doubles. Consequently, the contribution from each possible large R to the expected value of $2R$ grows as R grows, and the expected value of $2R$ is actually infinite.

Another way to combine estimates is to take the median of all estimates. The median is not affected by the occasional outsized value of $2R$, so the worry described above for the average should not carry over to the median. Unfortunately, the median suffers from another defect: it is always a power of 2. Thus, no matter how many hash functions we use, should the correct value of m be between two powers of 2, say 400, then it will be impossible to obtain a close estimate.

There is a solution to the problem, however. We can combine the two methods. First, group the hash functions into small groups, and take their average. Then, take the median of the averages. It is true that an occasional outsized $2R$ will bias some of the groups and make them too large. However, taking the median of group averages will reduce the influence of this effect almost to nothing. Moreover, if the groups themselves are large enough, then the averages can be essentially any number, which enables us to approach the true value m as long as we use enough hash functions. In order to guarantee that any possible average can be obtained, groups should be of size at least a small multiple of $\log_2 m$.

5.5.4 Space Requirements

Observe that as we read the stream it is not necessary to store the elements seen. The only thing we need to keep in main memory is one integer per hash function; this integer records the largest tail length seen so far for that hash function and any stream element. If we are processing only one stream, we could use millions of hash functions, which is far more than we need to get a close estimate. Only if we are trying to process many streams at the same time would main memory constrain the number of hash functions we could associate with any one stream. In practice, the time it takes to compute hash values

for each stream element would be the more significant limitation on the number of hash functions we use.

5.5.5 Exercises for Section 5.5

Exercise 5.5.1: Suppose our stream consists of the integers 3, 1, 4, 1, 5, 9, 2, 6, 5. Our hash functions will all be of the form $h(x) = ax + b \bmod 32$ for some a and b . You should treat the result as a 5-bit binary integer. Determine the tail length for each stream element and the resulting estimate of the number of distinct elements if the hash function is:

(a) $h(x) = 2x + 1 \bmod 32.$

(b) $h(x) = 3x + 7 \bmod 32.$

(c) $h(x) = 4x \bmod 32.$

Exercise 5.5.2: Do you see any problems with the choice of hash functions in Exercise 5.5.1? What advice could you give someone who was going to use a hash function of the form $h(x) = ax + b \bmod 2k$?

5.6 Estimating Moments

In this section we consider a generalization of the problem of counting distinct elements in a stream. The problem, called computing “moments,” involves the distribution of frequencies of different elements in the stream. We shall define moments of all orders and concentrate on computing second moments, from which the general algorithm for all moments is a simple extension.

5.6.1 Definition of Moments

Suppose a stream consists of elements chosen from a universal set. Assume the universal set is ordered so we can speak of the i th element for any i . Let m_i be the number of occurrences of the i th element for any i . Then the k th-order moment (or just k th moment) of the stream is the sum over all i of $(m_i)^k$.

Example 5.6: The 0th moment is the sum of 1 for each m_i that is greater than 0. That is, the 0th moment is a count of the number of distinct elements in the stream. We can use the method of Section 4.4 to estimate the 0th moment of a stream.

The 1st moment is the sum of the m_i 's, which must be the length of the stream. Thus, first moments are especially easy to compute; just count the length of the stream seen so far.

The second moment is the sum of the squares of the m_i 's. It is sometimes called the surprise number, since it measures how uneven the distribution of elements in the stream is. To see the distinction, suppose we have a stream of length 100, in which eleven different elements appear. The most even distribution of these eleven elements would have one appearing 10 times and the other ten appearing 9 times each. In this case, the surprise number is $10^2 + 10 \times 9^2 = 910$. At the other extreme, one of the eleven elements could appear 90 times and the other ten appear 1 time each. Then, the surprise number would be $90^2 + 10 \times 1^2 = 8110$.

As in Section 5.5, there is no problem computing moments of any order if we can afford to keep in main memory a count for each element that appears in the stream. However, also as in that section, if we cannot afford to use that much memory, then we need to estimate the k th moment by keeping a limited number of values in main memory and computing an estimate from these values. For the case of distinct elements, each of these values were counts of the longest tail produced by a single hash function. We shall see another form of value that is useful for second and higher moments.

5.6.2 The Alon-Matias-Szegedy Algorithm for Second Moments

For now, let us assume that a stream has a particular length n . We shall show how to deal with growing streams in the next section. Suppose we do not have enough space to count all the m_i 's for all the elements of the stream. We can still estimate the second moment of the stream using a limited amount of space; the more space we use, the more accurate the estimate will be. We compute some number of variables. For each variable X , we store:

1. A particular element of the universal set, which we refer to as $X.\text{element}$, and
2. An integer $X.\text{value}$, which is the value of the variable. To determine the value of a variable X , we choose a position in the stream between 1 and n , uniformly and at random. Set $X.\text{element}$ to be the element found there, and initialize $X.\text{value}$ to 1. As we read the stream, add 1 to $X.\text{value}$ each time we encounter another occurrence of $X.\text{element}$.

Example 5.7: Suppose the stream is a, b, c, b, d, a, c, d, a, b, d, c, a, a, b. The length of the stream is $n = 15$. Since a appears 5 times, b appears 4 times, and c and d appear three times each, the second moment for the stream is $52 + 42 + 32 + 32 = 59$. Suppose we keep three variables, X1, X2, and X3. Also, assume that at “random” we pick the 3rd, 8th, and 13th positions to define these three variables.

When we reach position 3, we find element c, so we set $X1.element = c$ and $X1.value = 1$. Position 4 holds b, so we do not change X1. Likewise, nothing happens at positions 5 or 6. At position 7, we see c again, so we set $X1.value = 2$.

At position 8 we find d, and so set $X2.element = d$ and $X2.value = 1$. Positions 9 and 10 hold a and b, so they do not affect X1 or X2. Position 11 holds d so we set $X2.value = 2$, and position 12 holds c so we set $X1.value = 3$. At position 13, we find element a, and so set $X3.element = a$ and $X3.value = 1$. Then, at position 14 we see another a and so set $X3.value = 2$. Position 15, with element b does not affect any of the variables, so we are done, with final values $X1.value = 3$ and $X2.value = X3.value = 2$.

We can derive an estimate of the second moment from any variable X. This estimate is $n(2X.value - 1)$.

Example 5.8: Consider the three variables from Example 4.7. From X1 we derive the estimate $n(2 X1.value - 1) = 15 \times (2 \times 3 - 1) = 75$. The other two variables, X2 and X3, each have value 2 at the end, so their estimates are $15 \times (2 \times 2 - 1) = 45$. Recall that the true value of the second moment for this stream is 59. On the other hand, the average of the three estimates is 55, a fairly close approximation.

5.7 Counting Ones in a Window

We now turn our attention to counting problems for streams. Suppose we have a window of length N on a binary stream. We want at all times to be able to answer queries of the form “how many 1’s are there in the last k bits?” for any $k \leq N$. As in previous sections, we focus on the situation where we cannot afford to store the entire window. After showing an approximate algorithm for the binary case, we discuss how this idea can be extended to summing numbers.

5.7.1 The Cost of Exact Counts

To begin, suppose we want to be able to count exactly the number of 1's in the last k bits for any $k \leq N$. Then we claim it is necessary to store all N bits of the window, as any representation that used fewer than N bits could not work. In proof, suppose we have a representation that uses fewer than N bits to represent the N bits in the window. Since there are 2^N sequences of N bits, but fewer than 2^N representations, there must be two different bit strings w and x that have the same representation. Since $w \neq x$, they must differ in at least one bit. Let the last $k - 1$ bits of w and x agree, but let them differ on the k th bit from the right end.

Example 5.10: If $w = 0101$ and $x = 1010$, then $k = 1$, since scanning from the right, they first disagree at position 1. If $w = 1001$ and $x = 0101$, then $k = 3$, because they first disagree at the third position from the right.

Suppose the data representing the contents of the window is whatever sequence of bits represents both w and x . Ask the query “how many 1's are in the last k bits?” The query-answering algorithm will produce the same answer, whether the window contains w or x , because the algorithm can only see their representation. But the correct answers are surely different for these two bit-strings. Thus, we have proved that we must use at least N bits to answer queries about the last k bits for any k .

In fact, we need N bits, even if the only query we can ask is “how many 1's are in the entire window of length N ?” The argument is similar to that used above. Suppose we use fewer than N bits to represent the window, and therefore we can find w , x , and k as above. It might be that w and x have the same number of 1's, as they did in both cases of Example 5.10. However, if we follow the current window by any $N - k$ bits, we will have a situation where the true window contents resulting from w and x are identical except for the leftmost bit, and therefore, their counts of 1's are unequal. However, since the representations of w and x are the same, the representation of the window must still be the same if we feed the same bit sequence to these representations. Thus, we can force the answer to the query “how many 1's in the window?” to be incorrect for one of the two possible window contents.

5.7.2 The Datar-Gionis-Indyk-Motwani Algorithm

We shall present the simplest case of an algorithm called DGIM. This version of the algorithm uses $O(\log^2 N)$ bits to represent a window of N bits, and allows us to estimate

the number of 1's in the window with an error of no more than 50%. Later, we shall discuss an improvement of the method that limits the error to any fraction $\epsilon > 0$, and still uses only $O(\log^2 N)$ bits (although with a constant factor that grows as ϵ shrinks).

To begin, each bit of the stream has a timestamp, the position in which it arrives. The first bit has timestamp 1, the second has timestamp 2, and so on. Since we only need to distinguish positions within the window of length N , we shall represent timestamps modulo N , so they can be represented by $\log_2 N$ bits. If we also store the total number of bits ever seen in the stream (i.e., the most recent timestamp) modulo N , then we can determine from a timestamp modulo N where in the current window the bit with that timestamp is.

We divide the window into buckets, consisting of:

1. The timestamp of its right (most recent) end.
2. The number of 1's in the bucket. This number must be a power of 2, and we refer to the number of 1's as the size of the bucket.

To represent a bucket, we need $\log_2 N$ bits to represent the timestamp (modulo N) of its right end. To represent the number of 1's we only need $\log_2 \log_2 N$ bits. The reason is that we know this number i is a power of 2, say 2^j , so we can represent i by coding j in binary. Since j is at most $\log_2 N$, it requires $\log_2 \log_2 N$ bits. Thus, $O(\log N)$ bits suffice to represent a bucket.

There are six rules that must be followed when representing a stream by buckets.

- The right end of a bucket is always a position with a 1.
- Every position with a 1 is in some bucket.
- No position is in more than one bucket.
- There are one or two buckets of any given size, up to some maximum size.
- All sizes must be a power of 2.
- Buckets cannot decrease in size as we move to the left (back in time).

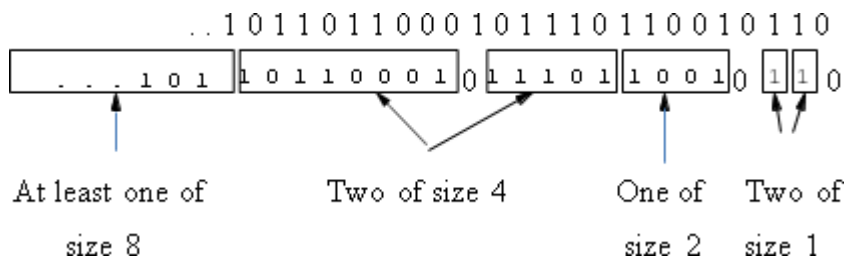


Figure 5.7.2.1 A bit-stream divided into buckets following the DGIM rules

Example 5.11: Figure 5.7.2.1. shows a bit stream divided into buckets in a way that satisfies the DGIM rules. At the right (most recent) end we see two buckets of size 1. To its left we see one bucket of size 2. Note that this bucket covers four positions, but only two of them are 1. Proceeding left, we see two buckets of size 4, and we suggest that a bucket of size 8 exists further left.

Notice that it is OK for some 0's to lie between buckets. Also, observe from Fig. 5.7.2.1 that the buckets do not overlap; there are one or two of each size up to the largest size, and sizes only increase moving left.

In the next sections, we shall explain the following about the DGIM algo- rithm:

1. Why the number of buckets representing a window must be small.
2. How to estimate the number of 1's in the last k bits for any k , with an error no greater than 50%.

How to maintain the DGIM conditions as new bits enter the stream.

5.7.3 Storage Requirements for the DGIM Algorithm

We observed that each bucket can be represented by $O(\log N)$ bits. If the window has length N , then there are no more than N 1's, surely. Suppose the largest bucket is of size 2^j . Then j cannot exceed $\log_2 N$, or else there are more 1's in this bucket than there are 1's in the entire window. Thus, there are at most two buckets of all sizes from $\log_2 N$ down to 1, and no buckets of larger sizes.

We conclude that there are $O(\log N)$ buckets. Since each bucket can be represented in $O(\log N)$ bits, the total space required for all the buckets representing a window of size N is $O(\log^2 N)$.

5.7.4 Query Answering in the DGIM Algorithm

Suppose we are asked how many 1's there are in the last k bits of the window, for some $1 \leq k \leq N$. Find the bucket b with the earliest timestamp that includes at least some of the k most recent bits. Estimate the number of 1's to be the sum of the sizes of all the buckets to the right (more recent) than bucket b , plus half the size of b itself.

Example 5.12: Suppose the stream is that of Fig. 4.2, and $k = 10$. Then the query asks for the number of 1's in the ten rightmost bits, which happen to be 0110010110. Let the current timestamp (time of the rightmost bit) be t . Then the two buckets with one 1, having timestamps $t - 1$ and $t - 2$ are completely included in the answer. The bucket of size 2, with timestamp $t - 4$, is also completely included. However, the rightmost bucket of size 4, with timestamp $t - 8$ is only partly included. We know it is the last bucket to contribute to the answer, because the next bucket to its left has timestamp less than $t - 9$ and thus is completely out of the window. On the other hand, we know the buckets to its right are completely inside the range of the query because of the existence of a bucket to their left with timestamp $t - 9$ or greater.

Our estimate of the number of 1's in the last ten positions is thus 6. This number is the two buckets of size 1, the bucket of size 2, and half the bucket of size 4 that is partially within range. Of course the correct answer is 5.

Suppose the above estimate of the answer to a query involves a bucket b of size 2^j that is partially within the range of the query. Let us consider how far from the correct answer c our estimate could be. There are two cases: the estimate could be larger or smaller than c .

Case 1: The estimate is less than c . In the worst case, all the 1's of b are actually within the range of the query, so the estimate misses half bucket b , or 2^{j-1} 1's. But in this case, c is at least 2^j ; in fact it is at least $2^{j+1} - 1$, since there is at least one bucket of each of the sizes $2^{j-1}, 2^{j-2}, \dots, 1$. We conclude that our estimate is at least 50% of c .

Case 2: The estimate is greater than c . In the worst case, only the rightmost bit of bucket b is within range, and there is only one bucket of each of the sizes smaller than b . Then c

$= 1 + 2^{j-1} + 2^{j-2} + \dots + 1 = 2^j$ and the estimate we give is $2^{j-1} + 2^{j-1} + 2^{j-2} + \dots + 1 = 2^j + 2^{j-1} - 1$. We see that the estimate is no more than 50% greater than c .

5.7.5 Maintaining the DGIM Conditions

Suppose we have a window of length N properly represented by buckets that satisfy the DGIM conditions. When a new bit comes in, we may need to modify the buckets, so they continue to represent the window and continue to satisfy the DGIM conditions. First, whenever a new bit enters:

- Check the leftmost (earliest) bucket. If its timestamp has now reached the current timestamp minus N , then this bucket no longer has any of its 1's in the window. Therefore, drop it from the list of buckets.

Now, we must consider whether the new bit is 0 or 1. If it is 0, then no further change to the buckets is needed. If the new bit is a 1, however, we may need to make several changes. First:

- Create a new bucket with the current timestamp and size 1.

If there was only one bucket of size 1, then nothing more needs to be done. However, if there are now three buckets of size 1, that is one too many. We fix this problem by combining the leftmost (earliest) two buckets of size 1.

- To combine any two adjacent buckets of the same size, replace them by one bucket of twice the size. The timestamp of the new bucket is the timestamp of the rightmost (later in time) of the two buckets.

Combining two buckets of size 1 may create a third bucket of size 2. If so, we combine the leftmost two buckets of size 2 into a bucket of size 4. That, in turn, may create a third bucket of size 4, and if so we combine the leftmost two into a bucket of size 8. This process may ripple through the bucket sizes, but there are at most $\log_2 N$ different sizes, and the combination of two adjacent buckets of the same size only requires constant time. As a result, any new bit can be processed in $O(\log N)$ time.

Example 4.13: Suppose we start with the buckets of Fig. 5.7.5.1 and a 1 enters. First, the leftmost bucket evidently has not fallen out of the window, so we do not drop any buckets. We create a new bucket of size 1 with the current timestamp, say t . There are now three buckets of size 1, so we combine the leftmost two. They are replaced with a single bucket

of size 2. Its timestamp is $t-2$, the timestamp of the bucket on the right (i.e., the rightmost bucket that actually appears in Fig. 5.7.2.1).

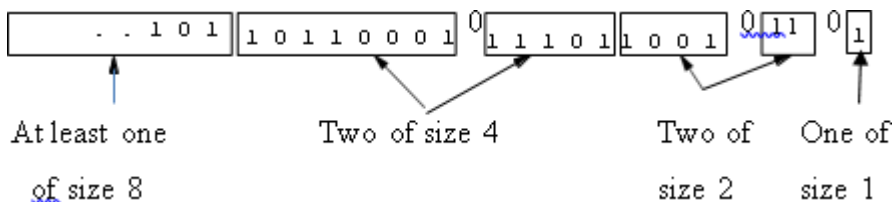


Figure 5.7.5.1: Modified buckets after a new 1 arrives in the stream

There are now two buckets of size 2, but that is allowed by the DGIM rules. Thus, the final sequence of buckets after the addition of the 1 is as shown in Fig. 5.7.5.1.

5.8 Decaying Windows

We have assumed that a sliding window held a certain tail of the stream, either the most recent N elements for fixed N , or all the elements that arrived after some time in the past. Sometimes we do not want to make a sharp distinction between recent elements and those in the distant past, but want to weight the recent elements more heavily. In this section, we consider “exponentially decaying windows,” and an application where they are quite useful: finding the most common “recent” elements.

5.8.1. The Problem of Most-Common Elements

Suppose we have a stream whose elements are the movie tickets purchased all over the world, with the name of the movie as part of the element. We want to keep a summary of the stream that is the most popular movies “currently.” While the notion of “currently” is imprecise, intuitively, we want to discount the popularity of a movie like Star Wars—Episode 4, which sold many tickets, but most of these were sold decades ago. On the other hand, a movie that sold n tickets in each of the last 10 weeks is probably more popular than a movie that sold $2n$ tickets last week but nothing in previous weeks.

One solution would be to imagine a bit stream for each movie. The i th bit has value 1 if the i th ticket is for that movie, and 0 otherwise. Pick a window size N , which is the number of most recent tickets that would be considered in evaluating popularity. Then, use the method of Section 4.6 to estimate the number of tickets for each movie, and rank

movies by their estimated counts. This technique might work for movies, because there are only thousands of movies, but it would fail if we were instead recording the popularity of items sold at Amazon, or the rate at which different Twitter-users tweet, because there are too many Amazon products and too many tweeters. Further, it only offers approximate answers.

5.8.2 Definition of the Decaying Window

An alternative approach is to redefine the question so that we are not asking for a count of 1's in a window. Rather, let us compute a smooth aggregation of all the 1's ever seen in the stream, with decaying weights, so the further back in the stream, the less weight is given. Formally, let a stream currently consist of the elements $a_1, a_2, \dots, a_t$, where a_1 is the first element to arrive and a_t is the current element. Let c be a small constant, such as 10^{-6} or 10^{-9} . Define the exponentially decaying window for this stream to be the sum

$$\sum_{i=0}^{t-1} a_{t-i}(1-c)^i$$

The effect of this definition is to spread out the weights of the stream elements as far back in time as the stream goes. In contrast, a fixed window with the same sum of the weights, $1/c$, would put equal weight 1 on each of the most recent $1/c$ elements to arrive and weight 0 on all previous elements. The distinction is suggested by Fig. 5.8.1.1.

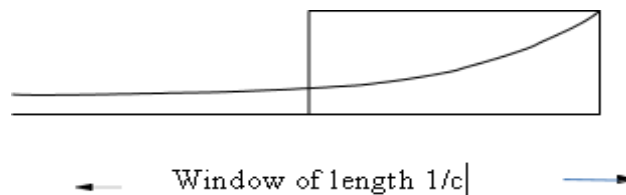


Figure 5.8.1.1: A decaying window and a fixed-length window of equal weight

It is much easier to adjust the sum in an exponentially decaying window than in a sliding window of fixed length. In the sliding window, we have to worry about the element that falls out of the window each time a new element arrives.

That forces us to keep the exact elements along with the sum, or to use an approximation scheme such as DGIM. However, when a new element a_{t+1} arrives at the stream input, all we need to do is:

- Multiply the current sum by $1 - c$.
- Add a_{t+1} .

The reason this method works is that each of the previous elements has now moved one position further from the current element, so its weight is multiplied by $1 - c$. Further, the weight on the current element is $(1 - c)^0 = 1$, so adding a_{t+1} is the correct way to include the new element's contribution.

5.9 Introduction to Real-Time Sentiment Analysis

Sentiment analysis refers to the computational study of opinions, attitudes, emotions, and subjectivity expressed in text. In the real-time context, the focus is on capturing, processing, and analyzing data as it is generated. Unlike offline models that operate on historical datasets, real-time sentiment systems must handle continuous data streams and deliver insights with minimal latency.

Key applications include:

- Monitoring financial markets (stock trends, investor sentiment)
- Brand management and customer feedback tracking.
- Political analysis, especially during live debates or elections
- Crisis detection (natural disasters, product recalls, pandemics)

5.9.1. Characteristics of Real-Time Sentiment Analysis

- Streaming Data: Data arrives continuously (e.g., tweets per second).
- Low Latency Requirements: Insights must be generated within seconds/minutes.
- High Volume: Millions of posts per day across platforms.
- Dynamic Language: Slang, sarcasm, emojis, and domain-specific jargon.

- Noise Handling: Bots, spam, irrelevant content.

5.9.2. Workflow for Real-Time Sentiment Analysis

➤ Data Acquisition

- APIs: Twitter Streaming API, Reddit Firehose, StockTwits, news APIs (Bloomberg, Reuters).
- Web Scraping: Collecting customer reviews, blog posts, or forums.
- Sensor-based Feeds: In IoT/healthcare, wearable device feedback streams.

➤ Preprocessing

- Tokenization, stemming/lemmatization.
- Removing URLs, mentions, hashtags, stop words.
- Normalizing emojis and slang (e.g., “🔥” → “exciting” / “trending”).

➤ Sentiment Detection Techniques

- Lexicon-based: Predefined dictionaries like VADER, SentiWordNet.
- Machine Learning: Logistic Regression, SVM, Naïve Bayes.
- Deep Learning: LSTMs, CNNs, Transformers (BERT, RoBERTa, FinBERT).
- Hybrid Models: Combining lexicon + ML to improve robustness.

➤ Real-Time Processing Frameworks

- Apache Kafka: Message streaming platform.
- Apache Spark Streaming / Flink: Distributed real-time data analytics.
- NoSQL Databases: MongoDB, Cassandra for rapid ingestion.
- Visualization Dashboards: Power BI, Grafana, Tableau.

5.9.3. Real-Time Sentiment Analysis: Case Studies

➤ Case Study 1: Twitter Sentiment During Elections

Objective: To measure public opinion shifts in real-time.

Method:

- Tweets collected during debates or campaign rallies.
- NLP models classified polarity (positive/negative/neutral).
- Hashtag co-occurrence graphs identified trending topics.

Outcome:

- Real-time dashboards visualized spikes in sentiment aligned with candidate speeches.
- Political strategists used results to adapt campaign messaging instantly.

➤ Case Study 2: Crisis Detection (COVID-19 Pandemic)

Objective: Identify early warning signals from public sentiment.

Method:

- Collected tweets mentioning symptoms, lockdowns, or vaccines.
- Sentiment models categorized emotions (fear, anger, hope, trust).
- Geo-tagging localized areas of rising concern.

Outcome:

- Real-time alerts generated for health agencies.
- Helped track misinformation spread and gauge vaccine hesitancy.

➤ Case Study 3: Brand Reputation in E-Commerce

Objective: Track real-time consumer response to product launches.

Method:

- Streaming reviews and social mentions were ingested via Kafka.
- VADER + BERT sentiment models scored each mention.
- Negative feedback auto-flagged for customer service response.

Outcome:

- Companies resolved issues faster, reducing customer churn.
- Detected viral complaints (e.g., defective batches) before escalation.

➤ Case Study 4: Real-Time Financial Market Sentiment

Objective: Predict stock price movements using investor mood.

Method:

- Data from Twitter + StockTwits + Bloomberg news headlines.
- Sentiment polarity and intensity scores aggregated.
- Combined with technical indicators in an LSTM model.

Outcome:

- Improved short-term prediction accuracy (intraday trading).
- Detected “fear” spikes during negative news, correlating with sudden stock drops.

5.9.4. Challenges and Limitations

- Sarcasm/Irony Detection: Hard to capture with traditional models.
- Language Diversity: Multilingual, code-mixed text streams.
- Concept Drift: Meanings of words change over time (e.g., “sick” → “cool”).
- Bot/Spam Filtering: Automated posts skew sentiment.

- Ethical Issues: Privacy, manipulation, and misinformation risks.

5.10. Case Studies – Stock Market Predictions

The stock market has always been a dynamic and uncertain environment, influenced by a wide range of factors such as company performance, investor psychology, macroeconomic policies, global events, and unexpected crises. Traditional methods of stock prediction relied on fundamental analysis (company earnings, balance sheets, industry trends) and technical analysis (historical prices, volume patterns, and indicators). While these techniques are still widely used, the rise of big data analytics, machine learning, and artificial intelligence has revolutionized the way predictions are made. By leveraging computational models, vast datasets, and real-time signals from news and social media, researchers and practitioners have demonstrated that stock market forecasts can be significantly improved. This chapter discusses notable case studies in stock market prediction, highlighting methodologies, outcomes, and lessons learned.

5.10.1 Data Sources for Prediction

Case studies in stock market prediction commonly draw on multiple sources of data:

- Historical Price Data: Closing price, open-high-low-close (OHLC), and trading volumes.
- Technical Indicators: Moving averages, RSI, MACD, Bollinger Bands.
- Fundamental Data: Earnings reports, P/E ratios, balance sheets, dividends.
- Macroeconomic Factors: Interest rates, inflation, exchange rates.
- News and Social Media Sentiment: Breaking financial headlines, analyst opinions, Twitter/X, and StockTwits.
- Event-Driven Data: Mergers, product launches, regulatory announcements.
- By integrating these heterogeneous data streams, modern predictive systems aim to capture both quantitative signals and qualitative sentiment that shape market behavior.

5.10.2. Methodologies in Stock Market Prediction

The following methods have been central in case studies:

- Statistical Models: ARIMA, GARCH, and regression models to capture time-series dynamics.
- Machine Learning Models: Support Vector Machines (SVM), Random Forests, Gradient Boosted Trees for classification of price trends.
- Deep Learning Architectures: LSTM, GRU, CNN, and transformer-based models that handle sequential and contextual information.
- Sentiment Analysis Integration: Combining price history with sentiment scores derived from news, blogs, and social media posts.
- Graph Analytics and GNNs: Modeling financial entities as nodes and relationships as edges to detect hidden correlations, influence spread, and systemic risks.

5.10.3. Case Studies in Stock Market Predictions

➤ Case Study 1: Twitter Sentiment and FTSE 100 Index

A notable study examined the predictive power of Twitter sentiment on the FTSE 100 index. Researchers collected millions of tweets over several weeks, applied sentiment classification (positive, neutral, negative), and aggregated results into daily investor sentiment scores. These scores were then combined with historical price data to train machine learning models. The results showed a measurable correlation: spikes in positive sentiment often preceded short-term upward trends, while negative sentiment predicted drops. The case proved that social mood acts as a real-time proxy for market behavior.

➤ Case Study 2: AZFinText – Using Breaking News for Predictions

AZFinText is one of the earliest and most influential systems that applied text mining to stock prediction. It collected breaking financial news, applied natural language processing (NLP) to extract key phrases, and then mapped these to expected stock movements within a 20-minute horizon. Unlike traditional models that rely solely on historical prices, AZFinText showed that qualitative information (news narratives, analyst statements) has

direct quantitative effects on intraday stock performance. This study highlighted the importance of event-driven prediction.

➤ Case Study 3: Deep Learning on StockTwits Data

StockTwits, a specialized social platform for traders, became the focus of another case study where deep learning models (CNNs and LSTMs) were trained on millions of trader posts labeled with bullish/bearish indicators. The models learned patterns of collective investor behavior and achieved higher accuracy than standard regression models. Importantly, the inclusion of temporal sentiment features improved predictions of intraday price movements, demonstrating that crowdsourced financial opinions can complement numerical data.

➤ Case Study 4: Graph Analytics for Market Predictions

In more recent research, stock prediction has been enhanced using graph analytics and graph neural networks (GNNs). In these models, companies, investors, and news sources are represented as nodes, while relationships such as co-mentions, co-movements, or ownership structures are represented as edges. For example, if two companies are frequently mentioned together in news reports, their stocks may show correlated movements. GNN-based case studies revealed that incorporating relational information improved predictive accuracy over standalone models, particularly during market shocks when contagion effects ripple across sectors.

➤ Case Study 5: Hybrid Models Combining Technical and Sentiment Features

Another successful approach is hybrid modeling, where technical indicators (moving averages, volatility measures) are combined with sentiment scores from news headlines. A case study applying LSTM networks showed that hybrid models outperformed purely technical models in predicting stock direction. This indicates that while price history captures market mechanics, sentiment analysis captures investor psychology, and both are crucial for robust predictions.

5.10.4. Evaluation Metrics

Case studies in stock prediction use a range of metrics to evaluate effectiveness:

- Accuracy and F1-score: Correct prediction of upward or downward movements.

- Mean Absolute Error (MAE) / Root Mean Square Error (RMSE): Precision of predicted prices.
- Direction Accuracy: Ability to predict the correct trend (up/down).
- Economic Metrics: Profitability, Sharpe ratio, risk-adjusted returns when applied in trading strategies.

5.10.5. Conclusion

The study of stock market predictions through real-world case studies illustrates the transition from traditional statistical models to AI-driven, data-rich systems that combine multiple sources of information. Whether it is Twitter sentiment predicting market mood, breaking news triggering intraday price shifts, or graph analytics uncovering hidden correlations, these approaches show that the future of stock forecasting lies in hybrid, adaptive, and intelligent systems. Although challenges such as noise, volatility, and ethical issues remain, the continuous evolution of computational techniques ensures that stock market prediction will remain an active and impactful field of research and practice.

5.11. Using Graph Analytics for Big Data – Graph Analytics

In the era of big data, the volume, velocity, and variety of information generated daily have made traditional analysis techniques insufficient for uncovering complex relationships. While relational databases and standard statistical methods excel at handling structured data, they often fail to capture the interconnected nature of modern datasets, where entities are linked through dynamic, multi-dimensional relationships. Graph analytics has emerged as a powerful paradigm to address this challenge,

representing data as nodes (entities) and edges (relationships). By focusing on relationships rather than isolated attributes, graph analytics enables the discovery of patterns, communities, anomalies, and hidden structures that would otherwise remain invisible. This approach is increasingly applied across domains such as finance, healthcare, cybersecurity, transportation, e-commerce, and social networks, making it an essential tool in big data analytics.

5.11.1. Fundamentals of Graph Analytics

At its core, graph analytics leverages graph theory to analyze relationships between entities. A graph can be:

- Undirected or Directed: showing bidirectional or one-way relationships.
- Weighted or Unweighted: where edges carry numerical values such as transaction amounts or distances.
- Static or Dynamic: representing relationships at a fixed point or evolving over time.

Key graph properties include degree, path length, connectivity, clustering coefficient, and centrality, all of which are used to measure influence, importance, or cohesion within a network.

5.11.2. Techniques in Graph Analytics

Graph analytics encompasses a wide variety of techniques, each suited to specific problem domains:

Centrality Analysis – Identifies the most influential nodes in a graph (e.g., detecting key influencers in a social network or important hubs in transportation).

Community Detection – Groups nodes into clusters or communities, useful for customer segmentation, fraud rings, or biological network analysis.

Shortest Path and Traversal Algorithms – Applied in logistics, routing, and recommendation systems to find efficient paths or connections.

Similarity and Link Prediction – Predicts future connections between nodes, widely used in social media (“People You May Know”) or e-commerce recommendations.

PageRank and Influence Scoring – Evaluates the relative importance of nodes, initially developed for search engine ranking but now applied in multiple domains.

Graph Embeddings – Converts graph nodes and edges into low-dimensional vectors, enabling integration with machine learning models for classification, clustering, and prediction tasks.

5.11.3. Big Data Frameworks for Graph Processing

The sheer scale of modern datasets necessitates distributed graph processing systems. Popular frameworks include:

- Neo4j – A native graph database optimized for storing and querying graph structures.
- Apache Giraph – A scalable graph processing platform based on Hadoop.
- Apache Spark GraphX – Provides APIs for graph-parallel computation integrated into Spark’s big data ecosystem.
- TigerGraph – A high-performance graph database capable of real-time analytics on massive datasets.
- NetworkX – A Python library widely used for small to medium-scale graph analytics and academic research.
- These platforms allow processing of billions of nodes and edges, enabling analytics at the scale of global social networks or financial transaction graphs.

5.11.4. Case Studies in Graph Analytics

➤ Social Network Analysis

Graph analytics has been extensively used in studying social media networks to identify influencers, track the spread of information, and detect misinformation campaigns. By applying community detection and centrality measures, researchers have identified key accounts responsible for viral content. This has been especially useful in election monitoring and crisis management.

➤ Fraud Detection in Banking and Finance

Financial institutions leverage graph analytics to detect fraudulent activities by examining transaction networks. Instead of analyzing accounts individually, graph-based models reveal suspicious clusters, such as fraud rings where multiple accounts are connected by abnormal transaction patterns. Link prediction algorithms have been employed to detect potential collusion before it escalates.

➤ Healthcare and Bioinformatics

In healthcare, graph analytics models protein–protein interactions, disease–gene relationships, and patient similarity networks. For example, detecting disease progression patterns or predicting drug–target interactions relies on graph embeddings and graph neural networks (GNNs).

➤ Supply Chain and Logistics

Supply chains are naturally graph-structured, consisting of suppliers, manufacturers, distributors, and retailers. Graph analytics helps in identifying bottlenecks, optimizing logistics routes through shortest path algorithms, and predicting cascading failures when a supplier is disrupted.

➤ Cybersecurity

Graph-based anomaly detection has been applied in cybersecurity to uncover suspicious login attempts, malicious IP connections, or unusual communication patterns within networks. Real-time streaming graph analytics enables security teams to detect and mitigate threats before they spread widely.

5.11.5. Integration of Graph Analytics with AI and Machine Learning

Recent advances combine graph analytics with machine learning to enhance predictive power. Graph Neural Networks (GNNs) are particularly transformative, as they integrate both node attributes (features such as sentiment, financial indicators) and graph structure (relationships between entities). Applications include:

- **Stock Market Predictions:** Modeling companies, news events, and investors as graphs to capture correlations.

- **Recommender Systems:** Using graph embeddings to improve personalized recommendations on platforms like Amazon or Netflix.
- **Risk Propagation Analysis:** Tracking systemic risks across interconnected financial institutions.

5.11.6. Challenges in Graph Analytics

Despite its potential, graph analytics faces several challenges:

- **Scalability:** Processing graphs with billions of nodes requires high computational resources.
- **Dynamic Graphs:** Continuously evolving graphs (e.g., social media) require real-time updating.
- **Data Quality:** Noise, incomplete data, and unreliable connections can distort results.
- **Interpretability:** Explaining results from advanced models like GNNs remains difficult.
- **Integration with Existing Systems:** Many organizations rely on relational databases, making transition to graph-based architectures complex.

5.11.7. Conclusion

Graph analytics provides a powerful lens for analyzing the interconnected world of big data. By focusing on relationships rather than isolated attributes, it uncovers patterns and insights that are otherwise invisible to traditional analytics. From detecting fraud in finance to monitoring misinformation in social networks, from optimizing supply chains to advancing drug discovery, graph analytics is reshaping how organizations leverage data. As data continues to grow in complexity, graph analytics—combined with advanced AI models—will play a central role in driving intelligent, scalable, and relationship-aware decision-making in the digital age.